

Full Stack Interview Bank

The questions that come up most often in full stack interviews, gathered from published 2026 interview guides and grouped by topic. Each one has a short model answer. Search, filter by level, mark what you know, or run a random practice round.

383 questions 0 marked known 383 shown

HTML

Semantics, accessibility, forms, performance hints. Usually the warm-up round.

What is semantic HTML and why does it matter?

JUNIOR

Using elements that describe meaning (`<header>` , `<nav>` , `<main>` , `<article>` , `<button>`) instead of generic `<div>` s.

- Screen readers use it to navigate by landmarks and headings.
- Search engines understand page structure better.
- Native elements bring keyboard and focus behaviour for free.

Difference between block, inline and inline-block elements?

JUNIOR

- **Block** (`div` , `p`): starts on a new line, takes full width, respects width/height.
- **Inline** (`span` , `a`): flows in text, ignores width/height and vertical margins.
- **Inline-block**: flows inline but respects width, height and padding.

What do `async` and `defer` do on a script tag?

JUNIOR

- No attribute: HTML parsing stops while the script downloads and runs.
- `async` : downloads in parallel, runs as soon as it arrives; order not guaranteed. Good for analytics.
- `defer` : downloads in parallel, runs after parsing, in document order. Good for app scripts.

What is the purpose of the `<!DOCTYPE html>` and the viewport meta tag?

JUNIOR

DOCTYPE puts the browser in standards mode instead of quirks mode. `<meta name="viewport" content="width=device-width, initial-scale=1">` tells mobile browsers to use the device width, so responsive CSS works.

How do you make a form accessible?

MID

- Every input has a `<label for>` (placeholder is not a label).
- Group related controls with `<fieldset>` and `<legend>`.
- Use correct `type` (email, tel) and `autocomplete` (e.g. `current-password`, `new-password`).
- Link errors with `aria-describedby` and announce with `aria-live`.
- Everything reachable and operable by keyboard.

When should you use ARIA?

MID

Only when no native element does the job. First rule of ARIA: prefer a native `<button>` over `<div role="button">`. Use ARIA for custom widgets (tabs, comboboxes), live regions and states like `aria-expanded`. Wrong ARIA is worse than none.

localStorage vs sessionStorage vs cookies?

MID

- **localStorage**: ~5 MB, persists until cleared, JS-readable, never sent to server.
- **sessionStorage**: same, but scoped to the tab and cleared when it closes.
- **Cookies**: ~4 KB, sent with every request, can be `HttpOnly`, `Secure`, `SameSite`. Use for session/refresh tokens.

Follow-up: never store auth tokens in localStorage; any XSS can read them.

What are preload, prefetch and preconnect ?

MID

- `preload` : fetch a resource needed for this page now (hero image, font).
- `prefetch` : low-priority fetch for a likely next navigation.
- `preconnect` : open DNS/TCP/TLS to an origin early.

How would you improve Core Web Vitals (LCP, INP, CLS)?

SENIOR

- **LCP**: preload hero image, serve from CDN, SSR/SSG the above-the-fold content, compress images (AVIF/WebP).
- **INP**: break long tasks, reduce JS, defer non-critical scripts, use `startTransition`.
- **CLS**: set width/height or `aspect-ratio` on media, reserve space for ads/embeds, use `font-display` with matched fallbacks.

CSS

Layout (flex, grid), cascade and specificity, responsive design, modern features.

Explain the CSS box model.

JUNIOR

Every element is content + padding + border + margin. With `box-sizing: content-box` (default) width covers content only; with `border-box` width includes padding and border, which is why most resets set `*{box-sizing:border-box}`.

How is specificity calculated?

JUNIOR

Compared as a tuple (inline, IDs, classes/attributes/pseudo-classes, elements). `#nav .item a` = (0,1,1,1). Higher wins; ties go to the later rule. `!important` overrides all normal declarations. `:where()` has zero specificity; `:is()` takes its most specific argument.

Flexbox vs Grid: when to use which?

JUNIOR

Flexbox is one-dimensional: a row or a column of items, content decides size (nav bars, button groups). Grid is two-dimensional: you define rows and columns and place items (page layouts, card galleries). They combine well: grid for the page, flex inside components.

How do you center a div horizontally and vertically?

JUNIOR

```
.parent { display: grid; place-items: center; }  
/* or */  
.parent { display: flex; justify-content: center; align-items: center; }
```

Explain `position`: static, relative, absolute, fixed, sticky.

MID

- **relative**: offset from its normal spot, still takes space; becomes containing block for absolute children.
- **absolute**: removed from flow, positioned against nearest positioned ancestor.
- **fixed**: positioned against the viewport.
- **sticky**: normal until a scroll threshold, then sticks within its parent.

What creates a stacking context, and why doesn't my z-index work?

MID

A stacking context is created by `position + z-index`, `opacity < 1`, `transform`, `filter`, `isolation: isolate` and more. z-index only compares siblings inside the same context, so a child with z-index 9999 cannot escape a parent context that sits below another element.

How do you build a responsive layout? Mobile-first?

MID

Mobile-first: write base styles for small screens, add `@media (min-width: ...)` for larger. Use fluid units (`%`, `rem`, `clamp()`), `grid-template-columns: repeat(auto-fit, minmax(240px, 1fr))`, and container queries for component-level responsiveness.

What are CSS custom properties and how do they differ from Sass variables?

MID

Custom properties (`--brand: #2c56c9`) live in the browser, cascade, inherit and can change at runtime (theming, dark mode via `data-theme`). Sass variables are compiled away at build time and cannot change after.

rem vs em vs px vs vh?

MID

`px` is absolute. `em` is relative to the element's font size (compounds when nested). `rem` is relative to the root font size (predictable, respects user settings). `vh/vw` are viewport percentages; prefer `dvh` on mobile to avoid the address-bar jump.

What are container queries and `:has()` ?

SENIOR

`@container (min-width: 400px)` styles a component by its container's size, not the viewport, so the same card works in a sidebar and a main column. `:has()` is a parent selector: `form:has(input:invalid) button { opacity:.5 }`.

CSS Modules vs Tailwind vs CSS-in-JS in a Next.js app?

SENIOR

- **CSS Modules:** scoped class names, zero runtime, works with Server Components.
- **Tailwind:** utility classes, zero runtime, small final CSS, fast iteration; can make markup noisy.
- **Runtime CSS-in-JS** (styled-components, Emotion): needs client components and adds runtime cost; not a good fit for RSC. Zero-runtime options (vanilla-extract, Panda) avoid that.

How do you avoid layout thrashing and janky animations?

SENIOR

Animate only `transform` and `opacity` (compositor-only). Batch DOM reads before writes; don't read `offsetHeight` inside a loop that writes styles. Use `will-change` sparingly and respect `prefers-reduced-motion`.

JavaScript & TypeScript

The language fundamentals every React and Next.js question sits on.

var vs let vs const ?

JUNIOR

`var` is function-scoped and hoisted as `undefined`. `let` / `const` are block-scoped and in the temporal dead zone until declared. `const` prevents reassignment, not mutation of the object.

== vs === ?

JUNIOR

`==` coerces types (`'1' == 1` is true, `null == undefined` is true). `===` compares type and value. Use `===`.

What is a closure? Give a use.

JUNIOR

A function that remembers variables from the scope where it was created, even after that scope finished.

```
function counter() {  
  let n = 0;  
  return () => ++n;  
}  
const next = counter(); next(); // 1, next(); // 2
```

Used for data privacy, memoization, debounce, and React hooks themselves.

Explain the event loop, microtasks and macrotasks.

MID

JS runs one call stack. When it empties, the loop drains **all microtasks** (Promise callbacks, `queueMicrotask`) then takes **one macrotask** (`setTimeout`, I/O, events), then renders if needed.

```
console.log(1);  
setTimeout(() => console.log(2));  
Promise.resolve().then(() => console.log(3));  
console.log(4); // 1 4 3 2
```

How does this work? Arrow functions?

MID

`this` depends on how a function is called: method call → the object; plain call → `undefined` in strict mode; `new` → the new instance; `call`/`apply`/`bind` → explicit. Arrow functions have no own `this`; they take it from the enclosing scope.

Promise.all vs allSettled vs race vs any?

MID

- `all` : resolves when all resolve, rejects on first rejection.
- `allSettled` : waits for all, gives status of each.
- `race` : first to settle (resolve or reject).
- `any` : first to resolve; rejects only if all reject.

Implement debounce.

MID

```
function debounce(fn, ms) {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), ms);
  };
}
```

Debounce waits for a pause (search input). Throttle runs at most once per interval (scroll).

What is event delegation?

MID

Attach one listener on a parent and use `event.target.closest(...)` to handle clicks on many children, including ones added later. Works because events bubble.

Shallow vs deep copy?

MID

Spread and `Object.assign` copy one level; nested objects are shared. `structuredClone(obj)` does a real deep copy (handles Dates, Maps, cycles; not functions).

TypeScript: `interface` vs `type` , and what are generics?

SENIOR

Both describe shapes. Interfaces can be merged and extended; types can express unions, intersections, mapped and conditional types. Generics make reusable typed code:

```
function first<T>(arr: T[]): T | undefined { return arr[0]; }
```

Also know `unknown` vs `any` , `Partial` , `Pick` , `Omit` , `Record` , and discriminated unions.

How does prototypal inheritance work?

SENIOR

Each object has a hidden `[[Prototype]]` link. Property lookup walks the chain until found or `null` . `class` is syntax sugar over this: methods live on `Class.prototype` .

JS Coding & Output

Machine-round questions: predict the output, write polyfills and utilities by hand.

Predict the output: closures in a loop with `var` vs `let` .

MID

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));  
// 3 3 3 (one shared i, loop finished before callbacks run)  
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j));  
// 0 1 2 (let creates a new binding per iteration)
```

Predict the output: event loop ordering.

MID

```
console.log('A');  
setTimeout(() => console.log('B'), 0);  
Promise.resolve().then(() => console.log('C'))  
    .then(() => console.log('D'));  
queueMicrotask(() => console.log('E'));  
(async () => { console.log('F'); await null; console.log('G'); })();  
console.log('H');  
// A F H C E G D B
```

Sync first (A, F, H), then all microtasks in queue order (C, E, G, then D which was queued by C), then the macrotask (B).

Predict the output: `this` in methods and arrows.

MID

```
const obj = {  
  name: 'x',  
  regular() { return this.name; },  
  arrow: () => this?.name,  
  nested() { return [1].map(() => this.name); }  
};  
obj.regular();           // 'x'  
obj.arrow();             // undefined (this from module scope)  
obj.nested();            // ['x'] (arrow takes nested's this)  
const f = obj.regular; f(); // TypeError in strict mode (this is undefined)
```

Write a polyfill for `Array.prototype.map`, `filter` and `reduce`.

MID

```
Array.prototype.myMap = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++)
    if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  return out;
};

Array.prototype.myFilter = function (cb) {
  const out = [];
  for (let i = 0; i < this.length; i++)
    if (i in this && cb(this[i], i, this)) out.push(this[i]);
  return out;
};

Array.prototype.myReduce = function (cb, init) {
  let i = 0, acc = init;
  if (arguments.length < 2) {
    if (!this.length) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[0]; i = 1;
  }
  for (; i < this.length; i++) acc = cb(acc, this[i], i, this);
  return acc;
};
```

Write a polyfill for `Promise.all`.

SENIOR

```
function promiseAll(list) {
  return new Promise((resolve, reject) => {
    const results = [];
    let done = 0;
    if (list.length === 0) return resolve([]);
    list.forEach((p, i) => {
      Promise.resolve(p).then(v => {
        results[i] = v; // keep original order
        if (++done === list.length) resolve(results);
      }, reject); // first rejection wins
    });
  });
}
```


Write `Function.prototype.bind` and a throttle function.

MID

```
Function.prototype.myBind = function (ctx, ...pre) {
  const fn = this;
  return function (...args) { return fn.apply(ctx, [...pre, ...args]); };
};

function throttle(fn, ms) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) { last = now; fn(...args); }
  };
}
```

Implement a deep clone and a deep equal.

SENIOR

```
function deepClone(v, seen = new WeakMap()) {
  if (v === null || typeof v !== 'object') return v;
  if (v instanceof Date) return new Date(v);
  if (seen.has(v)) return seen.get(v); // handle cycles
  const out = Array.isArray(v) ? [] : {};
  seen.set(v, out);
  for (const k of Object.keys(v)) out[k] = deepClone(v[k], seen);
  return out;
}

function deepEqual(a, b) {
  if (a === b) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || !a || !b) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every(k => deepEqual(a[k], b[k]));
}
```

Flatten a nested array without `.flat()`.

MID

```
function flatten(arr, depth = Infinity) {
  return arr.reduce((acc, x) =>
    Array.isArray(x) && depth > 0
      ? acc.concat(flatten(x, depth - 1))
      : acc.concat([x]), []);
}

flatten([1, [2, [3, [4]]]]); // [1,2,3,4]
flatten([1, [2, [3, [4]]], 1); // [1,2,[3,[4]]]
```

Implement an event emitter (pub/sub).

SENIOR

```
class Emitter {
  #map = new Map();
  on(evt, fn) {
    if (!this.#map.has(evt)) this.#map.set(evt, new Set());
    this.#map.get(evt).add(fn);
    return () => this.off(evt, fn); // unsubscribe
  }
  off(evt, fn) { this.#map.get(evt)?.delete(fn); }
  once(evt, fn) {
    const off = this.on(evt, (...a) => { off(); fn(...a); });
  }
  emit(evt, ...args) { this.#map.get(evt)?.forEach(fn => fn(...args)); }
}
```

Write a fetch wrapper with retry, timeout and exponential backoff.

SENIOR

```
async function fetchRetry(url, opts = {}, retries = 3, timeout = 5000) {
  for (let attempt = 0; ; attempt++) {
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), timeout);
    try {
      const res = await fetch(url, { ...opts, signal: ctrl.signal });
      if (res.status >= 500 && attempt < retries) throw new Error('retryable');
      return res;
    } catch (err) {
      if (attempt >= retries) throw err;
      await new Promise(r => setTimeout(r, 2 ** attempt * 300 + Math.random() * 100));
    } finally { clearTimeout(timer); }
  }
}
```

Only retry idempotent requests (GET) or ones with an idempotency key.

Curry a function: `sum(1)(2)(3)` .

MID

```
function curry(fn) {
  return function c(...args) {
    return args.length >= fn.length
      ? fn.apply(this, args)
      : (...more) => c.apply(this, [...args, ...more]);
  };
}

const add3 = curry((a, b, c) => a + b + c);
add3(1)(2)(3); add3(1, 2)(3); // 6
```

Memoize a function.

MID

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
}
```

React

Rendering model, hooks, state, performance, patterns.

What is the Virtual DOM and reconciliation?

JUNIOR

React builds a tree of elements in memory, diffs it against the previous one on each render, and applies only the minimal DOM changes. Elements of a different type are replaced; lists are matched by `key`.

Why do list items need a `key`? Why not use the index?

JUNIOR

Keys let React match items between renders. With index keys, inserting or reordering shifts keys, so React reuses the wrong component state (inputs show wrong values). Use a stable id.

Props vs state?

JUNIOR

Props are inputs passed from the parent, read-only. State is data a component owns and changes with a setter, which triggers a re-render.

Controlled vs uncontrolled components?

JUNIOR

Controlled: the input value lives in React state (`value` + `onChange`). Uncontrolled: the DOM holds the value and you read it via a ref or `FormData`. Controlled gives instant validation; uncontrolled is simpler and works well with Server Actions.

Explain `useEffect` : dependency array and cleanup.

MID

```
useEffect(() => {  
  const id = setInterval(tick, 1000);  
  return () => clearInterval(id); // cleanup  
}, [tick]);
```

- No array: runs after every render. `[]` : once after mount. `[a,b]` : when those change.
- Cleanup runs before the next effect and on unmount.
- Don't use effects to derive state; compute it during render instead.

`useMemo` vs `useCallback` vs `React.memo` ?

MID

`useMemo` caches a computed value; `useCallback` caches a function reference; `React.memo` skips re-rendering a component when props are shallow-equal. They work together: memo'd child + stable callback. With the React Compiler (React 19), much of this is done automatically.

When does a component re-render?

MID

When its state changes, its parent re-renders, or a context it reads changes. Props changing is not itself the trigger; the parent re-rendering is.

`useRef` uses?

MID

Access a DOM node (focus an input) and hold a mutable value that survives renders without causing a re-render (timer ids, previous value).

Context API vs Redux / Zustand? How do you pick state management?

MID

- Local UI state → `useState` .
- Server data → TanStack Query or RSC fetches (caching, refetch, dedupe).
- Rarely changing global values (theme, user) → Context.
- Frequent global client state → Zustand or Redux Toolkit (selectors avoid re-rendering every consumer).
- Shareable UI state → URL search params.

What is a custom hook? Write one.

MID

```
function useDebounce(value, ms = 300) {
  const [v, setV] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setV(value), ms);
    return () => clearTimeout(t);
  }, [value, ms]);
  return v;
}
```

A function starting with `use` that calls other hooks to share stateful logic.

Rules of hooks?

MID

Call hooks only at the top level (not in loops, conditions, or after early returns) and only from components or custom hooks. React relies on call order to match state to hooks.

What is new in React 19?

SENIOR

- Actions: async functions in transitions, with `useActionState` and `useFormStatus`.
- `useOptimistic` for optimistic UI.
- `use()` to read promises and context during render.
- Server Components and Server Actions stable.
- `ref` as a normal prop (no `forwardRef`), document metadata in components.
- React Compiler for automatic memoization.

Explain Suspense, `useTransition` and concurrent rendering.

SENIOR

Concurrent rendering lets React pause and prioritise work. `startTransition` marks an update as non-urgent so typing stays responsive while a heavy list re-renders. Suspense shows a fallback while children wait for code or data, and enables streaming HTML on the server.

How do you find and fix performance problems in a React app?

SENIOR

1. Measure: React DevTools Profiler, Lighthouse, Web Vitals.
2. Stop unnecessary renders: move state down, split context, memoize.
3. Virtualize long lists (TanStack Virtual).
4. Code-split with `lazy()` / dynamic imports.
5. Move data fetching and heavy libraries to Server Components.

What are Error Boundaries and what don't they catch?

SENIOR

Class components with `getDerivedStateFromError` / `componentDidCatch` that catch render errors in their subtree and show a fallback. They don't catch errors in event handlers, async code, or the boundary itself. In Next.js, `error.tsx` is an error boundary per route segment.

React & Next.js Coding

Build-it-live tasks: hooks, forms, lists, modals, auth middleware.

Write a `useFetch` hook that handles loading, errors and race conditions.

MID

```
function useFetch(url) {
  const [state, setState] = useState({ data: null, error: null, loading: true });
  useEffect(() => {
    const ctrl = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: ctrl.signal })
      .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then(data => setState({ data, error: null, loading: false }))
      .catch(err => {
        if (err.name !== 'AbortError') setState({ data: null, error: err, loading: false });
      });
    return () => ctrl.abort(); // cancels stale request when url changes
  }, [url]);
  return state;
}
```

Follow-up: in real apps prefer TanStack Query (caching, retries, dedupe) or Server Components.

Build a debounced search box that calls an API.

MID

```
function Search() {
  const [q, setQ] = useState('');
  const debounced = useDebounce(q, 300);
  const [results, setResults] = useState([]);

  useEffect(() => {
    if (!debounced) return setResults([]);
    const ctrl = new AbortController();
    fetch('/api/search?q=' + encodeURIComponent(debounced), { signal: ctrl.signal })
      .then(r => r.json()).then(setResults).catch(() => {});
    return () => ctrl.abort();
  }, [debounced]);

  return (<>
    <input value={q} onChange={e => setQ(e.target.value)} aria-label="Search" />
    <ul>{results.map(r => <li key={r.id}>{r.name}</li>)}</ul>
  </>);
}
```

Build a Todo list with add, toggle, delete and filter.

MID

```
function Todos() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');
  const [filter, setFilter] = useState('all');

  const add = e => {
    e.preventDefault();
    if (!text.trim()) return;
    setTodos(t => [...t, { id: crypto.randomUUID(), text, done: false }]);
    setText('');
  };

  const toggle = id => setTodos(t => t.map(x => x.id === id ? { ...x, done: !x.done } : x));
  const remove = id => setTodos(t => t.filter(x => x.id !== id));
  const visible = todos.filter(t =>
    filter === 'all' ? true : filter === 'done' ? t.done : !t.done);

  return (<form onSubmit={add}>
    <input value={text} onChange={e => setText(e.target.value)} />
    <button>Add</button>
    {[ 'all', 'active', 'done' ].map(f =>
      <button type="button" key={f} onClick={() => setFilter(f)}>{f}</button>)}
    <ul>{visible.map(t => <li key={t.id}>
      <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
      {t.text} <button type="button" onClick={() => remove(t.id)}>x</button>
    </li>)}</ul>
  </form>);
}
```

Interviewers watch for immutable updates, stable keys and derived (not duplicated) state.

Build a paginated / infinite-scroll list.

MID

```
function Feed() {
  const [items, setItems] = useState([]);
  const [cursor, setCursor] = useState(null);
  const [hasMore, setHasMore] = useState(true);
  const sentinel = useRef(null);

  useEffect(() => {
    const io = new IntersectionObserver(async ([entry]) => {
      if (!entry.isIntersecting || !hasMore) return;
      const res = await fetch('/api/posts?cursor=' + (cursor ?? ''));
      const { data, next } = await res.json();
      setItems(x => [...x, ...data]); setCursor(next); setHasMore(!next);
    });
    if (sentinel.current) io.observe(sentinel.current);
    return () => io.disconnect();
  }, [cursor, hasMore]);

  return (<{items.map(p => <Post key={p.id} {...p} />)}<div ref={sentinel} /></>);
}
```

Write a protected login form in Next.js with a Server Action and `useActionState` .

SENIOR

```
// app/login/actions.ts
'use server';
import { z } from 'zod';
const Schema = z.object({ email: z.string().email(), password: z.string().min(8) });

export async function login(prev, formData) {
  const parsed = Schema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: 'Enter a valid email and password.' };
  const res = await fetch(process.env.API_URL + '/auth/login', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(parsed.data),
  });
  if (!res.ok) return { error: 'Invalid email or password.' };
  const { refresh_token } = await res.json();
  (await cookies()).set('rt', refresh_token, { httpOnly: true, secure: true, sameSite: 'lax',
  path: '/' });
  redirect('/dashboard');
}

// app/login/page.tsx
'use client';
export default function LoginPage() {
  const [state, action, pending] = useActionState(login, null);
  return (<form action={action}>
    <input name="email" type="email" autoComplete="email" required />
    <input name="password" type="password" autoComplete="current-password" required />
    {state?.error && <p role="alert">{state.error}</p>}
    <button disabled={pending}>{pending ? 'Signing in...' : 'Sign in'}</button>
  </form>);
}
```

Write Next.js middleware that protects `/dashboard` routes.

SENIOR

```
// middleware.ts
import { NextResponse } from 'next/server';
import { jwtVerify } from 'jose';

const key = new TextEncoder().encode(process.env.JWT_SECRET);

export async function middleware(req) {
  const token = req.cookies.get('session')?.value;
  try {
    if (!token) throw new Error('no token');
    await jwtVerify(token, key, { issuer: 'myapp' });
    return NextResponse.next();
  } catch {
    const url = new URL('/login', req.url);
    url.searchParams.set('next', req.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
}

export const config = { matcher: ['/dashboard/:path*'] };
```

Follow-up: middleware is a first gate only; verify the session again where data is read.

Implement a simple global store with `useSyncExternalStore`.

SENIOR

```
function createStore(initial) {
  let state = initial;
  const subs = new Set();
  return {
    get: () => state,
    set: fn => { state = fn(state); subs.forEach(s => s()); },
    subscribe: cb => { subs.add(cb); return () => subs.delete(cb); },
  };
}

const cart = createStore({ items: [] });

function useStore(store, selector) {
  return useSyncExternalStore(store.subscribe, () => selector(store.get()));
}

// const count = useStore(cart, s => s.items.length);
```

This is the core idea behind Zustand.

Build an accessible modal component.

MID

```
function Modal({ open, onClose, title, children }) {
  const ref = useRef(null);
  useEffect(() => {
    const d = ref.current;
    if (open && !d.open) d.showModal(); // native focus trap + Esc
    if (!open && d.open) d.close();
  }, [open]);
  return createPortal(
    <dialog ref={ref} onClose={onClose} aria-labelledby="m-title">
      <h2 id="m-title">{title}</h2>{children}
      <button onClick={onClose}>Close</button>
    </dialog>, document.body);
}
```

Next.js

App Router, Server Components, rendering strategies, caching, Server Actions, middleware.

What is Next.js and why use it over plain React?

JUNIOR

A React framework with file-based routing, server rendering (SSR/SSG/ISR), React Server Components, API routes, image/font optimization, and built-in bundling. You get SEO, faster first paint, and a backend in one project.

App Router vs Pages Router?

JUNIOR

- **Pages** (`pages/`): every component is a client component; data via `getServerSideProps` / `getStaticProps` .
- **App** (`app/`): Server Components by default, nested layouts that don't remount, streaming, `loading.tsx` / `error.tsx` , Server Actions. The default for new projects.

What are the special files in the App Router?

JUNIOR

`page.tsx` (route UI), `layout.tsx` (shared, persists), `template.tsx` (remounts), `loading.tsx` (Suspense fallback), `error.tsx` (error boundary), `not-found.tsx` , `route.ts` (API handler), plus `middleware.ts` at the root.

Server Components vs Client Components?

MID

- **Server** (default): run only on the server, can be `async`, query the DB and read secrets directly, ship zero JS.
- **Client** (`'use client'`): needed for state, effects, event handlers and browser APIs.
- Server components can render client components and pass serializable props; client components can receive server components as `children`.

Tip: push `'use client'` as far down the tree as possible.

CSR vs SSR vs SSG vs ISR vs PPR?

MID

- **CSR**: browser fetches and renders. Dashboards behind login.
- **SSR**: HTML per request. Personalized, always fresh.
- **SSG**: HTML at build time. Blogs, docs.
- **ISR**: static, regenerated after `revalidate` seconds or on demand.
- **PPR** (Partial Prerendering): static shell served instantly with dynamic holes streamed in.

How does data fetching and caching work in the App Router?

MID

```
// async Server Component
export default async function Page() {
  const res = await fetch('https://api.example.com/posts',
    { next: { revalidate: 60, tags: ['posts'] } });
  const posts = await res.json();
  return <PostList posts={posts} />;
}
```

Control freshness with `revalidate`, `cache: 'no-store'`, `revalidatePath()` / `revalidateTag()`. Newer versions add the `'use cache'` directive and `cacheLife`. Know that caching is opt-in in Next 15+.

What are Server Actions?

MID

Async functions marked `'use server'` that run on the server and can be called from forms or client code, replacing many API routes for mutations.

```
async function createPost(formData: FormData) {
  'use server';
  const session = await auth();          // always re-check auth
  await db.post.create({ data: { title: formData.get('title') } });
  revalidatePath('/posts');
}
```

Follow-up: treat every action as a public endpoint: validate input (Zod) and check authorization inside it.

What does middleware do, and what are its limits?

MID

`middleware.ts` runs before a request is matched: redirects, rewrites, auth checks on cookies, A/B testing, geo, headers. Keep it light (it runs on every matching request). Don't rely on it as the only auth check; verify again in the data layer.

Dynamic routes, route groups and parallel routes?

MID

- `[id]` dynamic, `[...slug]` catch-all, `[[...slug]]` optional catch-all.
- `(marketing)` route group: organizes files without affecting the URL; allows separate layouts.
- `@modal` parallel routes and `(.)photo` intercepting routes for modals with shareable URLs.

How do you handle SEO and metadata?

MID

Export `metadata` or `generateMetadata()` from a page/layout for title, description, Open Graph. Add `sitemap.ts`, `robots.ts`, and `opengraph-image.tsx`. Server rendering makes content crawlable.

How do `next/image` and `next/font` help?

MID

`next/image` resizes, serves modern formats, lazy-loads and requires dimensions to prevent CLS.
`next/font` self-hosts fonts at build time and uses size-adjusted fallbacks to avoid layout shift.

How do you implement authentication in a Next.js App Router app?

SENIOR

- Use Auth.js (NextAuth), Clerk, or a custom session: httpOnly, Secure, SameSite cookie.
- Middleware for fast redirects of signed-out users.
- A server-only `verifySession()` in a data access layer, called in every Server Component, Route Handler and Server Action that touches data.
- Never pass secrets or full user rows to client components.

What is a hydration error and how do you fix it?

SENIOR

The server HTML doesn't match what the client renders first. Causes: `Date.now()` /random values, reading `window` or `localStorage` during render, invalid HTML nesting, browser extensions. Fix by moving browser-only logic into `useEffect`, using `dynamic(..., { ssr:false })`, or making output deterministic.

How does streaming work in Next.js?

SENIOR

The server sends the layout and fallbacks immediately, then streams each Suspense boundary's HTML as its data resolves. Wrap slow parts in `<Suspense>` or add `loading.tsx`. Start independent fetches in parallel with `Promise.all` to avoid waterfalls.

Route Handlers vs Server Actions: when to use which?

SENIOR

Server Actions for mutations from your own UI (forms, buttons). Route Handlers (`app/api/*/route.ts`) for public APIs, webhooks (Stripe), mobile clients, and GET endpoints other systems call.

How would you deploy Next.js outside Vercel, e.g. on Azure?

SENIOR

Build with `output: 'standalone'`, containerize, and run on Azure App Service (Linux), Azure Container Apps or AKS. Put Azure Front Door/CDN in front for static assets. Static-only sites can go to Azure Static Web Apps. Configure env vars via App Settings and secrets via Key Vault references.

Python

Language internals and idioms that interviewers probe.

What is the difference between a list, tuple, set and dict?

JUNIOR

In simple words: these are the four main ways to store a group of values in Python. They differ in whether you can change them, whether order is kept, and whether duplicates are allowed.

- **list** `[1, 2, 2]` : keeps order, you can change it, duplicates allowed.
- **tuple** `(1, 2, 2)` : keeps order, you **cannot** change it after creating it.
- **set** `{1, 2}` : no duplicates, no fixed order, very fast "is this item inside?" check.
- **dict** `{"name": "Ana"}` : stores key → value pairs, fast lookup by key, keeps insertion order.

```
fruits = ["apple", "kiwi", "apple"]    # list: duplicates are fine
point = (10, 20)                      # tuple: fixed pair of values
unique_fruits = set(fruits)            # set: {'apple', 'kiwi'}
person = {"name": "Ana", "age": 25}    # dict: look up by key
print(person["name"])                 # Ana
```

What are mutable and immutable types? What is the mutable default argument trap?

JUNIOR

In simple words: mutable objects (list, dict, set) can be changed after they are created. Immutable ones (int, str, tuple) cannot. The trap: a default value like `[]` is created only once, so every call shares the same list.

```
# WRONG: the empty list is created once and reused on every call
def add_item(item, bucket=[]):
    bucket.append(item)    # adds to the SAME shared list each time
    return bucket

print(add_item("a"))       # ['a']
print(add_item("b"))       # ['a', 'b'] surprise!

# RIGHT: use None and create a new list inside the function
def add_item_fixed(item, bucket=None):
    if bucket is None:     # no list was passed in
        bucket = []        # so make a brand new one
    bucket.append(item)     # add the item
    return bucket

print(add_item_fixed("a"))  # ['a']
print(add_item_fixed("b"))  # ['b']
```


What is the difference between `is` and `==` ?

JUNIOR

In simple words: `==` asks "do these have the same value?". `is` asks "are these the exact same object in memory?".

```
a = [1, 2]
b = [1, 2]
print(a == b)      # True - same values
print(a is b)      # False - two different list objects

c = a              # c points to the same list as a
print(a is c)      # True

value = None
if value is None:  # use 'is' only for None, True, False
    print("no value")
```

What are `*args` and `**kwargs` ?

JUNIOR

In simple words: `*args` lets a function accept any number of normal arguments (they arrive as a tuple). `**kwargs` lets it accept any number of named arguments (they arrive as a dict).

```
def show(*args, **kwargs):
    print("args:", args)      # a tuple of the plain values
    print("kwargs:", kwargs)  # a dict of the name=value pairs

show(1, 2, city="Pune", age=25)
# args: (1, 2)
# kwargs: {'city': 'Pune', 'age': 25}

numbers = [3, 4]
options = {"sep": "-"}
print(*numbers, **options)    # unpacking: same as print(3, 4, sep="-") -> 3-4
```

What is a decorator? Write one.

MID

In simple words: a decorator is a function that wraps another function to add extra behaviour (like timing or logging) without changing the original code. You apply it with `@name` above the function.

```
import time
import functools

def timed(func):
    # receives the original function
    @functools.wraps(func)      # keeps the original name and docstring
    def wrapper(*args, **kwargs):
        # the new function that replaces it
        start = time.perf_counter()  # note the start time
        result = func(*args, **kwargs)  # run the original function
        end = time.perf_counter()    # note the end time
        print(func.__name__, "took", round(end - start, 4), "seconds")
        return result              # give back the original result
    return wrapper               # return the wrapped version

@timed                          # same as: add = timed(add)
def add(a, b):
    return a + b

print(add(2, 3))
# add took 0.0 seconds
# 5
```

Real uses: login checks, caching, logging, retrying failed calls.

What are generators and `yield` ? Why use them?

MID

In simple words: a generator gives you values one at a time instead of building a whole list in memory. Each `yield` hands out one value and pauses until the next one is asked for.

```
def count_up_to(limit):
    number = 1
    while number <= limit:
        yield number          # hand out one value, then pause here
        number = number + 1    # continue from here on the next request

for n in count_up_to(3):
    # the loop asks for values one by one
    print(n)                  # 1, 2, 3

# A list builds everything at once (uses memory):
squares_list = [x * x for x in range(5)]
# A generator expression makes values only when needed:
squares_gen = (x * x for x in range(5))
print(list(squares_gen))      # [0, 1, 4, 9, 16]
```

Good for large files or millions of database rows, because memory use stays small.

What is the GIL? When do you use threads, processes or asyncio?

MID

In simple words: the GIL (Global Interpreter Lock) means only one thread runs Python code at a time in normal CPython. So threads don't make heavy calculations faster, but they still help when the program is mostly waiting.

- **Waiting on network, files or a database (I/O-bound):** use `asyncio` or threads.
- **Heavy calculations (CPU-bound):** use `multiprocessing`. Each process has its own GIL, so they really run in parallel.
- Python 3.13+ has an optional "free-threaded" build without the GIL. Just know it exists.

Explain `async` / `await` and the event loop in Python.

MID

In simple words: an `async` function can pause at `await` while it waits (for example for a web request). While it waits, the event loop runs other tasks, so many waits happen at the same time.

```
import asyncio

async def download(name, seconds):
    print("start", name)
    await asyncio.sleep(seconds)      # pretend to wait for the network
    print("done", name)
    return name

async def main():
    # run both downloads at the same time
    results = await asyncio.gather(
        download("file1", 2),
        download("file2", 1),
    )
    print(results)                    # ['file1', 'file2'] after about 2 seconds, not 3

asyncio.run(main())                  # start the event loop
```

Never call blocking code inside async functions (`time.sleep` , `requests.get`). Use `asyncio.sleep` and `httpx` instead.

What are context managers (the `with` statement)?

MID

In simple words: a context manager sets something up and **always** cleans it up afterwards, even if an error happens. The `with` statement uses it.

```
# The file is closed automatically when the block ends
with open("notes.txt", "w") as file:
    file.write("hello")
# here the file is already closed

# Writing your own, the easy way
from contextlib import contextmanager

@contextmanager
def announce(name):
    print("start", name)    # setup code
    try:
        yield              # the code inside 'with' runs here
    finally:
        print("end", name) # cleanup always runs

with announce("task"):
    print("working")
# start task / working / end task
```

Common uses: files, database transactions, locks.

What is the difference between a shallow copy and a deep copy?

MID

In simple words: a shallow copy makes a new outer list but still shares the inner items. A deep copy copies everything, including inner lists.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)    # new outer list, same inner lists
deep = copy.deepcopy(original)   # everything copied

original[0].append(99)           # change an inner list

print(shallow)    # [[1, 2, 99], [3, 4]] - affected, inner list is shared
print(deep)       # [[1, 2], [3, 4]]    - not affected
```

How does memory management and garbage collection work in Python?

SENIOR

In simple words: Python counts how many variables point to each object. When that count drops to zero, the object is deleted right away. A separate garbage collector cleans up objects that point to each other in a loop.

- **Reference counting:** the main method, frees memory immediately.
- **Cycle collector** (`gc` module): finds groups of objects that only reference each other.
- `__slots__` on a class saves memory when you create many objects.
- `weakref` lets a cache point to an object without keeping it alive.

What is the difference between a dataclass and a Pydantic model?

SENIOR

In simple words: both save you from writing boilerplate classes. A dataclass just stores data. A Pydantic model also **checks and converts** the data, which makes it right for data coming from outside (API requests).

```
from dataclasses import dataclass
from pydantic import BaseModel

@dataclass
class UserDC:
    name: str
    age: int

UserDC("Ana", "twenty")      # no error, the wrong type is accepted

class UserPD(BaseModel):
    name: str
    age: int

UserPD(name="Ana", age="25")  # OK, "25" is converted to 25
UserPD(name="Ana", age="twenty")  # raises ValidationError
```

Use dataclasses inside your code. Use Pydantic at the edges: FastAPI requests, config, JSON.

What is MRO and how does `super()` work with multiple inheritance?

SENIOR

In simple words: MRO (Method Resolution Order) is the order Python searches classes to find a method. `super()` calls the **next** class in that order, which is not always the direct parent.

```
class A:
    def hello(self):
        print("A")

class B(A):
    def hello(self):
        print("B")
        super().hello()      # go to the next class in the MRO

class C(A):
    def hello(self):
        print("C")
        super().hello()

class D(B, C):               # inherits from both B and C
    pass

print(D.__mro__)            # D -> B -> C -> A -> object
D().hello()                 # prints B, C, A (each only once)
```

Python Basics Coding

Simple Python programs asked in first rounds and online tests: numbers, strings, lists, dicts, loops, patterns, files, basic classes.

Check if a number is even or odd; positive, negative or zero.

JUNIOR

How it works: A number is even when dividing it by 2 leaves no remainder. The `%` operator gives the remainder. Compare the number with 0 to know if it is positive or negative.

```
def even_or_odd(number):
    # % gives the remainder after dividing by 2
    if number % 2 == 0:
        return "Even"    # remainder 0 means even
    else:
        return "Odd"     # remainder 1 means odd

def sign_of(number):
    if number > 0:
        return "Positive"    # bigger than zero
    elif number < 0:
        return "Negative"    # smaller than zero
    else:
        return "Zero"        # exactly zero

print(even_or_odd(7))    # Odd
print(sign_of(-3))      # Negative
```

Find the largest of three numbers.

JUNIOR

How it works: Assume the first number is the largest. Compare it with the second and third numbers, and replace it whenever you find a bigger one.

```
def largest_of_three(a, b, c):
    largest = a        # start by assuming a is the biggest
    if b > largest:     # is b bigger?
        largest = b    # then b is the new biggest
    if c > largest:     # is c bigger?
        largest = c    # then c is the new biggest
    return largest

print(largest_of_three(4, 9, 2))    # 9

# Shorter way (good to mention in the interview):
print(max(4, 9, 2))    # 9
```

Check whether a year is a leap year.

JUNIOR

How it works: A year is a leap year if it divides by 4. But years that divide by 100 are not leap years, unless they also divide by 400.

```
def is_leap(year):
    if year % 400 == 0:
        return True      # e.g. 2000 is a leap year
    if year % 100 == 0:
        return False     # e.g. 1900 is NOT a leap year
    if year % 4 == 0:
        return True      # e.g. 2024 is a leap year
    return False         # all other years

print(is_leap(2024))    # True
print(is_leap(1900))    # False
print(is_leap(2000))    # True
```

Print the multiplication table of a number.

JUNIOR

How it works: Loop from 1 to 10 and print the number multiplied by the loop counter each time.

```
number = 7
for i in range(1, 11):
    result = number * i      # multiply
    print(number, "x", i, "=", result)

# Output:
# 7 x 1 = 7
# 7 x 2 = 14
# ...
# 7 x 10 = 70
```


Sum of the first N natural numbers.

JUNIOR

How it works: Start a total at 0 and add every number from 1 to N to it.

```
def sum_up_to(n):
    total = 0                # start with nothing
    for i in range(1, n + 1): # i goes 1, 2, ... n
        total = total + i    # add the current number
    return total

print(sum_up_to(5))    # 15    (1+2+3+4+5)

# Shorter way (good to mention in the interview):
print(5 * (5 + 1) // 2)    # 15    formula n*(n+1)/2
print(sum(range(1, 6)))    # 15
```

FizzBuzz.

JUNIOR

How it works: For each number from 1 to 100, print "FizzBuzz" if it divides by both 3 and 5, "Fizz" if only by 3, "Buzz" if only by 5, otherwise the number. Check the "both" case first.

```
for number in range(1, 101):    # 1 to 100
    if number % 3 == 0 and number % 5 == 0:    # divides by 3 AND 5
        print("FizzBuzz")
    elif number % 3 == 0:    # divides by 3 only
        print("Fizz")
    elif number % 5 == 0:    # divides by 5 only
        print("Buzz")
    else:
        print(number)    # none of the above

# Output starts: 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

Factorial of a number.

JUNIOR

How it works: Factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1$. Start with 1 and multiply by every number from 2 up to N.

```
def factorial(n):
    result = 1                # start at 1 (not 0, or everything becomes 0)
    for i in range(2, n + 1): # i goes 2, 3, ... n
        result = result * i    # multiply step by step
    return result

print(factorial(5))    # 120

# Recursive version: the function calls itself with a smaller number
def factorial_recursive(n):
    if n <= 1:
        return 1                # stop here
    return n * factorial_recursive(n - 1)    # n x factorial of (n-1)

# Shorter way (good to mention in the interview):
import math
print(math.factorial(5))    # 120
```

Check if a number is prime; print all primes up to N.

JUNIOR

How it works: A prime number is bigger than 1 and divides only by 1 and itself. Try dividing by every number from 2 up to the number minus 1. If any divides exactly, it is not prime.

```
def is_prime(number):
    if number < 2:
        return False            # 0 and 1 are not prime
    for i in range(2, number):  # try 2, 3, ... number-1
        if number % i == 0:     # divides exactly?
            return False        # found a divisor, so not prime
    return True                 # no divisor found

print(is_prime(7))    # True
print(is_prime(10))   # False

# Print all primes from 2 to 30
for number in range(2, 31):
    if is_prime(number):
        print(number, end=" ")    # 2 3 5 7 11 13 17 19 23 29

# Faster check to mention: you only need to test up to the square root
# for i in range(2, int(number ** 0.5) + 1):
```

Print the first N Fibonacci numbers.

JUNIOR

How it works: The series starts 0, 1, and each next number is the sum of the previous two. Keep two variables for the last two numbers and move them forward.

```
def fibonacci(n):
    first = 0
    second = 1
    for i in range(n):
        print(first, end=" ")
        next_number = first + second
        first = second
        second = next_number
    print()

fibonacci(10)    # 0 1 1 2 3 5 8 13 21 34
```

Check if a number is a palindrome (e.g. 121).

JUNIOR

How it works: Build the reversed number digit by digit: take the last digit with `% 10`, add it to the reversed number, and remove it with `// 10`. If the reversed number equals the original, it is a palindrome.

```
def is_palindrome_number(number):
    original = number
    reversed_number = 0
    while number > 0:
        last_digit = number % 10
        reversed_number = reversed_number * 10 + last_digit
        number = number // 10
    return original == reversed_number

print(is_palindrome_number(121))    # True
print(is_palindrome_number(123))    # False

# Shorter way (good to mention in the interview):
print(str(121) == str(121)[::-1])    # True
```

Count digits in a number; sum of digits.

JUNIOR

How it works: Repeatedly take the last digit with `% 10` and remove it with `// 10`, counting and adding as you go.

```
def count_and_sum_digits(number):
    count = 0
    total = 0
    while number > 0:
        last_digit = number % 10    # get last digit
        total = total + last_digit  # add it
        count = count + 1           # one more digit
        number = number // 10       # remove last digit
    return count, total

print(count_and_sum_digits(12345))    # (5, 15)

# Shorter way (good to mention in the interview):
print(len(str(12345)))    # 5
```

Convert Celsius to Fahrenheit and a decimal number to binary.

JUNIOR

How it works: Temperature uses the formula $F = C \times 9/5 + 32$. For binary, keep dividing by 2; the remainders read from bottom to top are the binary digits.

```
def celsius_to_fahrenheit(celsius):
    return celsius * 9 / 5 + 32

print(celsius_to_fahrenheit(100))    # 212.0

def to_binary(number):
    if number == 0:
        return "0"
    binary = ""
    while number > 0:
        remainder = number % 2        # 0 or 1
        binary = str(remainder) + binary  # put it in front
        number = number // 2          # divide by 2
    return binary

print(to_binary(10))    # 1010

# Shorter way (good to mention in the interview):
print(bin(10))          # 0b1010
print(int("1010", 2))   # 10 (binary back to decimal)
```

Reverse a string, a list, and the order of words in a sentence.

JUNIOR

How it works: Go through the text one character at a time and put each character in front of what you have so far. For a sentence, split it into words and add each word in front.

```
def reverse_string(text):
    result = ""
    for letter in text:
        result = letter + result    # put letter at the front
    return result

print(reverse_string("hello"))    # olleh

def reverse_words(sentence):
    words = sentence.split()      # ["I", "love", "Python"]
    result = []
    for word in words:
        result.insert(0, word)    # put each word at the front
    return " ".join(result)      # join with spaces

print(reverse_words("I love Python"))    # Python love I

# Shorter way (good to mention in the interview):
print("hello"[::-1])             # olleh
numbers = [1, 2, 3]
numbers.reverse()                # reverses the list in place -> [3, 2, 1]
```

Count vowels, consonants, digits and spaces in a string.

JUNIOR

How it works: Look at each character. If it is in "aeiou" it is a vowel; if it is some other letter it is a consonant; also check for digits and spaces.

```
def count_characters(text):
    vowels = 0
    consonants = 0
    digits = 0
    spaces = 0
    for letter in text.lower():    # lower() so "A" counts as "a"
        if letter in "aeiou":
            vowels = vowels + 1
        elif letter.isalpha():     # any other letter
            consonants = consonants + 1
        elif letter.isdigit():    # 0-9
            digits = digits + 1
        elif letter == " ":
            spaces = spaces + 1
    return vowels, consonants, digits, spaces

print(count_characters("Hello World 2026"))    # (3, 7, 4, 2)
```

Count how many times each character appears in a string.

JUNIOR

How it works: Use a dictionary: the key is the character and the value is its count. If the character is already in the dictionary add 1, otherwise start it at 1.

```
def char_count(text):
    counts = {}                # empty dictionary
    for letter in text:
        if letter in counts:
            counts[letter] = counts[letter] + 1    # seen before: add 1
        else:
            counts[letter] = 1                    # first time: start at 1
    return counts

print(char_count("banana"))    # {'b': 1, 'a': 3, 'n': 2}

# Shorter way (good to mention in the interview):
from collections import Counter
print(Counter("banana"))      # Counter({'a': 3, 'n': 2, 'b': 1})
```

Remove vowels, spaces or duplicate characters from a string.

JUNIOR

How it works: Build a new string and only add the characters you want to keep.

```
def remove_vowels(text):
    result = ""
    for letter in text:
        if letter.lower() not in "aeiou":    # keep only non-vowels
            result = result + letter
    return result

print(remove_vowels("Hello World"))    # Hll Wrld

def remove_duplicates(text):
    result = ""
    for letter in text:
        if letter not in result:    # add only if not already added
            result = result + letter
    return result

print(remove_duplicates("banana"))    # ban

# Removing spaces:
print("a b c".replace(" ", ""))      # abc
```

Common string methods you should know.

JUNIOR

How it works: Strings have built-in methods. They return a new string; the original does not change.

```
text = "hello world"
print(text.upper())           # HELLO WORLD
print(text.title())           # Hello World
print(text.capitalize())      # Hello world
print(len(text.split()))      # 2 (number of words)
print(text.startswith("hello")) # True
print(text.find("world"))      # 6 (position, -1 if not found)
print(text.count("o"))         # 2
print(text.replace("world", "Python")) # hello Python
print(" hi ".strip())          # hi (removes spaces at both ends)
print("abc".isalpha())         # True (only letters?)
print("123".isdigit())         # True (only digits?)
```

Find the longest word in a sentence.

JUNIOR

How it works: Split the sentence into words. Keep the longest word seen so far and replace it when you find a longer one.

```
def longest_word(sentence):
    words = sentence.split()      # list of words
    longest = ""                  # nothing yet
    for word in words:
        if len(word) > len(longest): # longer than best so far?
            longest = word
    return longest

print(longest_word("Python is an amazing language")) # language

# Shorter way (good to mention in the interview):
print(max("Python is an amazing language".split(), key=len)) # language
```

Sum, average, largest and smallest of a list without built-ins.

JUNIOR

How it works: Walk through the list once. Add each number to a total, and update the largest and smallest whenever you find a bigger or smaller one.

```
numbers = [4, 9, 2, 7]
total = 0
largest = numbers[0]    # assume first is largest
smallest = numbers[0]   # assume first is smallest

for number in numbers:
    total = total + number    # add to total
    if number > largest:
        largest = number     # new largest
    if number < smallest:
        smallest = number    # new smallest

average = total / len(numbers)
print(total, average, largest, smallest)    # 22 5.5 9 2

# Shorter way (good to mention in the interview):
print(sum(numbers), max(numbers), min(numbers))    # 22 9 2
```

Find the second largest number in a list.

JUNIOR

How it works: Keep two variables: the largest and the second largest. When a number beats the largest, the old largest becomes second. When it only beats the second (and is different from the largest), update second.

```
def second_largest(numbers):
    largest = float("-inf")    # smaller than any number
    second = float("-inf")
    for number in numbers:
        if number > largest:
            second = largest    # old largest moves down
            largest = number
        elif number > second and number != largest:
            second = number
    return second

print(second_largest([10, 20, 4, 45, 99, 99]))    # 45

# Shorter way (good to mention in the interview):
print(sorted(set([10, 20, 4, 45, 99]))[-2])    # 45
```


Separate even and odd numbers; square every number.

JUNIOR

How it works: Create empty lists and loop once, appending each number to the right list.

```
numbers = [1, 2, 3, 4, 5, 6]
evens = []
odds = []
squares = []

for number in numbers:
    if number % 2 == 0:
        evens.append(number)      # even goes here
    else:
        odds.append(number)       # odd goes here
        squares.append(number * number) # square of every number

print(evens)    # [2, 4, 6]
print(odds)     # [1, 3, 5]
print(squares)  # [1, 4, 9, 16, 25, 36]

# Shorter way (good to mention in the interview):
evens = [n for n in numbers if n % 2 == 0]  # list comprehension
```

Common elements of two lists; elements in one list but not the other.

JUNIOR

How it works: Loop over the first list and check with `in` whether each item is also in the second list.

```
list_a = [1, 2, 3, 4]
list_b = [3, 4, 5]

common = []
only_in_a = []
for item in list_a:
    if item in list_b:
        common.append(item)      # in both lists
    else:
        only_in_a.append(item)   # only in list_a

print(common)    # [3, 4]
print(only_in_a) # [1, 2]

# Shorter way (good to mention in the interview):
print(set(list_a) & set(list_b))  # {3, 4} common
print(set(list_a) - set(list_b))  # {1, 2} difference
```

Remove duplicates from a list, keeping the original order.

JUNIOR

How it works: Make a new empty list and add each item only if it is not already in the new list.

```
def remove_duplicates(numbers):
    unique = []
    for number in numbers:
        if number not in unique:    # not added yet?
            unique.append(number)
    return unique

print(remove_duplicates([3, 1, 3, 2, 1]))    # [3, 1, 2]

# Shorter way (good to mention in the interview):
print(list(dict.fromkeys([3, 1, 3, 2, 1])))    # [3, 1, 2]
print(list(set([3, 1, 3, 2, 1])))            # removes duplicates, order may change
```

Create a dictionary from two lists; count items in a list.

JUNIOR

How it works: Use the index to pair the key list with the value list. For counting, add 1 to a dictionary entry each time you see an item.

```
keys = ["name", "age", "city"]
values = ["Ana", 25, "Pune"]

person = {}
for i in range(len(keys)):
    person[keys[i]] = values[i]    # pair key with value at same position
print(person)    # {'name': 'Ana', 'age': 25, 'city': 'Pune'}

fruits = ["apple", "kiwi", "apple"]
counts = {}
for fruit in fruits:
    counts[fruit] = counts.get(fruit, 0) + 1    # get() returns 0 if missing
print(counts)    # {'apple': 2, 'kiwi': 1}

# Shorter way (good to mention in the interview):
print(dict(zip(keys, values)))
```

Linear search: find the position of an item in a list.

JUNIOR

How it works: Check each position one by one. Return the position when you find the item, or -1 if you reach the end.

```
def linear_search(items, target):
    for i in range(len(items)):    # i = 0, 1, 2 ...
        if items[i] == target:
            return i               # found it at position i
    return -1                     # not found

print(linear_search([5, 3, 8], 8))    # 2
print(linear_search([5, 3, 8], 4))    # -1

# Shorter way (good to mention in the interview):
print([5, 3, 8].index(8))    # 2 (gives an error if the item is missing)
```

Print star patterns: triangle, pyramid, number triangle.

JUNIOR

How it works: The outer loop is the row number. On each row, print that many stars. For a pyramid, print spaces first to push the stars to the middle.

```
rows = 4

# Right triangle
for i in range(1, rows + 1):
    print("*" * i)          # row 1 has 1 star, row 2 has 2 ...
# *
# **
# ***
# ****

# Pyramid
for i in range(1, rows + 1):
    spaces = " " * (rows - i)    # fewer spaces each row
    stars = "*" * (2 * i - 1)    # 1, 3, 5, 7 stars
    print(spaces + stars)
#   *
#  ***
# *****
# *******

# Number triangle
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j, end=" ")    # print on the same line
    print()                  # go to next line
# 1
# 1 2
# 1 2 3
# 1 2 3 4
```

Simple calculator with a function (and handle divide by zero).

JUNIOR

How it works: Check which operator was given with if/elif and do that calculation. Before dividing, check the second number is not 0.

```
def calculator(first, operator, second):
    if operator == "+":
        return first + second
    elif operator == "-":
        return first - second
    elif operator == "*":
        return first * second
    elif operator == "/":
        if second == 0:
            return "Cannot divide by zero"
        return first / second
    else:
        return "Unknown operator"

print(calculator(10, "+", 5))    # 15
print(calculator(10, "/", 0))    # Cannot divide by zero
```

Keep asking the user until they enter a valid whole number.

JUNIOR

How it works: Loop forever. Try converting the input to an integer; if it works, stop the loop with `break`. If it fails, Python raises `ValueError`, so print a message and ask again.

```
while True:
    answer = input("Enter your age: ")
    try:
        age = int(answer)        # fails if answer is "abc"
        break                    # success, leave the loop
    except ValueError:
        print("Please enter a whole number.")

print("Your age is", age)
```

Read a file and count lines, words and characters; write to a file.

JUNIOR

How it works: Open the file with `with open(...)`, which closes it automatically. Read it line by line and add up the counts. Use mode "w" to write (overwrites) and "a" to append.

```
line_count = 0
word_count = 0
char_count = 0

with open("notes.txt") as file:      # "r" (read) is the default
    for line in file:                # one line at a time
        line_count = line_count + 1
        word_count = word_count + len(line.split())
        char_count = char_count + len(line)

print(line_count, word_count, char_count)

with open("output.txt", "w") as file: # "w" = write, replaces old content
    file.write("Hello\n")             # \n = new line

with open("output.txt", "a") as file: # "a" = append to the end
    file.write("One more line\n")
```

What do `break`, `continue` and `pass` do?

JUNIOR

How it works: `break` stops the loop completely. `continue` skips the rest of this round and goes to the next one. `pass` does nothing; it is a placeholder where Python needs a line.

```
for number in range(10):
    if number == 3:
        continue      # skip 3, go to 4
    if number == 6:
        break          # stop the loop at 6
    print(number)      # prints 0 1 2 4 5

def not_ready_yet():
    pass               # empty function, fill it in later

# for ... else: the else runs only if the loop did NOT break
for number in [1, 3, 5]:
    if number % 2 == 0:
        print("Found an even number")
        break
else:
    print("No even number") # this prints
```

Slicing: get the first or last items, every second item, or copy a list.

JUNIOR

How it works: Slicing uses `list[start:stop:step]`. Start is included, stop is not. Negative numbers count from the end.

```
numbers = [0, 1, 2, 3, 4, 5]
print(numbers[:3])      # [0, 1, 2]    first 3 items
print(numbers[-2:])     # [4, 5]      last 2 items
print(numbers[::2])     # [0, 2, 4]   every second item
print(numbers[1:4])     # [1, 2, 3]   positions 1 to 3
print(numbers[::-1])    # [5, 4, 3, 2, 1, 0] reversed
copy = numbers[:]       # a copy of the list
print("python"[1:4])    # yth (works on strings too)
```

Functions: default values, keyword arguments, and returning more than one value.

JUNIOR

How it works: A default value is used when the caller does not pass that argument. Keyword arguments let you pass values by name. Returning several values with commas gives back a tuple you can unpack.

```
def greet(name, greeting="Hello"):    # greeting has a default
    return greeting + ", " + name + "!"

print(greet("Ana"))                  # Hello, Ana!
print(greet("Bo", greeting="Hi"))   # Hi, Bo!

def smallest_and_largest(numbers):
    return min(numbers), max(numbers) # returns two values

low, high = smallest_and_largest([3, 1, 9]) # unpack them
print(low, high)                       # 1 9

def add_all(*numbers):               # *numbers collects any amount of values
    total = 0
    for number in numbers:
        total = total + number
    return total

print(add_all(1, 2, 3))              # 6
```

Write a simple BankAccount class with deposit, withdraw and balance.

JUNIOR

How it works: A class is a blueprint. `__init__` runs when you create an account and stores the owner and balance. Methods change the balance, with checks for bad amounts.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner          # save the owner's name
        self.balance = balance      # starting money

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit must be more than 0")
            return
        self.balance = self.balance + amount    # add money

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money")
            return
        self.balance = self.balance - amount    # take money out

account = BankAccount("Ana")    # create an account
account.deposit(500)
account.withdraw(200)
print(account.balance)          # 300
```


Inheritance: Animal, Dog and Cat with method overriding.

JUNIOR

How it works: Dog and Cat inherit (reuse) everything from Animal. Each one overrides `speak()` with its own version. `super()` calls the parent's method.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some sound"

class Dog(Animal):
    # Dog is a kind of Animal
    def speak(self):
        # overrides Animal.speak
        return self.name + " says Woof"

class Cat(Animal):
    def speak(self):
        return self.name + " says Meow"

class Puppy(Dog):
    def __init__(self, name, age):
        super().__init__(name)    # let the parent save the name
        self.age = age           # then add our own field

print(Dog("Rex").speak())        # Rex says Woof
print(Cat("Tom").speak())        # Tom says Meow
print(Puppy("Bo", 1).speak())    # Bo says Woof
```

Bubble sort and selection sort by hand.

How it works: Bubble sort compares neighbours and swaps them if they are in the wrong order, so the biggest value moves to the end each round. Selection sort finds the smallest remaining value and puts it at the front.

```
def bubble_sort(numbers):
    n = len(numbers)
    for i in range(n):                # repeat n rounds
        for j in range(n - 1 - i):    # compare neighbours
            if numbers[j] > numbers[j + 1]:
                temp = numbers[j]      # swap the two
                numbers[j] = numbers[j + 1]
                numbers[j + 1] = temp
    return numbers

print(bubble_sort([5, 2, 9, 1]))    # [1, 2, 5, 9]

def selection_sort(numbers):
    for i in range(len(numbers)):
        smallest_index = i           # assume current is smallest
        for j in range(i + 1, len(numbers)):
            if numbers[j] < numbers[smallest_index]:
                smallest_index = j    # found a smaller one
        temp = numbers[i]            # swap it to the front
        numbers[i] = numbers[smallest_index]
        numbers[smallest_index] = temp
    return numbers

print(selection_sort([5, 2, 9, 1])) # [1, 2, 5, 9]

# Shorter way (good to mention in the interview):
print(sorted([5, 2, 9, 1]))        # [1, 2, 5, 9]
```

Python Coding

Python coding round: predict-the-output traps, lists and dicts, OOP, iterators and generators, decorators, concurrency, file handling, sorting, testing.

Predict the output: mutable default argument, lambdas in a loop, and a list of lists.

MID

How it works: Three classic traps. In each one, two names end up pointing to the *same* object, or a value is read later than you expect.

```
# Trap 1: a list as a default argument
def add_item(item, bucket=[]):    # the default list is created ONE time only
    bucket.append(item)          # add the item to that same list
    return bucket

add_item(1)
print(add_item(2))
# Output: [1, 2]
# Why: every call shares the same default list, so it remembers old items.

# Fix: use None and create a new list inside
def add_item(item, bucket=None):
    if bucket is None:           # no list was given
        bucket = []             # so make a brand new one
    bucket.append(item)
    return bucket
```

```
# Trap 2: lambdas created in a loop
functions = []
for i in range(3):
    functions.append(lambda: i)  # the lambda remembers the NAME i, not its value

for f in functions:
    print(f())
# Output: 2 2 2
# Why: all lambdas read i when they are called, and by then the loop has finished with i = 2.

# Fix: store the current value as a default argument
functions = []
for i in range(3):
    functions.append(lambda value=i: value)  # value is saved right now
# Output now: 0 1 2
```

```
# Trap 3: making a grid with *
grid = [[0] * 3] * 3           # copies the SAME inner list 3 times
grid[0][0] = 1                  # change one cell
print(grid)
# Output: [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
# Why: all three rows are the same list, so changing one changes all.

# Fix: build a new row each time
grid = []
for i in range(3):
    grid.append([0] * 3)        # a fresh list for every row
```

Predict the output: `is` vs `==` , floats, booleans and tuples.

How it works: `==` checks whether two values are equal. `is` checks whether they are the exact same object in memory. Floats and booleans also have a few surprises.

```
a = 256
b = 256
print(a is b)
# Output: True
# Why: Python keeps one shared copy of small numbers (-5 to 256).
```

```
a = 1000
b = 1000
print(a == b)
# Output: True (always use == to compare numbers)
# Note: a is b may be False here, so never use "is" for numbers.
```

```
print(0.1 + 0.2 == 0.3)
# Output: False
# Why: computers store 0.1 and 0.2 slightly inaccurately.
```

```
import math
print(math.isclose(0.1 + 0.2, 0.3))
# Output: True (the correct way to compare floats)
```

```
print(True + True)
# Output: 2
# Why: True counts as 1 and False counts as 0.

data = {1: "a", True: "b"}
print(data)
# Output: {1: 'b'}
# Why: 1 and True are equal, so the second one replaces the first value.
```

```
my_tuple = (1, [2])          # a tuple that holds a list
my_tuple[1].append(3)        # allowed: we change the list, not the tuple
print(my_tuple)
# Output: (1, [2, 3])
# Why: the tuple cannot change, but the list inside it can.
```

```
x = [1, 2, 3]
y = x                        # y is another name for the SAME list
y.append(4)                  # change the list through y
print(x)
# Output: [1, 2, 3, 4]
# Why: x and y point to one list. Use y = x.copy() to get a separate list.
```

Predict the output: variable scope, `global` and `nonlocal` .

MID

How it works: If a function assigns to a variable anywhere inside it, Python treats that variable as local for the whole function. Python looks up names in this order: Local, Enclosing function, Global, Built-in (LEGB).

```
x = 10                                # a global variable

def show():
    print(x)                          # tries to print x
    x = 5                             # this line makes x local to the whole function

show()
# Output: UnboundLocalError
# Why: Python sees "x = 5", decides x is local, and the local x has no value yet on the print line.
```

```
# Fix 1: tell Python you mean the global x
count = 0

def increase():
    global count                      # use the global variable
    count = count + 1                # now this changes the global count

increase()
print(count)
# Output: 1
```

```
# Fix 2: nonlocal, for a variable in an outer function
def make_counter():
    number = 0                        # variable in the outer function

    def add_one():
        nonlocal number              # use the outer function's variable
        number = number + 1          # change it
        return number

    return add_one                    # give back the inner function

counter = make_counter()
print(counter())
print(counter())
# Output: 1 then 2
# Why: the inner function remembers "number" between calls. This is called a closure.
```

Swap two variables, find the missing number in 1..n, and find duplicates.

JUNIOR

How it works: To find the missing number, the sum of 1..n minus the sum of the list gives the missing one. For duplicates, remember what you have seen; if you see it again, it is a duplicate.

```
# Swap two variables
a = 5
b = 9
temp = a                # keep a safe
a = b                   # put b into a
b = temp                # put the old a into b
print(a, b)             # 9 5
```

```
# Shorter way (good to mention in the interview):
a, b = b, a
```

```
# Missing number in 1..n
def find_missing(numbers, n):
    expected_total = n * (n + 1) // 2    # sum of 1 + 2 + ... + n
    actual_total = 0
    for number in numbers:               # add up the list
        actual_total = actual_total + number
    return expected_total - actual_total # the difference is the missing number

print(find_missing([1, 2, 4, 5], 5))    # 3
```

```
# Find duplicates
def find_duplicates(numbers):
    seen = set()                        # numbers we have met once
    duplicates = set()                  # numbers we have met more than once
    for number in numbers:
        if number in seen:              # met it before?
            duplicates.add(number)
        else:
            seen.add(number)             # first time: remember it
    return list(duplicates)

print(find_duplicates([1, 3, 3, 5, 1]))  # [1, 3]
```

Sort a dict by value, sort a list of dicts, and merge two dicts.

JUNIOR

How it works: `sorted()` takes a `key` function that says what to sort by. A small named function is the easiest to read.

```
scores = {"a": 3, "b": 1, "c": 2}

def get_score(pair):          # pair looks like ("a", 3)
    return pair[1]            # sort by the number part

sorted_pairs = sorted(scores.items(), key=get_score, reverse=True) # highest first
print(dict(sorted_pairs))     # {'a': 3, 'c': 2, 'b': 1}
```

```
users = [
    {"name": "Ana", "age": 30},
    {"name": "Bo", "age": 25},
]

def get_age(user):
    return user["age"]        # sort by age

print(sorted(users, key=get_age))
# [{'name': 'Bo', 'age': 25}, {'name': 'Ana', 'age': 30}]
```

```
# Merge two dicts
first = {"x": 1, "y": 2}
second = {"y": 20, "z": 3}

merged = {}                    # start empty
for key in first:
    merged[key] = first[key]    # copy everything from first
for key in second:
    merged[key] = second[key]   # copy second; it wins if a key repeats
print(merged)                  # {'x': 1, 'y': 20, 'z': 3}

# Shorter way (good to mention in the interview):
merged = first | second        # Python 3.9+
merged = {**first, **second}
```

Comprehensions: squares of even numbers, invert a dict, flatten a matrix.

JUNIOR

How it works: A comprehension is a short way to write a loop that builds a list or dict. Learn the loop first, then the short form.

```
numbers = [1, 2, 3, 4, 5, 6]

# Squares of even numbers, with a loop
squares = []
for number in numbers:
    if number % 2 == 0:          # keep only even numbers
        squares.append(number * number)
print(squares)                  # [4, 16, 36]

# Shorter way (good to mention in the interview):
squares = [number * number for number in numbers if number % 2 == 0]
```

```
# Invert a dict (swap keys and values)
colors = {"red": 1, "blue": 2}
inverted = {}
for key in colors:
    value = colors[key]
    inverted[value] = key        # value becomes the key
print(inverted)                 # {1: 'red', 2: 'blue'}

# Shorter way:
inverted = {value: key for key, value in colors.items()}
```

```
# Flatten a matrix (list of lists) into one list
matrix = [[1, 2], [3, 4]]
flat = []
for row in matrix:              # go through each row
    for item in row:            # go through each item in the row
        flat.append(item)
print(flat)                     # [1, 2, 3, 4]
```


Flatten a nested list of any depth.

MID

How it works: Go through each item. If it is a list, flatten it too by calling the same function (recursion). If not, add it to the result.

```
def flatten(items):
    result = []                # the final flat list
    for item in items:
        if isinstance(item, list):  # is this item itself a list?
            inner = flatten(item)   # flatten that inner list first
            for value in inner:
                result.append(value) # add its values one by one
        else:
            result.append(item)     # a normal value: just add it
    return result

print(flatten([1, [2, [3, [4]]], 5])) # [1, 2, 3, 4, 5]
```

Split a list into chunks of size n.

MID

How it works: Jump through the list in steps of n and cut a slice each time. The last chunk can be shorter.

```
def split_into_chunks(items, size):
    chunks = []
    for start in range(0, len(items), size): # 0, size, 2*size, ...
        end = start + size
        chunk = items[start:end]             # take the slice
        chunks.append(chunk)
    return chunks

print(split_into_chunks([1, 2, 3, 4, 5, 6, 7], 3))
# [[1, 2, 3], [4, 5, 6], [7]]
```

Armstrong number, sum of digits, reverse an integer, GCD and LCM.

JUNIOR

How it works: Use `% 10` to get the last digit and `// 10` to remove it. Repeat until the number becomes 0.

```
# Armstrong: 153 = 1^3 + 5^3 + 3^3
def is_armstrong(number):
    digits = str(number)          # "153"
    power = len(digits)           # 3 digits, so power 3
    total = 0
    for digit in digits:
        total = total + int(digit) ** power
    return total == number

print(is_armstrong(153))          # True
```

```
# Sum of digits
def digit_sum(number):
    total = 0
    while number > 0:
        last_digit = number % 10   # 123 -> 3
        total = total + last_digit
        number = number // 10      # 123 -> 12
    return total

print(digit_sum(123))            # 6
```

```
# Reverse an integer
def reverse_number(number):
    reversed_number = 0
    while number > 0:
        last_digit = number % 10
        reversed_number = reversed_number * 10 + last_digit # push digit on the end
        number = number // 10
    return reversed_number

print(reverse_number(1234))      # 4321
```

```
# GCD (greatest common divisor), Euclid's method
def gcd(a, b):
    while b != 0:
        remainder = a % b
        a = b
        b = remainder
    return a

def lcm(a, b):
    return a * b // gcd(a, b)

print(gcd(12, 18), lcm(4, 6))   # 6 12
```

```
# Shorter way:
import math
math.gcd(12, 18)
```

Write a generator that gives Fibonacci numbers forever, then take the first 10.

MID

How it works: `yield` hands back one value and pauses the function. The next time you ask, it continues from where it stopped, so it never builds a huge list.

```
def fibonacci():
    current = 0
    following = 1
    while True:
        yield current          # runs forever, but only when asked
        new_value = current + following
        current = following    # give one number and pause here
        following = new_value

numbers = []
for number in fibonacci():
    if len(numbers) == 10:    # stop after 10 numbers
        break
    numbers.append(number)
print(numbers)               # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Write your own iterator class (`__iter__` and `__next__`).

MID

How it works: A `for` loop calls `__iter__` once, then calls `__next__` again and again until it raises `StopIteration`.

```
class Countdown:
    def __init__(self, start):
        self.current = start    # where we begin

    def __iter__(self):
        return self             # this object is its own iterator

    def __next__(self):
        if self.current < 0:    # finished?
            raise StopIteration # tell the for loop to stop
        value = self.current    # the value to give back
        self.current = self.current - 1 # move down for next time
        return value

for number in Countdown(3):
    print(number)
# Output: 3 2 1 0
```

Read a very large log file and count how often each status code appears.

MID

How it works: Looping over an open file reads one line at a time, so even a 10 GB file uses very little memory. A dict keeps the counts.

```
# Example log line:
# 1.2.3.4 - - [28/Sep/2026] "GET / HTTP/1.1" 200 512

def count_status_codes(path):
    counts = {}                                # status code -> how many times
    with open(path) as file:                  # the file closes automatically at the end
        for line in file:                     # reads ONE line at a time
            parts = line.split()              # split the line on spaces
            if len(parts) < 9:                 # skip short or broken lines
                continue
            status = parts[8]                  # the 9th word is the status code
            if status in counts:
                counts[status] = counts[status] + 1
            else:
                counts[status] = 1
    return counts

# print(count_status_codes("access.log"))    # {'200': 950, '404': 30, '500': 2}

# Shorter way:
# from collections import Counter and use counts[status] += 1 with counts = Counter()
```

Read a CSV file and add up total sales per region; read and write JSON.

MID

How it works: `csv.DictReader` gives each row as a dict whose keys are the column names. Add each amount to that region's total.

```
import csv

# sales.csv:
# region,amount
# North,100
# South,50
# North,25

def sales_by_region(path):
    totals = {} # region -> total amount
    with open(path, newline="") as file:
        reader = csv.DictReader(file) # each row becomes a dict
        for row in reader:
            region = row["region"]
            amount = float(row["amount"]) # text -> number
            if region not in totals:
                totals[region] = 0 # first time we see this region
            totals[region] = totals[region] + amount
    return totals

# print(sales_by_region("sales.csv")) # {'North': 125.0, 'South': 50.0}
```

```
import json

text = '{"name": "Ana", "age": 25}'
data = json.loads(text) # JSON text -> Python dict
print(data["name"]) # Ana

back_to_text = json.dumps(data, indent=2) # Python dict -> JSON text
print(back_to_text)
```

Use regex to check an email and find all numbers in a text.

How it works: A regular expression is a pattern. `re.match` checks whether the text fits the pattern; `re.findall` returns every piece of the text that matches.

```
import re

# Pattern parts:
# [\w.+~]+ one or more letters, digits, dots, plus or minus signs
# @ the @ symbol
# [\w-]+ domain name, like "gmail"
# \.[a-z]+ a dot and an ending, like ".com"
email_pattern = r"^[\\w.+~]+@[\\w-]+\\. [a-z]+$"

def is_valid_email(email):
    match = re.match(email_pattern, email)
    return match is not None # True if the pattern matched

print(is_valid_email("dev@gmail.com")) # True
print(is_valid_email("dev@@gmail")) # False
```

```
# Find all whole numbers in a sentence
text = "I have 2 cats and 15 fish"
numbers = re.findall(r"\\d+", text) # \\d+ means "one or more digits"
print(numbers) # ['2', '15']
```

Follow-up: the real way to prove an email is valid is to send a verification link. Regex only catches typing mistakes.

OOP: write a class with `@property`, `@classmethod`, `@staticmethod` and dunder methods.

How it works: `@property` runs a check whenever a value is set. `@classmethod` is another way to create an object. `@staticmethod` is a helper function kept inside the class. Methods like `__add__` let you use `+` on your objects.

```
class Money:
    def __init__(self, amount, currency="INR"):
        self.amount = amount          # this goes through the setter below
        self.currency = currency

    @property
    def amount(self):
        return self._amount           # runs when you READ money.amount

    @amount.setter
    def amount(self, value):
        if value < 0:
            raise ValueError("Amount cannot be negative")
        self._amount = value          # store it in a "private" variable

    @classmethod
    def from_text(cls, text):
        parts = text.split()           # ["100", "USD"]
        amount = float(parts[0])
        currency = parts[1]
        return cls(amount, currency)   # cls means "this class"

    @staticmethod
    def round_two(value):
        return round(value, 2)         # helper; does not need self

    def __add__(self, other):
        # lets you write money1 + money2
        if self.currency != other.currency:
            raise ValueError("Different currencies")
        total = self.amount + other.amount
        return Money(total, self.currency)

    def __eq__(self, other):
        # lets you write money1 == money2
        return self.amount == other.amount and self.currency == other.currency

    def __repr__(self):
        # what print() shows
        return f"Money({self.amount}, {self.currency})"

a = Money(100)
b = Money(50)
print(a + b)                          # Money(150, INR)
print(Money.from_text("20 USD"))      # Money(20.0, USD)
print(Money.round_two(3.14159))       # 3.14
```

Abstract base classes and polymorphism: payment gateways.

MID

How it works: An abstract class is a template. It says every child class *must* have a `charge` method. Polymorphism means you can call `charge()` on any child without knowing which one it is.

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):          # the template
    @abstractmethod
    def charge(self, amount):        # every child MUST write this method
        pass

class Stripe(PaymentGateway):
    def charge(self, amount):
        return f"Charged {amount} with Stripe"

class Razorpay(PaymentGateway):
    def charge(self, amount):
        return f"Charged {amount} with Razorpay"

def checkout(gateway, amount):
    return gateway.charge(amount)    # works with ANY gateway

print(checkout(Stripe(), 500))      # Charged 500 with Stripe
print(checkout(Razorpay(), 500))    # Charged 500 with Razorpay

# PaymentGateway() -> TypeError: you cannot create the template itself
```


Implement a Singleton (only one object of a class can exist).

SENIOR

How it works: `__new__` is the method that creates the object. We create it only once, save it, and return the saved one every time after that.

```
# Way 1: using __new__
class Config:
    _instance = None           # nothing created yet

    def __new__(cls):
        if cls._instance is None:    # first time?
            cls._instance = super().__new__(cls)    # create the one object
        return cls._instance        # always return the same object

first = Config()
second = Config()
print(first is second)        # True: both names point to one object
```

```
# Way 2 (simplest and most "Pythonic"): one object in a module
# settings.py
class Settings:
    debug = False

settings = Settings()        # created once when the module is imported

# other files just do: from settings import settings
```

In FastAPI people usually put `@lru_cache` on a `get_settings()` function to get the same effect.

Write a decorator that takes an argument (repeat a function N times).

How it works: A decorator is a function that wraps another function to add behaviour. A decorator that takes an argument needs one extra outer function to receive that argument.

```
import functools

# First, a simple decorator with no argument
def shout(func):
    @functools.wraps(func)          # keeps the original function's name
    def wrapper():
        result = func()            # call the original function
        return result.upper()      # change its result
    return wrapper

@shout
def greet():
    return "hello"

print(greet())                     # HELLO
```

```
# Now a decorator WITH an argument
def repeat(times):
    def decorator(func):           # 1. receives the argument
        @functools.wraps(func)    # 2. receives the function
        def wrapper():            # 3. runs instead of the function
            results = []
            for i in range(times):
                results.append(func()) # call the function "times" times
            return results
        return wrapper
    return decorator

@repeat(3)
def say_hi():
    return "hi"

print(say_hi())                   # ['hi', 'hi', 'hi']
```

Write a caching decorator that remembers results (memoize), with an expiry time.

MID

How it works: Keep a dict of input → result. If we have seen the input before and the saved result is still fresh, return it without calling the slow function again.

```
import functools
import time

def cache_for(seconds):
    def decorator(func):
        saved = {}                                # input -> (result, time saved)

        @functools.wraps(func)
        def wrapper(number):
            now = time.time()
            if number in saved:                    # seen this input before?
                result, saved_at = saved[number]
                if now - saved_at < seconds:        # still fresh?
                    return result                  # use the saved result
            result = func(number)                  # otherwise do the slow work
            saved[number] = (result, now)           # and remember it
            return result
        return wrapper
    return decorator

@cache_for(60)
def slow_square(number):
    time.sleep(2)                                # pretend this is slow
    return number * number

print(slow_square(4))                            # 16 (takes 2 seconds)
print(slow_square(4))                            # 16 (instant, from the cache)

# Shorter way (built in, no expiry):
# @functools.lru_cache(maxsize=128)
```

Use `map`, `filter`, `zip`, `enumerate`, `any` and `all`.

MID

How it works: These built-in functions replace common loops. `map` changes every item, `filter` keeps some items, `zip` pairs two lists, and `enumerate` gives you the index as well.

```
numbers = [1, 2, 3, 4, 5]

def double(x):
    return x * 2

def is_odd(x):
    return x % 2 == 1

print(list(map(double, numbers)))    # [2, 4, 6, 8, 10]  change every item
print(list(filter(is_odd, numbers))) # [1, 3, 5]        keep the odd ones

names = ["Ana", "Bo"]
ages = [25, 30]
print(list(zip(names, ages)))        # [('Ana', 25), ('Bo', 30)] pair them up

for index, name in enumerate(names): # index and value together
    print(index, name)               # 0 Ana, then 1 Bo

print(any(n > 4 for n in numbers))   # True: at least one is bigger than 4
print(all(n > 0 for n in numbers))   # True: every one is bigger than 0

from functools import reduce

def multiply(total, x):
    return total * x

print(reduce(multiply, numbers))     # 120 = 1*2*3*4*5
```

Make a counter safe to use from many threads.

SENIOR

How it works: `count += 1` is really three steps: read, add, write. Two threads can read the same old value, and one update is lost. A `Lock` lets only one thread do those steps at a time.

```
import threading

class SafeCounter:
    def __init__(self):
        self.value = 0
        self.lock = threading.Lock()    # only one thread can hold it at a time

    def increase(self):
        with self.lock:                 # wait for the lock, then take it
            self.value = self.value + 1 # safe: no other thread is inside here
            # the lock is released automatically here

counter = SafeCounter()

def work():
    for i in range(10000):
        counter.increase()

threads = []
for i in range(4):                    # start 4 threads
    t = threading.Thread(target=work)
    t.start()
    threads.append(t)

for t in threads:
    t.join()                           # wait for every thread to finish

print(counter.value)                  # 40000 every time
```

Without the lock, the total can come out lower than 40000. The GIL does not prevent this, because a thread can be paused between the read and the write.

Producer and consumer using `asyncio.Queue` .

SENIOR

How it works: The producer puts jobs into a queue. Workers take jobs out and handle them at the same time. A special value `None` tells a worker to stop.

```
import asyncio

async def producer(queue):
    for job in range(5):
        await queue.put(job)           # add a job to the queue
        print("added", job)
    await queue.put(None)             # stop signal for worker 1
    await queue.put(None)             # stop signal for worker 2

async def worker(name, queue):
    while True:
        job = await queue.get()        # wait until a job is available
        if job is None:                # stop signal?
            break
        await asyncio.sleep(1)         # pretend to do slow work (e.g. an API call)
        print(name, "finished job", job)

async def main():
    queue = asyncio.Queue()
    await asyncio.gather(              # run all three at the same time
        producer(queue),
        worker("worker-1", queue),
        worker("worker-2", queue),
    )

asyncio.run(main())
# 5 jobs of 1 second each finish in about 3 seconds, because 2 workers share them.
```

Speed up CPU-heavy work by using several processes.

MID

How it works: Because of the GIL, threads cannot run Python calculations in parallel. Separate processes each have their own Python, so they can use all CPU cores.

```
from concurrent.futures import ProcessPoolExecutor

def is_prime(number):
    if number < 2:
        return False
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            return False
    return True

if __name__ == "__main__":
    # required so child processes don't re-run this
    numbers = list(range(1, 100000))
    with ProcessPoolExecutor() as pool: # one worker process per CPU core
        results = list(pool.map(is_prime, numbers)) # split the work across processes

    primes = []
    for i in range(len(numbers)):
        if results[i]: # True means that number is prime
            primes.append(numbers[i])
    print(len(primes)) # 9592
```

Create custom exceptions and use `try` / `except` / `else` / `finally` .

MID

How it works: `try` runs code that might fail. `except` handles a specific error. `else` runs only if there was no error. `finally` always runs, error or not.

```
class InsufficientFunds(Exception):    # our own error type
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFunds(f"Balance is only {balance}")
    return balance - amount

try:
    new_balance = withdraw(100, 500)    # this will fail
except InsufficientFunds as error:      # catch only OUR error
    print("Error:", error)              # Error: Balance is only 100
else:
    print("New balance:", new_balance)  # runs only if no error happened
finally:
    print("Done")                      # always runs

# Output:
# Error: Balance is only 100
# Done
```

Always catch specific errors. A bare `except:` hides bugs.

Build a stack that can also return its smallest value instantly.

MID

How it works: Store each value together with the smallest value seen so far. The top of the stack then always knows the current minimum.

```
class MinStack:
    def __init__(self):
        self.items = []                # each entry is (value, smallest so far)

    def push(self, value):
        if len(self.items) == 0:
            smallest = value            # first item is the smallest
        else:
            current_min = self.items[-1][1] # smallest before this push
            smallest = min(value, current_min)
            self.items.append((value, smallest))

    def pop(self):
        entry = self.items.pop()        # remove the top entry
        return entry[0]                  # return just the value

    def get_min(self):
        return self.items[-1][1]        # smallest is stored at the top

stack = MinStack()
stack.push(5)
stack.push(2)
stack.push(8)
print(stack.get_min())                  # 2
stack.pop()                             # removes 8
stack.pop()                             # removes 2
print(stack.get_min())                  # 5
```

Rotate a matrix 90° clockwise.

MID

How it works: In the new matrix, row i is column i of the old matrix read from bottom to top.

```
def rotate_clockwise(matrix):
    rows = len(matrix)
    cols = len(matrix[0])
    result = []
    for col in range(cols):          # each old column becomes a new row
        new_row = []
        for row in range(rows - 1, -1, -1): # read that column from bottom to top
            new_row.append(matrix[row][col])
        result.append(new_row)
    return result

print(rotate_clockwise([[1, 2, 3],
                        [4, 5, 6],
                        [7, 8, 9]]))

# [[7, 4, 1],
#  [8, 5, 2],
#  [9, 6, 3]]
```

Find the top N most common words in a text.

MID

How it works: Clean the text (lowercase, remove punctuation), split it into words, count them in a dict, then sort by count.

```
import string

def top_words(text, n):
    text = text.lower()                # "The" and "the" become the same
    for mark in string.punctuation:    # remove . , ! ? etc.
        text = text.replace(mark, "")
    words = text.split()               # split on spaces

    counts = {}
    for word in words:
        if word in counts:
            counts[word] = counts[word] + 1
        else:
            counts[word] = 1

    def get_count(pair):
        return pair[1]

    ordered = sorted(counts.items(), key=get_count, reverse=True) # biggest count first
    return ordered[:n]                                           # keep the first n

print(top_words("The cat and the hat. The end!", 2))
# [('the', 3), ('cat', 1)]

# Shorter way:
# from collections import Counter
# Counter(words).most_common(n)
```

Write tests with pytest: a simple test, a fixture, parametrize and a mock.

SENIOR

How it works: pytest runs every function whose name starts with `test_`. `assert` checks a result. A fixture prepares data for tests. Parametrize runs one test with many inputs. A mock replaces a real function, such as sending an email, with a fake one.

```
import pytest
from unittest.mock import patch

def is_palindrome(text):
    return text == text[::-1]

# 1. A simple test
def test_racecar_is_palindrome():
    assert is_palindrome("racecar") == True

# 2. The same test with many inputs
@pytest.mark.parametrize("text, expected", [
    ("racecar", True),
    ("hello", False),
    ("", True),
])
def test_many_words(text, expected):
    assert is_palindrome(text) == expected

# 3. A fixture: reusable test data
@pytest.fixture
def sample_user():
    return {"name": "Ana", "email": "ana@example.com"}

def test_user_has_email(sample_user):      # pytest passes the fixture in
    assert "@" in sample_user["email"]

# 4. Expecting an error
def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        1 / 0

# 5. A mock: don't really send emails during tests
def test_signup_sends_email():
    with patch("app.send_email") as fake_send:  # replace app.send_email with a fake
        signup("ana@example.com")              # code that normally sends an email
        fake_send.assert_called_once()          # check it was called one time
```

Run all tests with the command `pytest`.

Write binary search, bubble sort and merge sort in Python.

MID

How it works: Bubble sort keeps swapping neighbours that are in the wrong order. Merge sort splits the list in half, sorts each half, then merges them. Binary search on a sorted list keeps cutting the search range in half.

```
# Bubble sort:  $O(n^2)$ , easy to explain
def bubble_sort(items):
    n = len(items)
    for i in range(n):
        for j in range(n - 1 - i):
            if items[j] > items[j + 1]:           # wrong order?
                temp = items[j]                  # swap them
                items[j] = items[j + 1]
                items[j + 1] = temp
    return items

print(bubble_sort([5, 1, 4, 2]))                 # [1, 2, 4, 5]
```

```
# Merge sort:  $O(n \log n)$ 
def merge_sort(items):
    if len(items) <= 1:                          # 0 or 1 item is already sorted
        return items
    middle = len(items) // 2
    left = merge_sort(items[:middle])            # sort the left half
    right = merge_sort(items[middle:])           # sort the right half

    merged = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):       # take the smaller front item
        if left[i] <= right[j]:
            merged.append(left[i])
            i = i + 1
        else:
            merged.append(right[j])
            j = j + 1
    merged.extend(left[i:])                      # add whatever is left over
    merged.extend(right[j:])
    return merged

print(merge_sort([8, 3, 5, 1]))                 # [1, 3, 5, 8]
```

```
# Binary search:  $O(\log n)$ , list must be sorted
def binary_search(items, target):
    low = 0
    high = len(items) - 1
    while low <= high:
        middle = (low + high) // 2
        if items[middle] == target:
            return middle                        # found it: return the index
```

```

        elif items[middle] < target:
            low = middle + 1                # look in the right half
        else:
            high = middle - 1              # look in the left half
    return -1                             # not found

print(binary_search([1, 3, 5, 7, 9], 7))  # 3

```

Find all permutations of a string, and count how often a word appears.

MID

How it works: For a permutation, pick each letter as the first letter, then find all permutations of the remaining letters (recursion).

```

def permutations(text):
    if len(text) <= 1:                    # one letter: only one arrangement
        return [text]
    result = []
    for i in range(len(text)):
        first = text[i]                  # choose this letter first
        rest = text[:i] + text[i + 1:]  # all the other letters
        for arrangement in permutations(rest):
            result.append(first + arrangement)
    return result

print(permutations("abc"))
# ['abc', 'acb', 'bac', 'bca', 'cab', 'cba']

# Shorter way:
# from itertools import permutations

```

```

# Count how many times a word appears
sentence = "banana"
print(sentence.count("an"))              # 2

```

Use a `dataclass` with default values.

MID

How it works: `@dataclass` writes `__init__`, `__repr__` and `__eq__` for you, from the fields you list.

```
from dataclasses import dataclass, field

@dataclass
class Task:
    title: str                # required
    priority: int = 1          # has a default value
    tags: list = field(default_factory=list)  # a NEW empty list for each Task
    # never write tags: list = [] (every Task would share one list)

task = Task("Write tests", priority=2)
print(task)                  # Task(title='Write tests', priority=2, tags=[])
task.tags.append("backend")

print(Task("A") == Task("A"))  # True: __eq__ compares the fields
```

```
# frozen=True makes it read-only
@dataclass(frozen=True)
class Point:
    x: int
    y: int

p = Point(1, 2)
# p.x = 5    -> error: cannot change a frozen dataclass
```

Python Backend (FastAPI / Django)

API design and framework questions for the backend half of the role.

FastAPI vs Django vs Flask: when would you pick each?

JUNIOR

In simple words: all three build web backends in Python. They differ in how much comes built in.

- **FastAPI:** best for APIs. It is fast, supports async, checks request data automatically using type hints, and creates API docs for you at `/docs`.
- **Django:** "batteries included". It comes with a database ORM, an admin panel, login and migrations. Best for full websites and admin-heavy apps.
- **Flask:** very small. You pick and add every piece yourself. Good for small services.

How does dependency injection (`Depends`) work in FastAPI?

MID

In simple words: instead of creating things like a database connection inside every route, you write a small function that provides it. FastAPI calls that function for you and passes the result into your route.

```
from fastapi import FastAPI, Depends

app = FastAPI()

# 1. A function that provides a database session
def get_db():
    db = SessionLocal()      # open a database session
    try:
        yield db             # give it to the route
    finally:
        db.close()           # always close it after the request

# 2. A function that finds the logged-in user (it needs the db too)
def get_current_user(db = Depends(get_db)):
    user = db.query(User).first() # simplified: load the user
    return user

# 3. The route just asks for what it needs
@app.get("/me")
def read_me(user = Depends(get_current_user)):
    return {"email": user.email}
```

Bonus: in tests you can swap the real database for a fake one with `app.dependency_overrides` .

How do you structure a production FastAPI project?

MID

In simple words: keep each kind of code in its own place, so the project stays easy to find your way around as it grows.

```
app/
  main.py          # creates the FastAPI app and adds routers
  config.py        # settings read from environment variables
  api/
    users.py       # routes (URLs) for users
    auth.py        # routes for login, signup, reset password
  schemas/         # Pydantic models: shape of request and response data
  models/          # SQLAlchemy models: database tables
  services/        # business logic (what actually happens)
  db.py            # database connection
  migrations/      # Alembic migration files
  tests/           # pytest tests
```

Run it with Uvicorn (or Gunicorn with Uvicorn workers) behind a reverse proxy.

What is the N+1 query problem and how do you fix it?

MID

In simple words: you run 1 query to get a list, then 1 more query for **each** item in that list. 100 posts means 101 queries. The fix is to load the related data together, up front.

```
# PROBLEM (Django): 1 query for posts + 1 query per post for its author
posts = Post.objects.all()
for post in posts:
    print(post.author.name)          # hits the database every time

# FIX: load authors in the same query using a JOIN
posts = Post.objects.select_related("author")

# For many-to-many (e.g. tags) use a second batched query
posts = Post.objects.prefetch_related("tags")

# SQLAlchemy equivalent
from sqlalchemy.orm import selectinload
query = select(Post).options(selectinload(Post.tags))
```

How do you run background jobs in Python?

MID

In simple words: slow work (sending emails, making reports) should not make the user wait. Put it in a background job and reply to the user straight away.

- **Small and quick:** FastAPI `BackgroundTasks`. It runs after the response is sent.
- **Important work that must retry if it fails:** a task queue like Celery or RQ with Redis, or Azure Service Bus with Azure Functions.
- Make jobs safe to run twice (idempotent), because queues can deliver the same job again.

```
from fastapi import BackgroundTasks

def send_welcome_email(email):
    print("sending email to", email)    # slow work goes here

@app.post("/signup")
def signup(email: str, background_tasks: BackgroundTasks):
    # schedule the email; it runs after we reply
    background_tasks.add_task(send_welcome_email, email)
    return {"message": "Account created"} # user gets this immediately
```

Explain Django's request lifecycle and middleware.

MID

In simple words: a request passes through a fixed set of steps on the way in, and back through them on the way out.

1. The request arrives at the server (WSGI or ASGI).
2. It passes through each **middleware** in order (sessions, CSRF check, login, security headers).
3. The URL router finds the matching view.
4. The view runs your code and builds a response (HTML or JSON).
5. The response goes back through the middleware in reverse order.
6. It is sent to the browser.

How do you version and document a REST API?

SENIOR

In simple words: put a version in the URL so old apps keep working when you make big changes, and publish a clear description of every endpoint.

- Use URL versions: `/api/v1/users` , then `/api/v2/users` for breaking changes.
- Don't break existing clients: adding a field is fine, renaming or removing one is not.
- FastAPI generates OpenAPI docs automatically (open `/docs`).
- From that OpenAPI file you can generate a typed TypeScript client for the Next.js front end.

How do you test a FastAPI app?

SENIOR

In simple words: use pytest and FastAPI's `TestClient` to call your API like a real user would, and swap real services (database, login) for test versions.

```
from fastapi.testclient import TestClient
from app.main import app, get_current_user

# a fake user so tests don't need a real login
def fake_current_user():
    return {"id": 1, "email": "test@example.com"}

# tell FastAPI to use the fake instead of the real dependency
app.dependency_overrides[get_current_user] = fake_current_user

client = TestClient(app)

def test_read_me():
    response = client.get("/me")                # call the endpoint
    assert response.status_code == 200          # it worked
    assert response.json()["email"] == "test@example.com"
```

Also useful: a test database that is rolled back after each test, and fixtures that create sample data.

Backend Coding (Python / SQL)

Write the endpoint: auth, password reset, rate limiting, async, SQL queries.

Write FastAPI signup and login endpoints with password hashing and JWT.

MID

How it works: on signup we store a **hash** of the password, never the password itself. On login we check the password against that hash. If it matches, we give back a short-lived access token (JWT) and put a longer-lived refresh token in a secure cookie.

```
import hashlib
import secrets
from datetime import datetime, timedelta, timezone

import jwt                                # pip install pyjwt
from argon2 import PasswordHasher         # pip install argon2-cffi
from fastapi import APIRouter, HTTPException, Response
from pydantic import BaseModel, EmailStr

router = APIRouter(prefix="/auth")
password_hasher = PasswordHasher()
JWT_SECRET = "change-me"                  # in real apps, read this from Key Vault / env
JWT_ALGORITHM = "HS256"

# ----- request bodies -----
class SignupRequest(BaseModel):
    email: EmailStr
    password: str

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

# ----- small helper functions -----
def now_utc():
    # current time in UTC
    return datetime.now(timezone.utc)

def hash_password(plain_password):
    # Argon2 is slow on purpose, which makes guessing passwords expensive
    return password_hasher.hash(plain_password)

def password_matches(stored_hash, plain_password):
    # returns True if the password is correct, False otherwise
    try:
        password_hasher.verify(stored_hash, plain_password)
        return True
    except Exception:
        return False

def hash_token(token):
    # we store only a hash of tokens, so a leaked database can't be used to log in
    return hashlib.sha256(token.encode()).hexdigest()

def create_access_token(user_id):
    # a short-lived token (15 minutes) the browser sends with API calls
```

```

payload = {
    "sub": str(user_id),                # who the token belongs to
    "iat": now_utc(),                  # issued at
    "exp": now_utc() + timedelta(minutes=15), # expires at
}
return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

# ----- endpoints -----
@router.post("/signup", status_code=202)
async def signup(body: SignupRequest):
    existing_user = await find_user_by_email(body.email)    # your database function
    if existing_user is None:
        new_user = await create_user(
            email=body.email,
            password_hash=hash_password(body.password),    # store the hash, never the
password
        )
        await send_verification_email(new_user)            # send "verify your email" link
    # Same message whether or not the email exists,
    # so attackers can't use this form to find out who has an account.
    return {"message": "Check your inbox to verify your email."}

@router.post("/login")
async def login(body: LoginRequest, response: Response):
    user = await find_user_by_email(body.email)

    # One generic error for every failure, so we don't reveal which part was wrong
    if user is None:
        raise HTTPException(status_code=401, detail="Invalid email or password")
    if not password_matches(user.password_hash, body.password):
        raise HTTPException(status_code=401, detail="Invalid email or password")
    if not user.email_verified:
        raise HTTPException(status_code=401, detail="Please verify your email first")

    # Create a long random refresh token and save only its hash
    refresh_token = secrets.token_urlsafe(32)
    await save_refresh_token(
        user_id=user.id,
        token_hash=hash_token(refresh_token),
        expires_at=now_utc() + timedelta(days=14),
    )

    # httponly: JavaScript can't read it (protects against XSS)
    # secure: only sent over HTTPS
    # samesite: not sent on most cross-site requests (helps against CSRF)
    response.set_cookie(
        key="refresh_token",
        value=refresh_token,
        httponly=True,
        secure=True,
        samesite="lax",
        max_age=14 * 24 * 60 * 60,

```

```
)  
return {"access_token": create_access_token(user.id), "token_type": "bearer"}
```

`find_user_by_email`, `create_user`, `save_refresh_token` and `send_verification_email` are your own database and email functions.

Write the forgot-password and reset-password endpoints.

MID

How it works:

1. The user enters their email.
2. We create a random one-time token, save its **hash** with a 30-minute expiry, and email a link containing the token.
3. When the user submits a new password with that token, we check it, save the new password, delete the token, and log the user out everywhere.

```
import hashlib
import secrets
from datetime import datetime, timedelta, timezone
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, EmailStr

router = APIRouter(prefix="/auth")
RESET_LINK_VALID_FOR = timedelta(minutes=30)
FRONTEND_URL = "https://myapp.com" # fixed value from settings, never from the request

class ForgotPasswordRequest(BaseModel):
    email: EmailStr

class ResetPasswordRequest(BaseModel):
    token: str
    new_password: str

def now_utc():
    return datetime.now(timezone.utc)

def hash_token(token):
    # store only the hash: if the database leaks, the tokens are useless
    return hashlib.sha256(token.encode()).hexdigest()

async def create_reset_token(user_id):
    token = secrets.token_urlsafe(32) # long random value, impossible to guess
    await delete_reset_tokens_for_user(user_id) # old reset links stop working
    await save_reset_token(
        user_id=user_id,
        token_hash=hash_token(token),
        expires_at=now_utc() + RESET_LINK_VALID_FOR,
    )
    return token # the plain token goes only in the email

@router.post("/forgot-password", status_code=202)
async def forgot_password(body: ForgotPasswordRequest):
    user = await find_user_by_email(body.email)
    if user is not None:
        token = await create_reset_token(user.id)
        reset_link = FRONTEND_URL + "/reset?token=" + token
        await send_email(user.email, "Reset your password", reset_link)
    # Always the same reply, so nobody can check which emails are registered
```

```

    return {"message": "If an account exists, we sent a reset link."}

@router.post("/reset-password")
async def reset_password(body: ResetPasswordRequest):
    # 1. Find the token by its hash
    saved = await find_reset_token(hash_token(body.token))

    # 2. Reject it if it doesn't exist or has expired
    if saved is None:
        raise HTTPException(status_code=400, detail="This link is invalid or has expired.")
    if saved.expires_at < now_utc():
        raise HTTPException(status_code=400, detail="This link is invalid or has expired.")

    # 3. Save the new password (hashed)
    await update_user_password(saved.user_id, hash_password(body.new_password))

    # 4. The token can only be used once
    await delete_reset_tokens_for_user(saved.user_id)

    # 5. Log out all devices, in case an attacker was logged in
    await revoke_all_refresh_tokens(saved.user_id)

    # 6. Tell the user their password changed (warns them if it wasn't them)
    await send_email_to_user(saved.user_id, "Your password was changed")
    return {"message": "Password updated. Please sign in."}

```

`hash_password` is the Argon2 helper from the signup question. Functions like `find_user_by_email` and `save_reset_token` are your own database code.

Write a FastAPI dependency that reads the current user from a JWT and checks their role.

MID

How it works: the browser sends `Authorization: Bearer <token>`. We check the token's signature and expiry, load the user, then optionally check they have the right role. A 401 means "you're not logged in"; a 403 means "you're logged in but not allowed".

```
import jwt
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

JWT_SECRET = "change-me"
bearer_scheme = HTTPBearer()  # reads the "Authorization: Bearer ..." header

def decode_token(token):
    # checks the signature and the expiry time; raises an error if either is bad
    try:
        return jwt.decode(token, JWT_SECRET, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Invalid or expired token")

async def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme)):
    payload = decode_token(credentials.credentials)  # the raw token string
    user_id = payload["sub"]  # who the token belongs to
    user = await find_user_by_id(user_id)  # your database function
    if user is None:
        raise HTTPException(status_code=401, detail="User not found")
    return user

def require_role(allowed_role):
    # returns a dependency that only lets users with this role through
    async def check_role(user = Depends(get_current_user)):
        if user.role != allowed_role:
            raise HTTPException(status_code=403, detail="Not allowed")
        return user
    return check_role

# Any logged-in user
@app.get("/me")
async def read_me(user = Depends(get_current_user)):
    return {"email": user.email}

# Admins only
@app.delete("/admin/users/{user_id}")
async def delete_user(user_id: str, admin = Depends(require_role("admin"))):
    await remove_user(user_id)
    return {"deleted": user_id}
```

Implement a simple rate limiter with Redis (e.g. max 5 login attempts per minute).

SENIOR

How it works: we keep a counter in Redis for each IP address. Each request adds 1. The counter deletes itself after the time window ends. If the counter goes above the limit, we return HTTP 429 "Too many requests".

```
import redis.asyncio as redis
from fastapi import Request, HTTPException, Depends

redis_client = redis.Redis(host="localhost", port=6379)

async def is_allowed(key, limit, window_seconds):
    count = await redis_client.incr(key)          # add 1 to the counter (creates it at 1)
    if count == 1:
        await redis_client.expire(key, window_seconds)  # first hit: start the timer
    return count <= limit                          # True if still under the limit

def rate_limit(name, limit, window_seconds):
    async def check(request: Request):
        ip_address = request.client.host
        key = "rate:" + name + ":" + ip_address      # e.g. "rate:login:1.2.3.4"
        allowed = await is_allowed(key, limit, window_seconds)
        if not allowed:
            raise HTTPException(
                status_code=429,
                detail="Too many requests. Try again later.",
                headers={"Retry-After": str(window_seconds)},
            )
    return check

# 5 login attempts per 60 seconds per IP
@app.post("/auth/login", dependencies=[Depends(rate_limit("login", 5, 60))])
async def login():
    ...
```

This is a "fixed window" counter. A "sliding window" (using a Redis sorted set) is smoother. Mention it as an improvement.

Write a retry decorator that waits longer after each failure (exponential backoff).

MID

How it works: if the function fails, wait a bit and try again. Each wait is double the previous one (0.5s, 1s, 2s...). After the last attempt, give up and raise the error.

```
import time
import functools

def retry(times=3, first_wait=0.5):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            wait = first_wait
            for attempt in range(1, times + 1):
                # attempt 1, 2, 3
                try:
                    return func(*args, **kwargs)
                # success: return the result
                except Exception as error:
                    if attempt == times:
                        # this was the last try
                        raise
                        # give up, pass the error on
                    print("Attempt", attempt, "failed:", error, "- retrying in", wait, "s")
                    time.sleep(wait)
                    # wait before trying again
                    wait = wait * 2
                    # double the wait next time
            return wrapper
        return decorator

@retry(times=3)
def call_payment_api():
    ... # code that might fail sometimes
```

For `async` functions, write the same thing with `async def wrapper`, `await func(...)` and `await asyncio.sleep(wait)`. Libraries like `tenacity` do this for you.

Fetch many URLs at the same time, but at most 10 at once.

MID

How it works: `asyncio.gather` starts all the requests together. A **Semaphore** works like a gate with 10 tickets, so only 10 requests run at any moment and the server isn't overloaded.

```
import asyncio
import httpx                                # async HTTP client

async def fetch_one(client, gate, url):
    async with gate:                        # wait for a free "ticket"
        response = await client.get(url)    # do the request
        return url, response.status_code    # e.g. ("https://...", 200)

async def fetch_all(urls, max_at_once=10):
    gate = asyncio.Semaphore(max_at_once)  # only 10 requests at the same time
    async with httpx.AsyncClient(timeout=10) as client:
        tasks = []
        for url in urls:
            tasks.append(fetch_one(client, gate, url))
        results = await asyncio.gather(*tasks) # run them all and wait
        return results

urls = ["https://example.com"] * 30
print(asyncio.run(fetch_all(urls)))
```

Write a context manager that measures how long a block of code takes.

MID

How it works: note the time when the `with` block starts, and print the elapsed time when it ends, even if an error happened inside.

```
import time
from contextlib import contextmanager

# Way 1: with a decorator (easiest)
@contextmanager
def timer(label):
    start = time.perf_counter()          # runs when the 'with' block starts
    try:
        yield                            # the code inside 'with' runs here
    finally:
        seconds = time.perf_counter() - start
        print(label, "took", round(seconds, 3), "seconds")  # runs at the end, always

with timer("loading data"):
    time.sleep(1)
# loading data took 1.0 seconds

# Way 2: as a class with __enter__ and __exit__
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()  # start the clock
        return self
    def __exit__(self, error_type, error, traceback):
        self.seconds = time.perf_counter() - self.start  # stop the clock

with Timer() as t:
    time.sleep(0.5)
print(t.seconds)  # about 0.5
```

Write SQL: second highest salary, top 3 earners per department, users with no orders, duplicate emails.

SENIOR

How it works: these four questions come up again and again. Learn the pattern behind each one: a subquery, a window function, a LEFT JOIN, and GROUP BY with HAVING.

```
-- 1. Second highest salary
-- Find the biggest salary that is smaller than the maximum salary
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 2. Top 3 earners in each department
-- DENSE_RANK numbers people 1, 2, 3... inside each department, highest salary first
SELECT *
FROM (
    SELECT name, dept_id, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept
    FROM employees
) ranked
WHERE rank_in_dept <= 3;          -- keep only rank 1, 2 and 3

-- 3. Users who never placed an order
-- LEFT JOIN keeps every user; users with no order get NULL in the order columns
SELECT u.id, u.email
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;             -- NULL means no matching order was found

-- 4. Emails that appear more than once
-- Group rows by email and keep groups with more than one row
SELECT email, COUNT(*) AS how_many
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Let the browser upload a file straight to Azure Blob Storage using a SAS link (Python backend).

SENIOR

How it works: the backend doesn't receive the file itself. It creates a special upload link (a SAS URL) that only works for 10 minutes and only for one file name. The browser then uploads the file directly to Azure using that link. This keeps the API fast and puts no storage password in the code.

```
import uuid
from datetime import datetime, timedelta, timezone
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient, generate_blob_sas, BlobSasPermissions

ACCOUNT_URL = "https://mystorage.blob.core.windows.net"
CONTAINER = "uploads"

# DefaultAzureCredential uses the app's managed identity: no keys in code
blob_service = BlobServiceClient(ACCOUNT_URL, credential=DefaultAzureCredential())

def make_blob_name(user_id, original_name):
    # put each user's files in their own folder, with a random id to avoid clashes
    safe_name = original_name.replace("/", "_")
    return str(user_id) + "/" + str(uuid.uuid4()) + "-" + safe_name

@app.post("/uploads/link")
async def get_upload_link(file_name: str, user = Depends(get_current_user)):
    blob_name = make_blob_name(user.id, file_name)
    start = datetime.now(timezone.utc)
    expiry = start + timedelta(minutes=10)  # link works for 10 minutes only

    # a temporary signing key issued by Azure for our identity
    delegation_key = blob_service.get_user_delegation_key(start, expiry)

    sas_token = generate_blob_sas(
        account_name=blob_service.account_name,
        container_name=CONTAINER,
        blob_name=blob_name,
        user_delegation_key=delegation_key,
        permission=BlobSasPermissions(create=True, write=True),  # upload only, no reading
        expiry=expiry,
    )
    upload_url = ACCOUNT_URL + "/" + CONTAINER + "/" + blob_name + "?" + sas_token
    return {"upload_url": upload_url}

# In the browser: send the file with a PUT request to upload_url
# and add the header x-ms-blob-type: BlockBlob
```

SQL Queries

Write-the-query questions: joins, grouping, nth highest salary, duplicates, window functions, CTEs, running totals, upserts. PostgreSQL syntax, with sample tables and results.

SQL basics: SELECT, WHERE, ORDER BY and LIMIT.

JUNIOR

How it works: SELECT picks the columns, FROM picks the table, WHERE keeps only matching rows, ORDER BY sorts them, and LIMIT keeps only the first few.

```
-- employees(id, name, dept, salary)
SELECT name, salary          -- the columns we want
FROM employees               -- the table to read from
WHERE dept = 'IT'            -- keep only IT employees
    AND salary > 50000        -- and only those earning more than 50,000
ORDER BY salary DESC         -- highest salary first
LIMIT 5;                     -- only the top 5 rows

-- Pattern matching and ranges
SELECT * FROM employees
WHERE name ILIKE 'a%'         -- names starting with a/A (ILIKE ignores case)
    AND salary BETWEEN 40000 AND 90000 -- inclusive range
    AND dept IN ('IT', 'HR');  -- any of these values
```

What does DISTINCT do? Count unique values.

JUNIOR

How it works: DISTINCT removes duplicate rows from the result. COUNT(DISTINCT col) counts how many different values a column has.

```
SELECT DISTINCT dept          -- each department listed once
FROM employees;

SELECT COUNT(DISTINCT dept)   -- how many different departments
FROM employees;
-- Result: 3

-- PostgreSQL extra: one row per dept (the highest paid person)
SELECT DISTINCT ON (dept) dept, name, salary
FROM employees
ORDER BY dept, salary DESC;   -- DISTINCT ON keeps the FIRST row of each dept
```


GROUP BY with COUNT, SUM, AVG, MIN, MAX and HAVING.

JUNIOR

How it works: GROUP BY puts rows with the same value into one bucket. Aggregate functions (COUNT, SUM, AVG...) then give one number per bucket. HAVING filters the buckets.

```
SELECT dept,                -- one row per department
       COUNT(*)             AS people, -- how many employees
       ROUND(AVG(salary)) AS avg_pay, -- average salary
       MAX(salary)          AS top_pay -- highest salary
FROM employees
GROUP BY dept                -- make one bucket per department
HAVING COUNT(*) > 2          -- keep only departments with more than 2 people
ORDER BY avg_pay DESC;
-- Result:
-- dept | people | avg_pay | top_pay
-- IT   | 4      | 72500   | 95000
```

Rule: every column in SELECT must be inside an aggregate or listed in GROUP BY.

WHERE vs HAVING: what is the difference?

JUNIOR

How it works: WHERE filters single rows *before* grouping. HAVING filters groups *after* grouping, so it can use COUNT, SUM, etc.

```
SELECT dept, AVG(salary) AS avg_pay
FROM employees
WHERE salary > 30000      -- step 1: drop low-paid rows (row filter)
GROUP BY dept            -- step 2: group what is left
HAVING AVG(salary) > 60000; -- step 3: keep groups with high average (group filter)

-- WRONG: WHERE AVG(salary) > 60000 -> error, aggregates are not allowed in WHERE
```

Explain INNER, LEFT, RIGHT, FULL OUTER and CROSS JOIN with a tiny example.

JUNIOR

How it works: A join puts rows from two tables side by side when a condition matches. The join type decides what happens to rows that have no match.

```
-- customers          orders
-- id | name              id | customer_id | amount
-- 1 | Ana                10 |             1 |    100
-- 2 | Bo                 11 |             1 |     50
-- 3 | Cy                 12 |             9 |     70  (customer 9 does not exist)

SELECT c.name, o.amount
FROM customers c
INNER JOIN orders o ON o.customer_id = c.id; -- only rows that match on both sides
-- Ana 100, Ana 50

-- LEFT JOIN  : all customers, order columns NULL if none  -> Ana 100, Ana 50, Bo NULL, Cy NULL
-- RIGHT JOIN : all orders, customer NULL if none          -> Ana 100, Ana 50, NULL 70
-- FULL JOIN  : everything from both sides                 -> all 5 rows above
-- CROSS JOIN : every customer paired with every order     -> 3 x 3 = 9 rows
```

Self join: find employees who earn more than their manager.

MID

How it works: We join the employees table to itself. One copy plays "employee", the other plays "manager", linked by manager_id.

```
-- employees(id, name, salary, manager_id)
SELECT e.name    AS employee,
       e.salary  AS emp_salary,
       m.name    AS manager,
       m.salary  AS mgr_salary
FROM employees e
JOIN employees m
  ON e.manager_id = m.id
WHERE e.salary > m.salary;

-- Result:
-- employee | emp_salary | manager | mgr_salary
-- Joe      |    70000   | Sam     |    60000
```

Find the Nth highest salary (3 ways). Example: 2nd highest.

MID

How it works: Sort distinct salaries from high to low and skip the first N-1. DENSE_RANK is the safest when salaries repeat.

```
-- Way 1: LIMIT / OFFSET
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC    -- highest first
LIMIT 1 OFFSET 1;      -- skip 1 row, take the next -> 2nd highest

-- Way 2: subquery
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees); -- highest below the top one

-- Way 3: DENSE_RANK (works for any N, handles ties)
SELECT DISTINCT salary
FROM (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk -- 1, 2, 3... ties share a rank
    FROM employees
) ranked
WHERE rnk = 2;          -- change 2 to any N
-- salaries 100, 100, 90, 80 -> Result: 90
```

Highest paid employee in each department.

MID

How it works: Rank employees inside each department by salary, then keep rank 1. PARTITION BY restarts the ranking for every department.

```
SELECT dept, name, salary
FROM (
    SELECT dept, name, salary,
           RANK() OVER (
               PARTITION BY dept    -- restart numbering for each department
               ORDER BY salary DESC -- highest salary gets rank 1
           ) AS rnk
    FROM employees
) t
WHERE rnk = 1;          -- keeps ties too (two people with the same top pay)

-- Other way: join to the max per department
SELECT e.dept, e.name, e.salary
FROM employees e
JOIN (SELECT dept, MAX(salary) AS max_sal FROM employees GROUP BY dept) m
ON m.dept = e.dept AND m.max_sal = e.salary;
```

Top 3 products by sales in each category (top N per group).

MID

How it works: Number the rows inside each category with ROW_NUMBER, ordered by sales, then keep numbers 1 to 3.

```
WITH ranked AS (                                     -- a named temporary result (CTE)
    SELECT category, product, total_sales,
           ROW_NUMBER() OVER (
               PARTITION BY category           -- numbering starts again per category
               ORDER BY total_sales DESC      -- best seller = 1
           ) AS rn
    FROM product_sales
)
SELECT category, product, total_sales
FROM ranked
WHERE rn <= 3                                         -- top 3 per category
ORDER BY category, rn;
```

Find duplicate emails, then delete duplicates but keep one row.

MID

How it works: GROUP BY + HAVING COUNT(*) > 1 shows values that appear more than once. To delete, number the copies and remove every copy except number 1.

```
-- 1. Find duplicates
SELECT email, COUNT(*) AS copies
FROM users
GROUP BY email
HAVING COUNT(*) > 1;                                -- only emails that appear more than once

-- 2. Delete duplicates, keep the row with the smallest id
DELETE FROM users
WHERE id IN (
    SELECT id
    FROM (
        SELECT id,
               ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn  -- 1 = oldest copy
        FROM users
    ) numbered
    WHERE rn > 1                                     -- every extra copy
);

-- 3. Stop it happening again
ALTER TABLE users ADD CONSTRAINT users_email_key UNIQUE (email);
```

Find customers who never placed an order.

JUNIOR

How it works: LEFT JOIN keeps every customer; customers with no order get NULL in the order columns, so we keep those NULL rows. NOT EXISTS says the same thing directly.

```
-- Way 1: LEFT JOIN + IS NULL
SELECT c.id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id -- keep all customers
WHERE o.id IS NULL;                        -- no matching order found

-- Way 2: NOT EXISTS (clear and fast)
SELECT c.id, c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id -- is there any order for this customer?
);
-- Result: Bo, Cy
```

IN vs EXISTS, and the NULL trap with NOT IN.

MID

How it works: IN compares against a list of values. EXISTS just checks whether the subquery returns any row, and stops at the first match. NOT IN returns **no rows at all** if the list contains a NULL.

```
SELECT name FROM customers
WHERE id IN (SELECT customer_id FROM orders);      -- customers who ordered

SELECT name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id); -- same result

-- The trap:
SELECT name FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders);
-- If any orders.customer_id is NULL, this returns 0 rows!
-- Why: "id <> NULL" is unknown, not true, so no row passes.
-- Fix: use NOT EXISTS, or add WHERE customer_id IS NOT NULL in the subquery.
```

UNION vs UNION ALL.

JUNIOR

How it works: Both stack the results of two queries on top of each other. UNION removes duplicate rows (slower, it has to compare). UNION ALL keeps everything (faster).

```
SELECT email FROM customers
UNION          -- duplicates removed
SELECT email FROM newsletter_subscribers;

SELECT email FROM customers
UNION ALL      -- duplicates kept
SELECT email FROM newsletter_subscribers;
-- Both queries must return the same number of columns with compatible types.
-- Related: INTERSECT (in both), EXCEPT (in first but not second)
```

Subquery vs CTE (WITH clause): when to use which?

MID

How it works: Both are a query inside a query. A CTE gives the inner query a name at the top, which makes long queries much easier to read and lets you reuse it.

```
-- Subquery version
SELECT name, total
FROM (
    SELECT customer_id, SUM(amount) AS total FROM orders GROUP BY customer_id
) t
JOIN customers c ON c.id = t.customer_id
WHERE total > 1000;

-- Same thing with a CTE
WITH customer_totals AS (          -- step 1: total spent per customer
    SELECT customer_id, SUM(amount) AS total
    FROM orders
    GROUP BY customer_id
)
SELECT c.name, ct.total             -- step 2: use it like a table
FROM customer_totals ct
JOIN customers c ON c.id = ct.customer_id
WHERE ct.total > 1000;
```

Performance is usually the same in PostgreSQL 12+ (simple CTEs are inlined).

Recursive CTE: show the full reporting chain (org chart) under a manager.

SENIOR

How it works: A recursive CTE has two parts: a starting row (the boss), and a step that finds the people who report to the rows found so far. It repeats until no new rows are found.

```
WITH RECURSIVE team AS (  
  -- starting point: the CEO (no manager)  
  SELECT id, name, manager_id, 1 AS level  
  FROM employees  
  WHERE manager_id IS NULL  
  
  UNION ALL  
  
  -- repeat: find everyone whose manager is already in "team"  
  SELECT e.id, e.name, e.manager_id, t.level + 1  
  FROM employees e  
  JOIN team t ON e.manager_id = t.id  
)  
SELECT level, name  
FROM team  
ORDER BY level, name;  
-- Result:  
-- level | name  
--      1 | Sam      (CEO)  
--      2 | Joe  
--      2 | Mia  
--      3 | Raj
```

ROW_NUMBER vs RANK vs DENSE_RANK.

MID

How it works: All three number rows in order. They only differ when there is a tie.

```
SELECT name, score,  
       ROW_NUMBER() OVER (ORDER BY score DESC) AS row_num,  -- always 1,2,3,4 (ties get  
different numbers)  
       RANK() OVER (ORDER BY score DESC) AS rnk,           -- ties share, then a gap  
       DENSE_RANK() OVER (ORDER BY score DESC) AS dense_rnk -- ties share, no gap  
FROM results;  
-- Result:  
-- name | score | row_num | rnk | dense_rnk  
-- Ana  | 90    | 1       | 1   | 1  
-- Bo   | 90    | 2       | 1   | 1  
-- Cy   | 80    | 3       | 3   | 2  
-- Dev  | 70    | 4       | 4   | 3
```

Running total and 7-day moving average.

MID

How it works: SUM() OVER (ORDER BY day) adds up everything from the first day to the current day. A frame like ROWS BETWEEN 6 PRECEDING AND CURRENT ROW looks at only the last 7 rows.

```
SELECT day,
       amount,
       SUM(amount) OVER (ORDER BY day) AS running_total,      -- total so far
       ROUND(AVG(amount) OVER (
         ORDER BY day
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW             -- this row + 6 before it
       ), 2) AS moving_avg_7d
FROM daily_sales
ORDER BY day;
-- day          | amount | running_total
-- 2026-09-01   |    100 |           100
-- 2026-09-02   |     50 |           150
-- 2026-09-03   |     70 |           220
```

Monthly revenue and month-over-month change (date_trunc, LAG).

MID

How it works: date_trunc('month', ...) turns every date into the first day of its month, so we can group by month. LAG looks at the previous row's value, so we can subtract.

```
WITH monthly AS (
  SELECT date_trunc('month', created_at)::date AS month,  -- 2026-09-17 -> 2026-09-01
         SUM(amount) AS revenue
  FROM orders
  GROUP BY 1                                              -- group by the first column
)
SELECT month,
       revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,  -- last month's value
       revenue - LAG(revenue) OVER (ORDER BY month) AS change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
         / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS pct_change -- NULLIF
       avoids divide by zero
FROM monthly
ORDER BY month;
-- LEAD() is the opposite: it looks at the NEXT row.
```


Find users who logged in on 3 or more consecutive days.

SENIOR

How it works: Number each user's login days 1, 2, 3... and subtract that number from the date. For days in a row, the result stays the same, so it works as a "streak id" we can group by.

```
WITH days AS (  
    SELECT DISTINCT user_id, login_at::date AS day    -- one row per user per day  
    FROM logins  
)  
grouped AS (  
    SELECT user_id, day,  
           day - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day))::int AS streak_id  
           -- Sep 1 - 1 = Aug 31, Sep 2 - 2 = Aug 31, Sep 3 - 3 = Aug 31  -> same streak  
    FROM days  
)  
SELECT user_id,  
       MIN(day) AS streak_start,  
       MAX(day) AS streak_end,  
       COUNT(*) AS days_in_row  
FROM grouped  
GROUP BY user_id, streak_id  
HAVING COUNT(*) >= 3;           -- streaks of 3 days or more
```

Percentage of total: each department's share of the salary bill.

MID

How it works: SUM(...) OVER () with empty brackets means "the sum over all rows", so we can divide each group by the grand total in the same query.

```
SELECT dept,  
       SUM(salary) AS dept_total,  
       ROUND(100.0 * SUM(salary) / SUM(SUM(salary)) OVER (), 1) AS pct_of_total  
       -- inner SUM = this department, outer SUM() OVER () = all departments  
FROM employees  
GROUP BY dept  
ORDER BY pct_of_total DESC;  
-- dept | dept_total | pct_of_total  
-- IT   | 290000   | 52.7
```

Tip: use 100.0 (not 100) so PostgreSQL does decimal division, not integer division.

Median and percentiles in PostgreSQL.

SENIOR

How it works: percentile_cont(0.5) returns the middle value (averaging the two middle values if the count is even). Change 0.5 to 0.9 for the 90th percentile.

```
SELECT dept,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary,
       percentile_cont(0.9) WITHIN GROUP (ORDER BY salary) AS p90_salary
FROM employees
GROUP BY dept;
-- salaries 40, 50, 60, 100 -> median = 55
-- percentile_disc returns an actual value from the data instead of averaging.
```

Pivot rows into columns (orders per status per month).

MID

How it works: Group by month, then count each status into its own column using CASE or PostgreSQL's FILTER clause.

```
SELECT date_trunc('month', created_at)::date AS month,
       COUNT(*) FILTER (WHERE status = 'paid') AS paid,      -- count only paid rows
       COUNT(*) FILTER (WHERE status = 'refunded') AS refunded,
       SUM(CASE WHEN status = 'paid' THEN amount ELSE 0 END) AS paid_amount -- CASE works in
any database
FROM orders
GROUP BY 1
ORDER BY 1;
-- month      | paid | refunded | paid_amount
-- 2026-08-01 | 120  |         4 |         18500
```

How do you handle NULL? IS NULL, COALESCE, NULLIF.

JUNIOR

How it works: NULL means "unknown". Anything compared with NULL using = gives unknown, so you must use IS NULL. COALESCE picks the first non-NULL value.

```
SELECT * FROM users WHERE phone IS NULL;      -- correct
SELECT * FROM users WHERE phone = NULL;      -- WRONG: always returns nothing

SELECT name, COALESCE(phone, 'not given') AS phone -- replace NULL with a default
FROM users;

SELECT total / NULLIF(count, 0) FROM stats;    -- NULLIF turns 0 into NULL, avoids divide by
zero

-- Aggregates skip NULLs: COUNT(phone) counts only non-NULL, COUNT(*) counts all rows.
```

DELETE vs TRUNCATE vs DROP.

JUNIOR

How it works: They remove different things.

- **DELETE:** removes chosen rows (can use WHERE), row by row, fires triggers, can be rolled back.
- **TRUNCATE:** removes all rows at once, very fast, resets space; the table stays. In PostgreSQL it can also be rolled back inside a transaction.
- **DROP:** removes the whole table, its data and structure.

```
DELETE FROM orders WHERE created_at < '2020-01-01'; -- only old orders
TRUNCATE TABLE logs RESTART IDENTITY;              -- empty the table, reset ids
DROP TABLE IF EXISTS temp_import;                  -- table is gone
```

Primary key vs foreign key vs unique vs check constraints.

JUNIOR

How it works: Constraints are rules the database enforces so bad data can't get in.

```
CREATE TABLE users (
  id          BIGSERIAL PRIMARY KEY,          -- unique + not null, identifies each row
  email       TEXT NOT NULL UNIQUE,          -- no two users with the same email (one NULL
allowed if nullable)
  age         INT CHECK (age >= 18),          -- custom rule
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE orders (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT NOT NULL
              REFERENCES users(id)          -- foreign key: must point to a real user
              ON DELETE CASCADE,            -- deleting a user deletes their orders
  amount      NUMERIC(10,2) NOT NULL
);
```

A table has one primary key but can have many UNIQUE constraints. Index foreign key columns yourself (PostgreSQL doesn't do it automatically).

UPDATE using values from another table (UPDATE with JOIN).

MID

How it works: PostgreSQL uses UPDATE ... FROM to bring in a second table, and the WHERE clause links the two.

```
-- Give everyone in the 'Sales' department a 10% raise
UPDATE employees e
SET salary = e.salary * 1.10          -- new value
FROM departments d                    -- the second table
WHERE d.id = e.dept_id                -- how the tables connect
    AND d.name = 'Sales'              -- which rows to change
RETURNING e.id, e.salary;             -- PostgreSQL: show the updated rows
```

Tip: run the same WHERE as a SELECT first, and do it inside BEGIN ... COMMIT so you can ROLLBACK.

UPSERT: insert a row, or update it if it already exists.

MID

How it works: INSERT ... ON CONFLICT tries to insert; if the unique key already exists it runs the UPDATE instead. EXCLUDED means "the row we tried to insert".

```
INSERT INTO product_stock (product_id, qty)
VALUES (42, 5)
ON CONFLICT (product_id)              -- needs a UNIQUE or PRIMARY KEY on product_id
DO UPDATE SET qty = product_stock.qty + EXCLUDED.qty; -- add to the existing quantity

-- Insert only if missing, otherwise do nothing
INSERT INTO tags (name) VALUES ('python')
ON CONFLICT (name) DO NOTHING;
```

In what order does SQL run the clauses of a query?

JUNIOR

How it works: SQL is written starting with SELECT, but the database runs it in a different order. This explains why you can't use a SELECT alias in WHERE.

```
-- Written order:  SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
-- Run order:
-- 1. FROM / JOIN   pick tables and join them
-- 2. WHERE         filter rows
-- 3. GROUP BY      make groups
-- 4. HAVING        filter groups
-- 5. SELECT        compute columns and aliases (window functions run here)
-- 6. DISTINCT     remove duplicates
-- 7. ORDER BY     sort (can use SELECT aliases)
-- 8. LIMIT/OFFSET cut the result
```

```
SELECT salary * 12 AS yearly FROM employees
WHERE yearly > 100000;          -- ERROR: alias doesn't exist yet at WHERE
-- Fix: WHERE salary * 12 > 100000
```

Count orders per customer including customers with zero orders.

MID

How it works: LEFT JOIN keeps customers with no orders. COUNT(o.id) counts only real orders, so those customers get 0 (COUNT(*) would wrongly give 1).

```
SELECT c.name,
       COUNT(o.id) AS order_count,    -- 0 when no orders
       COALESCE(SUM(o.amount), 0) AS total_spent -- SUM of nothing is NULL, so default to 0
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name
ORDER BY total_spent DESC;
-- name | order_count | total_spent
-- Ana  |           2 |         150
-- Bo   |           0 |           0
```

Find records from the last 7 days / this month / today.

MID

How it works: `now()` is the current timestamp; subtracting an interval gives a past moment.
`date_trunc` rounds down to the start of the day or month.

```
SELECT * FROM orders
WHERE created_at >= now() - interval '7 days';           -- last 7 days

SELECT * FROM orders
WHERE created_at >= date_trunc('month', now());          -- since the 1st of this month

SELECT * FROM orders
WHERE created_at >= current_date                         -- today from midnight
  AND created_at < current_date + 1;                    -- until tomorrow midnight

-- Avoid WHERE created_at::date = current_date on big tables: the cast can stop the index being
used.
```

A query is slow. How do you find out why and fix it?

SENIOR

How it works: Ask PostgreSQL to show its plan with `EXPLAIN ANALYZE`, look for slow steps, then usually add the right index or rewrite the query.

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42 ORDER BY created_at DESC LIMIT 10;
-- Look for: "Seq Scan" on a big table (reading every row),
--           big gaps between estimated and actual rows,
--           slow Sort or Nested Loop steps.

CREATE INDEX CONCURRENTLY idx_orders_customer_created
  ON orders (customer_id, created_at DESC);  -- matches the WHERE + ORDER BY
-- Re-run EXPLAIN ANALYZE: should now show "Index Scan".
```

- Select only needed columns, not `SELECT *`.
- Avoid functions on indexed columns in `WHERE`.
- Keep statistics fresh with `ANALYZE`; paginate with `keyset` instead of big `OFFSET`s.

PostgreSQL

PostgreSQL concepts: data types, JSONB, index types, `EXPLAIN`, MVCC, `VACUUM`, locking, partitioning, replication, pooling, security, Python drivers, Azure.

What is PostgreSQL, and why pick it over MySQL?

JUNIOR

In simple words: PostgreSQL (Postgres) is a free, open-source relational database. It is known for being strict about data correctness and for having many advanced features built in.

- **Strong standards and correctness:** full ACID transactions, including for schema changes (you can roll back an ALTER TABLE).
- **Rich data types:** JSONB, arrays, UUID, ranges, full-text search, geospatial data (PostGIS).
- **Advanced SQL:** window functions, CTEs, partial and expression indexes, materialized views, row-level security.
- **Extensible:** extensions such as pg_stat_statements, pgvector (AI embeddings) and PostGIS.
- **MySQL** is also good: very common for simple read-heavy web apps and easy to host. Postgres is usually the default pick for new projects that need complex queries or JSON.

Which data types should you use for ids, text, time and money?

JUNIOR

In simple words: Pick the type that matches the data exactly, so the database can check it for you and store it efficiently.

```
CREATE TABLE orders (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, -- auto-increment number (modern  
way)  
  public_id   UUID NOT NULL DEFAULT gen_random_uuid(),          -- random id safe to show in URLs  
  customer    TEXT NOT NULL,                                     -- TEXT has no length limit, same  
speed as VARCHAR  
  code        VARCHAR(10),                                       -- use VARCHAR(n) only when a max  
length is a real rule  
  amount      NUMERIC(12, 2) NOT NULL,                           -- exact decimal: always use for  
money, never FLOAT  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),               -- stores time in UTC, converts for  
each user's timezone  
  is_paid     BOOLEAN NOT NULL DEFAULT false,  
  tags        TEXT[],                                           -- an array of text values  
  details     JSONB                                             -- flexible extra data  
);
```

- **SERIAL vs IDENTITY:** both auto-increment. SERIAL is the old shortcut; `GENERATED ... AS IDENTITY` is the SQL standard and is recommended now.
- **FLOAT for money is wrong:** `0.1 + 0.2` is not exactly 0.3 in floating point.
- **Always prefer TIMESTAMPTZ** over `TIMESTAMP`, so times from different timezones stay correct.

JSON vs JSONB, and how do you query JSONB?

MID

In simple words: JSON stores the text exactly as given. JSONB stores it in a parsed binary form, which is a little slower to write but much faster to search and can be indexed. Use JSONB almost always.

```
-- a products table with a JSONB column
CREATE TABLE products (id INT PRIMARY KEY, attrs JSONB);
INSERT INTO products VALUES (1, '{"color": "red", "size": {"w": 10}, "tags": ["new"]}');

SELECT attrs -> 'size'          FROM products; -- {"w": 10}    (-> returns JSON)
SELECT attrs ->> 'color'        FROM products; -- red              (->> returns plain text)
SELECT attrs -> 'size' ->> 'w' FROM products; -- 10                (go deeper step by step)

-- @> means "contains": find products whose attrs include color red
SELECT id FROM products WHERE attrs @> '{"color": "red"}';

-- a GIN index makes @> searches fast on big tables
CREATE INDEX idx_products_attrs ON products USING GIN (attrs);
```

Follow-up: don't put everything in JSONB. Columns you filter, join or sort on often should be normal columns.

What index types does PostgreSQL have and when do you use each?

MID

In simple words: An index is like the index at the back of a book: it helps the database find rows without reading the whole table. Different index types suit different kinds of data.

- **B-tree** (default): equality and ranges on normal columns: `=`, `<`, `>`, `BETWEEN`, `ORDER BY`. Use this 90% of the time.
- **Hash**: only `=` comparisons. Rarely better than B-tree.
- **GIN**: for values that contain many items: JSONB, arrays, full-text search.
- **GiST**: for geometry, ranges and "nearest" searches (PostGIS, overlapping time ranges).
- **BRIN**: tiny index for huge tables where data is stored in natural order, like log rows by time.

```
CREATE INDEX idx_users_email ON users (email); -- B-tree
CREATE INDEX idx_posts_tags  ON posts USING GIN (tags); -- GIN on an array
CREATE INDEX idx_logs_time   ON logs  USING BRIN (created_at); -- BRIN on a big log table
```


What are composite, partial and expression indexes?

MID

In simple words: A composite index covers several columns, a partial index covers only some rows, and an expression index covers a calculated value.

```
-- Composite: helps WHERE user_id = ? AND created_at > ?
-- Column order matters: it also helps "WHERE user_id = ?" alone,
-- but NOT "WHERE created_at > ?" alone (leftmost-column rule).
CREATE INDEX idx_orders_user_date ON orders (user_id, created_at);

-- Partial: index only the rows you actually search (smaller and faster)
CREATE INDEX idx_orders_unpaid ON orders (created_at) WHERE is_paid = false;

-- Expression: makes case-insensitive email lookups use the index
CREATE INDEX idx_users_lower_email ON users (lower(email));
SELECT * FROM users WHERE lower(email) = 'ana@example.com';    -- uses the index

-- Unique partial index: an email must be unique only among active users
CREATE UNIQUE INDEX uniq_active_email ON users (email) WHERE deleted_at IS NULL;
```

How do you read EXPLAIN and EXPLAIN ANALYZE?

MID

In simple words: EXPLAIN shows the plan Postgres will use to run a query. EXPLAIN ANALYZE actually runs it and shows the real time and row counts, so you can see what is slow.

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE user_id = 42;

-- Example output (simplified):
-- Index Scan using idx_orders_user on orders
--   (cost=0.43..8.45 rows=5 width=64)
--   (actual time=0.020..0.031 rows=4 loops=1)
-- Execution Time: 0.05 ms
```

- **Seq Scan:** reads the whole table. Fine for small tables, a warning sign on big ones.
- **Index Scan:** uses an index to jump to matching rows.
- **Index Only Scan:** answers from the index alone, without touching the table (fastest).
- **Bitmap Heap Scan:** collects many matching locations from an index, then reads them in bulk. Used when many rows match.
- **cost:** the planner's estimate (startup..total), not milliseconds.
- **rows estimate vs actual:** a big difference means outdated statistics; run `ANALYZE table_name;`

Careful: EXPLAIN ANALYZE really runs the query. Wrap an UPDATE or DELETE in `BEGIN; ... ROLLBACK;`

What is MVCC?

MID

In simple words: MVCC (Multi-Version Concurrency Control) means Postgres keeps several versions of a row. Readers see a consistent snapshot, and writers create a new version instead of overwriting, so **readers never block writers and writers never block readers**.

- An UPDATE writes a new row version and marks the old one as dead. A DELETE only marks the row as dead.
- Each transaction sees only the versions that were committed when its snapshot was taken.
- Dead versions stay on disk until VACUUM cleans them up (see the next question).

What are VACUUM, autovacuum and table bloat?

MID

In simple words: Because of MVCC, updates and deletes leave dead rows behind. VACUUM cleans them so the space can be reused. Autovacuum is a background worker that does this automatically.

- **Bloat:** too many dead rows make tables and indexes bigger and queries slower.
- **VACUUM:** marks dead space as reusable; doesn't lock out normal reads and writes.
- **VACUUM FULL:** rewrites the table to shrink it on disk, but locks the table. Avoid on busy production tables (use the `pg_repack` extension instead).
- **ANALYZE:** refreshes statistics so the planner makes good choices. Autovacuum also runs this.
- VACUUM also prevents **transaction ID wraparound**, which is why autovacuum must never be switched off.

```
-- which tables have the most dead rows?
SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 5;
```

Transactions and isolation levels in PostgreSQL.

MID

In simple words: A transaction groups several statements so they all succeed or all fail together. The isolation level decides how much one transaction can see of other transactions running at the same time.

```
BEGIN;                                -- start the transaction
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  -- take money from account 1
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  -- give it to account 2
COMMIT;                                -- save both (or ROLLBACK to undo both)

BEGIN ISOLATION LEVEL SERIALIZABLE;  -- choose a stricter level for this transaction
-- ...
COMMIT;
```

- **Read Committed** (default): each statement sees data committed before that statement started.
- **Repeatable Read:** the whole transaction sees one snapshot from its start.
- **Serializable:** behaves as if transactions ran one after another. Postgres may abort one with a serialization error; your code must **retry** it.
- Postgres treats Read Uncommitted as Read Committed; dirty reads never happen.

What do SELECT ... FOR UPDATE and SKIP LOCKED do? Build a job queue.

SENIOR

In simple words: `FOR UPDATE` locks the selected rows so no other transaction can change them until you finish. `SKIP LOCKED` tells other workers to skip rows that are already locked instead of waiting. Together they let many workers take jobs from one table safely.

```
-- each worker runs this to grab one job
BEGIN;
SELECT id, payload
FROM jobs
WHERE status = 'pending'      -- only jobs not done yet
ORDER BY created_at          -- oldest first
LIMIT 1
FOR UPDATE SKIP LOCKED;      -- lock it; skip jobs other workers already took

-- ...the worker does the job in the app...

UPDATE jobs SET status = 'done' WHERE id = 123;  -- mark the job we took as done
COMMIT;                                           -- releases the lock
```

Also useful: `FOR UPDATE` alone to avoid "lost updates", for example when two requests read and change the same wallet balance at the same time.

What causes a deadlock and how do you avoid it?

SENIOR

In simple words: A deadlock is when transaction A waits for a row locked by B, while B waits for a row locked by A. Neither can continue. Postgres detects this after about a second (`deadlock_timeout`) and cancels one of them with an error.

```
-- Transaction A          -- Transaction B
UPDATE accounts SET ... WHERE id=1;  UPDATE accounts SET ... WHERE id=2;
UPDATE accounts SET ... WHERE id=2;  UPDATE accounts SET ... WHERE id=1;  -- deadlock!
```

- **Lock rows in the same order** everywhere (for example, always lowest id first).
- Keep transactions short; don't wait on HTTP calls inside a transaction.
- Retry the transaction when you get a deadlock error.
- Find blocked queries with `pg_stat_activity` and `pg_blocking_pids(pid)` .

View vs materialized view?

MID

In simple words: A view is a saved query; it runs again every time you use it, so data is always fresh. A materialized view stores the result like a table; it's fast to read but must be refreshed to see new data.

```
-- normal view: always up to date
CREATE VIEW active_users AS
SELECT id, email FROM users WHERE deleted_at IS NULL;

-- materialized view: stores the result (good for heavy reports)
CREATE MATERIALIZED VIEW daily_sales AS
SELECT date_trunc('day', created_at) AS day, SUM(amount) AS total
FROM orders
GROUP BY 1;

-- a unique index is required for CONCURRENTLY
CREATE UNIQUE INDEX ON daily_sales (day);

-- refresh without blocking people who are reading it
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales;
```

Refresh on a schedule (cron, `pg_cron`, or an Azure Function timer).

Functions, stored procedures and triggers: write an updated_at trigger.

MID

In simple words: A function is code stored in the database that returns a value. A procedure (`CALL`) can also control transactions. A trigger runs a function automatically when rows are inserted, updated or deleted.

```
-- 1. a function that sets updated_at to the current time
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();    -- NEW is the row being saved
    RETURN NEW;               -- save the changed row
END;
$$ LANGUAGE plpgsql;

-- 2. run it before every update on users
CREATE TRIGGER trg_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();

UPDATE users SET name = 'Ana' WHERE id = 1;    -- updated_at is now set automatically
```

Follow-up: keep most business logic in the application; triggers are hidden and harder to test. They're good for audit logs and timestamps.

Which constraints do you use to keep data correct?

JUNIOR

In simple words: Constraints are rules the database enforces, so bad data can't get in even if the application has a bug.

```
CREATE TABLE users (  
  id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, -- unique and not null  
  email   TEXT NOT NULL UNIQUE,                             -- no two users with the same email  
  age     INT CHECK (age >= 18)                               -- must be an adult  
);  
  
CREATE TABLE orders (  
  id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  user_id BIGINT NOT NULL  
          REFERENCES users(id) ON DELETE CASCADE, -- deleting a user deletes their orders  
  amount  NUMERIC(12,2) CHECK (amount > 0)  
);
```

- **ON DELETE options:** CASCADE (delete children), RESTRICT (block the delete), SET NULL.
- **EXCLUDE** constraint: blocks overlapping values, for example two bookings of the same room at overlapping times (uses GiST).
- Index foreign key columns yourself; Postgres doesn't do it automatically.

What is a sequence? Why can IDs have gaps?

MID

In simple words: A sequence is a counter object that hands out the next number. IDENTITY and SERIAL columns use one behind the scenes.

```
CREATE SEQUENCE invoice_no START 1000;  
SELECT nextval('invoice_no'); -- 1000  
SELECT nextval('invoice_no'); -- 1001  
SELECT currval('invoice_no'); -- 1001 (last value in this session)
```

- Numbers are never given back: if a transaction rolls back, its number is lost, so **gaps are normal**.
- If you need gap-free invoice numbers (for legal reasons), use a separate counter table updated inside the same transaction.

What is table partitioning? Partition a table by month.

SENIOR

In simple words: Partitioning splits one big table into smaller child tables (for example one per month) that still look like one table. Queries that filter by date only read the matching partitions, and old data can be dropped instantly.

```
-- parent table, split by created_at
CREATE TABLE events (
  id          BIGINT GENERATED ALWAYS AS IDENTITY,
  created_at  TIMESTAMPTZ NOT NULL,
  data        JSONB
) PARTITION BY RANGE (created_at);

-- one partition per month
CREATE TABLE events_2026_09 PARTITION OF events
  FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE TABLE events_2026_10 PARTITION OF events
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

-- only scans events_2026_09 (partition pruning)
SELECT count(*) FROM events WHERE created_at >= '2026-09-10' AND created_at < '2026-09-20';

-- removing old data is instant, no slow DELETE
DROP TABLE events_2026_09;
```

Other types: LIST (by region) and HASH (spread evenly). Use partitioning for very large tables (tens of millions of rows or more), not small ones.

How does replication work? Read replicas vs logical replication.

SENIOR

In simple words: Replication copies data from the main (primary) server to other servers, for safety and to spread out read traffic.

- **Streaming (physical) replication:** the primary sends its WAL (write-ahead log) to replicas, which become exact copies. Replicas are read-only, and are used for **read replicas** and failover.
- **Synchronous vs asynchronous:** async is faster but a replica can be a little behind (replication lag); sync waits for the replica, so it's safer but slower.
- **Logical replication:** copies changes for chosen tables using publications and subscriptions. Useful for upgrades with little downtime, or sending data to another system.
- **In the app:** send writes to the primary and heavy reads (reports) to replicas. Remember lag: read your own writes from the primary.

Why use connection pooling (PgBouncer)?

MID

In simple words: Every Postgres connection is a separate server process that uses memory. Hundreds of app instances each opening many connections can overload the database. A pooler keeps a small set of real connections and shares them.

- **App-side pool:** SQLAlchemy or psycopg pools inside each app process.
- **PgBouncer:** sits between apps and Postgres. In **transaction mode**, a real connection is only borrowed for the length of one transaction.
- Serverless functions (Azure Functions) especially need a pooler, because each instance opens its own connections.
- Azure Database for PostgreSQL Flexible Server has PgBouncer built in (port 6432).

```
SHOW max_connections;           -- the server's limit, often 100
SELECT count(*) FROM pg_stat_activity;  -- how many are open right now
```

How do you back up and restore PostgreSQL? What is PITR?

MID

In simple words: pg_dump makes a logical copy (SQL or archive file) of a database. For big production systems, you take physical base backups plus the WAL files, which lets you restore to any exact moment (Point-In-Time Recovery).

```
# logical backup of one database (compressed custom format)
pg_dump -Fc -h localhost -U app mydb > mydb.dump
```

```
# restore it into another database
pg_restore -d mydb_copy mydb.dump
```

```
# plain SQL backup of just the schema
pg_dump --schema-only mydb > schema.sql
```

- **PITR:** base backup + archived WAL means you can restore to, for example, 10:41, one minute before someone ran a bad DELETE.
- Managed services (Azure Flexible Server) do automated backups with PITR for you (7–35 days of retention).
- A backup you've never restored isn't a backup; test restores regularly.

How does full-text search work in PostgreSQL?

MID

In simple words: Postgres can turn text into searchable words (a `tsvector`) and match them against a search query (a `tsquery`). It handles word forms like "running" matching "run", and a GIN index makes it fast.

```
-- add a generated column that holds searchable words
ALTER TABLE articles ADD COLUMN search tsvector
    GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED;

-- index it
CREATE INDEX idx_articles_search ON articles USING GIN (search);

-- find articles about "postgres" AND "index", best matches first
SELECT id, title, ts_rank(search, q) AS rank
FROM articles, to_tsquery('english', 'postgres & index') AS q
WHERE search @@ q          -- @@ means "matches"
ORDER BY rank DESC
LIMIT 10;

-- for user-typed input use websearch_to_tsquery('english', 'postgres index -mysql')
```

For typo-tolerant search use the `pg_trgm` extension; for very large search needs use Elasticsearch or Azure AI Search.

How does Row-Level Security (RLS) help in a multi-tenant app?

SENIOR

In simple words: RLS adds an automatic WHERE filter to every query on a table, so each tenant can only see its own rows, even if the application forgets the filter.

```
-- turn on RLS for the table
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;

-- only rows whose tenant_id equals the current tenant setting are visible
CREATE POLICY tenant_isolation ON projects
    USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- the app sets the tenant at the start of each transaction
BEGIN;
SET LOCAL app.tenant_id = '6f1c...'; -- the logged-in user's tenant
SELECT * FROM projects;              -- returns only this tenant's projects
COMMIT;
```

- Table owners and superusers bypass RLS by default; the app should connect as a normal role (or use `FORCE ROW LEVEL SECURITY`).
- Still index `tenant_id`, because the policy is part of every query.

How do you find slow queries in production?

SENIOR

In simple words: Turn on the `pg_stat_statements` extension. It records every query shape with how often it runs and how long it takes, so you can fix the ones costing the most total time.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- queries using the most total time
SELECT query,
       calls,                                -- how many times it ran
       round(total_exec_time) AS total_ms,    -- time spent overall
       round(mean_exec_time, 1) AS avg_ms     -- time per call
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

-- what is running right now, and for how long?
SELECT pid, now() - query_start AS running_for, state, query
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY running_for DESC;
```

- Also set `log_min_duration_statement = 500` to log queries slower than 500 ms.
- Then run `EXPLAIN ANALYZE` on the worst ones, add indexes or rewrite the query.
- Stop a stuck query with `SELECT pg_cancel_backend(pid);`.

How do you run database migrations without downtime?

SENIOR

In simple words: Avoid changes that lock a big table for a long time, and make changes in small steps that work with both the old and the new version of your app.

```
-- 1. Adding a nullable column is instant
ALTER TABLE users ADD COLUMN phone TEXT;

-- (PG 11+) adding a column with a constant default is also instant
ALTER TABLE users ADD COLUMN is_active BOOLEAN NOT NULL DEFAULT true;

-- 2. Build indexes without blocking writes (can't run inside a transaction)
CREATE INDEX CONCURRENTLY idx_users_phone ON users (phone);

-- 3. Add a constraint without a long lock: add it first, check existing rows later
ALTER TABLE orders ADD CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id) REFERENCES users(id) NOT VALID;
ALTER TABLE orders VALIDATE CONSTRAINT fk_orders_user;

-- 4. Always set a lock timeout so a migration can't freeze the app
SET lock_timeout = '5s';
```

- **Renaming a column:** add the new column, write to both, backfill in batches, switch reads, then drop the old one in a later release (expand and contract).
- Use a migration tool: Alembic (Python), Prisma or Drizzle (Node), Flyway.

How do you use PostgreSQL from Python safely?

MID

In simple words: Use a driver (psycopg) or an ORM (SQLAlchemy). Always pass values as **parameters**, never by building the SQL string yourself, to prevent SQL injection. Reuse connections with a pool.

```
# psycopg 3 with a connection pool
from psycopg_pool import ConnectionPool

pool = ConnectionPool("postgresql://app:secret@localhost/mydb", min_size=1, max_size=10)

def get_user_by_email(email):
    with pool.connection() as conn:                # borrow a connection from the pool
        with conn.cursor() as cur:
            cur.execute(
                "SELECT id, name FROM users WHERE email = %s",    # %s is a placeholder
                (email,),                                         # the value is sent separately
            )
            return cur.fetchone()                            # one row or None

print(get_user_by_email("ana@example.com"))    # (1, 'Ana')

# WRONG, open to SQL injection:
# cur.execute(f"SELECT * FROM users WHERE email = '{email}'")
```

```
# SQLAlchemy 2.0 style
from sqlalchemy import create_engine, text
engine = create_engine("postgresql+psycopg://app:secret@localhost/mydb", pool_size=5)
with engine.begin() as conn:                # starts and commits a
transaction
    rows = conn.execute(text("SELECT id FROM users WHERE age > :age"), {"age": 30}).all()
```

What is Azure Database for PostgreSQL Flexible Server?

MID

In simple words: It's Azure's managed PostgreSQL service. Azure runs the server, patches, backups and high availability, and you use it like normal Postgres.

- **Compute tiers:** Burstable (dev/test), General Purpose, Memory Optimized; you can scale up or down.
- **High availability:** optional zone-redundant standby with automatic failover.
- **Backups:** automatic, with point-in-time restore (7–35 days).
- **Read replicas** for scaling reads, and built-in **PgBouncer**.
- **Security:** private access (VNet integration or private endpoint), firewall rules, TLS, and **Microsoft Entra ID authentication**, so the app can connect with a managed identity instead of a password.
- **Extensions:** allow them in server parameters (for example `pg_stat_statements`, `pgvector`, `PostGIS`).

MongoDB

Documents vs tables, CRUD and query operators, schema design, aggregation pipeline, indexes, transactions, replica sets, sharding, Mongoose and PyMongo, NoSQL injection.

What is MongoDB? How do database, collection and document compare to SQL?

JUNIOR

In simple words: MongoDB is a NoSQL database that stores data as JSON-like documents instead of rows in tables. Each document can have its own shape, and related data can live inside one document.

- **Database** = database
- **Collection** = table
- **Document** = row
- **Field** = column
- **_id** = primary key (added automatically)
- **Embedded document / array** = what would be a separate joined table in SQL

```
// One document in the "users" collection
{
  _id: ObjectId("66f1c2a9e4b0a1b2c3d4e5f6"), // unique id, created automatically
  name: "Ana",                               // simple field
  email: "ana@example.com",
  address: { city: "Pune", pin: "411001" },    // embedded document (no join needed)
  skills: ["python", "react"]                 // array field
}
```

When would you choose MongoDB over PostgreSQL, and when not?

MID

In simple words: Pick MongoDB when your data is naturally shaped like documents and changes shape often. Pick PostgreSQL when data is very relational and you need strong joins, constraints and complex transactions.

- **MongoDB fits:** product catalogs with different attributes per product, content/CMS, event logs, user profiles and settings, fast prototyping, very large scale with sharding.
- **PostgreSQL fits:** payments, orders and inventory, reporting with many joins, strict data integrity (foreign keys, CHECK constraints).
- PostgreSQL's JSONB covers many "flexible field" needs, so many teams use Postgres by default and add MongoDB only when document scale or shape really needs it.

Interview tip: say the trade-off out loud: flexible schema and easy horizontal scaling vs. joins, constraints and mature SQL reporting.

What is BSON, and what is inside an ObjectId?

JUNIOR

In simple words: BSON is "binary JSON", the format MongoDB uses to store documents. It is faster to read and supports extra types that JSON does not have, like dates, 64-bit integers, decimals and binary data.

An **ObjectId** is a 12-byte id made of:

- 4 bytes: the time it was created (seconds since 1970)
- 5 bytes: a random value unique to the machine/process
- 3 bytes: a counter that increases

```
const id = ObjectId()           // create a new id
id.getTimestamp()               // when it was created, e.g. ISODate("2026-09-29T10:15:00Z")
// Because the time comes first, sorting by _id is roughly sorting by creation time
```

Basic CRUD in mongosh: insert, find, update, delete.

JUNIOR

In simple words: Create with `insertOne/insertMany` , read with `find` , change with `updateOne/updateMany` , remove with `deleteOne/deleteMany` .

```
// CREATE
db.users.insertOne({ name: "Ana", age: 28, city: "Pune" })           // add one document
db.users.insertMany([                                             // add several at once
  { name: "Bo", age: 35, city: "Delhi" },
  { name: "Cy", age: 22, city: "Pune" }
])

// READ
db.users.find({ city: "Pune" })                                     // all users in Pune
db.users.find({ city: "Pune" }, { name: 1, _id: 0 })              // projection: only return name
// Result: { name: "Ana" }, { name: "Cy" }
db.users.findOne({ name: "Bo" })                                   // first match only

// UPDATE
db.users.updateOne({ name: "Ana" }, { $set: { city: "Mumbai" } }) // change one field
db.users.updateMany({ city: "Pune" }, { $inc: { age: 1 } })       // add 1 to age for
everyone in Pune

// DELETE
db.users.deleteOne({ name: "Cy" })                                 // remove one
db.users.deleteMany({ age: { $lt: 18 } })                         // remove all under 18
```

Explain update operators: \$set, \$unset, \$inc, \$push, \$addToSet, \$pull, and upsert.

MID

In simple words: Update operators change only part of a document instead of replacing it. Upsert means "update if found, otherwise insert a new one".

```
// Sample: { _id: 1, name: "Ana", points: 10, tags: ["new"] }

db.users.updateOne({ _id: 1 }, { $set: { city: "Pune" } }) // add or change a field
db.users.updateOne({ _id: 1 }, { $unset: { city: "" } }) // remove a field
db.users.updateOne({ _id: 1 }, { $inc: { points: 5 } }) // points: 10 -> 15
db.users.updateOne({ _id: 1 }, { $push: { tags: "vip" } }) // add to array (duplicates
allowed)
db.users.updateOne({ _id: 1 }, { $addToSet: { tags: "vip" } }) // add only if not already
there
db.users.updateOne({ _id: 1 }, { $pull: { tags: "new" } }) // remove matching items from
array
// Result: { _id: 1, name: "Ana", points: 15, tags: ["vip"] }

// Upsert: create the document if no match exists
db.counters.updateOne(
  { page: "/home" }, // look for this page
  { $inc: { views: 1 } }, // add 1 to views
  { upsert: true } // not found? insert { page: "/home", views: 1 }
)
```

Query operators: \$eq, \$gt, \$in, \$and, \$or, \$exists, \$regex, \$elemMatch.

JUNIOR

In simple words: Operators let you filter by more than exact equality, like "greater than", "one of these", or "field exists".

```
// Sample products:
// { name: "Pen", price: 20, tags: ["office"], stock: [{ shop: "A", qty: 5 }] }
// { name: "Phone", price: 900, tags: ["tech", "sale"], stock: [{ shop: "B", qty: 0 }] }

db.products.find({ price: { $eq: 20 } }) // price equals 20 (same as { price: 20
})
db.products.find({ price: { $gt: 100 } }) // price greater than 100 -> Phone
db.products.find({ price: { $gte: 10, $lte: 50 } }) // between 10 and 50 -> Pen
db.products.find({ tags: { $in: ["sale", "promo"] } }) // has any of these tags -> Phone
db.products.find({ $or: [{ price: { $lt: 50 } }, { tags: "sale" }] }) // either condition ->
both
db.products.find({ discount: { $exists: false } }) // documents with no "discount" field
db.products.find({ name: { $regex: "^ph", $options: "i" } }) // name starts with "ph", any
case

// $elemMatch: ONE array item must match ALL conditions
db.products.find({ stock: { $elemMatch: { shop: "A", qty: { $gt: 0 } } } }) // -> Pen
```


Sorting and pagination: why is skip slow, and what is range (cursor) pagination?

MID

In simple words: `skip(n)` still walks past `n` documents every time, so page 5000 is slow. Range pagination remembers the last item you saw and asks for items after it, which uses an index and stays fast.

```
// Offset pagination (simple, gets slow on deep pages)
db.posts.find().sort({ createdAt: -1 }).skip(20).limit(10)    // page 3, 10 per page

// Range / cursor pagination (fast)
// Page 1:
db.posts.find().sort({ _id: -1 }).limit(10)
// Remember the _id of the last post on this page, e.g. lastId
// Next page: only posts older than lastId
db.posts.find({ _id: { $lt: lastId } }).sort({ _id: -1 }).limit(10)
```

Follow-up: make sure the sort field has an index, and add `_id` as a tie-breaker if the sort field is not unique.

Embedding vs referencing: how do you model one-to-few, one-to-many and many-to-many?

MID

In simple words: Embed data you always read together and that stays small. Reference (store an id) when the related data is large, grows without limit, or is shared by many documents.

- **One-to-few** (a user's 2–3 addresses): embed the array inside the user.
- **One-to-many** (a blog post with thousands of comments): put comments in their own collection with `postId`.
- **Many-to-many** (students and courses): store an array of ids on one or both sides, or a separate "enrollments" collection.

```
// Embed: addresses always shown with the user
{ _id: 1, name: "Ana", addresses: [{ city: "Pune" }, { city: "Goa" }] }

// Reference: comments can grow forever, so keep them separate
{ _id: 101, title: "Hello" }           // posts collection
{ _id: 9001, postId: 101, text: "Nice post!" } // comments collection

// Many-to-many with id arrays
{ _id: "s1", name: "Ana", courseIds: ["c1", "c2"] } // students
{ _id: "c1", title: "Databases" }                 // courses
```

Rule of thumb: "data that is accessed together should be stored together".

Name some schema design patterns: bucket, computed, subset, extended reference.

SENIOR

In simple words: These are common tricks to shape documents so reads are fast and documents don't grow too big.

- **Bucket:** group many small items into one document per time window, e.g. one document per sensor per hour holding 60 readings, instead of 60 tiny documents.
- **Computed:** store a pre-calculated value (like `totalReviews` or `avgRating`) and update it on write, so reads don't recalculate every time.
- **Subset:** keep only the most recent or most used items embedded (e.g. latest 10 reviews on the product), and the full list in another collection.
- **Extended reference:** copy a few frequently needed fields from a related document (e.g. customer name on the order) to avoid a `$lookup` on every read.

```
// Bucket example: one document per sensor per hour
{
  sensorId: "s1",
  hour: ISODate("2026-09-29T10:00:00Z"),
  count: 3,                                // how many readings are in this bucket
  readings: [ { m: 0, t: 21.5 }, { m: 1, t: 21.6 }, { m: 2, t: 21.4 } ]
}
```

MongoDB also has built-in **time series collections** that do bucketing for you.

How do you join collections with \$lookup?

MID

In simple words: `$lookup` is an aggregation stage that pulls matching documents from another collection into an array field, like a LEFT JOIN in SQL.

```
// orders:    { _id: 1, customerId: 10, total: 500 }
// customers: { _id: 10, name: "Ana" }

db.orders.aggregate([
  {
    $lookup: {
      from: "customers",           // the other collection
      localField: "customerId",    // field in orders
      foreignField: "_id",         // matching field in customers
      as: "customer"              // put the matches in this array
    }
  },
  { $unwind: "$customer" }        // turn the one-item array into a plain object
])
// Result: { _id: 1, customerId: 10, total: 500, customer: { _id: 10, name: "Ana" } }
```

Follow-up: make sure `foreignField` is indexed. If you need `$lookup` on every request, consider embedding or the extended reference pattern instead.

Explain the aggregation pipeline. Show total sales per category and the top 3 customers.

MID

In simple words: An aggregation pipeline is a list of steps. Each step takes documents in, changes them, and passes them to the next step, like an assembly line.

- `$match` filter (like WHERE), `$group` group and sum (like GROUP BY), `$project` choose/rename fields, `$sort`, `$limit`, `$unwind` split an array into one document per item, `$lookup` join, `$addFields` add computed fields, `$facet` run several pipelines at once.

```
// orders: { customer: "Ana", status: "paid", items: [ { category: "books", price: 200, qty: 2 } ] }
```

```
// Total sales per category
```

```
db.orders.aggregate([
  { $match: { status: "paid" } },           // only paid orders
  { $unwind: "$items" },                   // one document per item
  { $group: {
    _id: "$items.category",                // group by category
    total: { $sum: { $multiply: ["$items.price", "$items.qty"] } } // price x qty
  } },
  { $sort: { total: -1 } }                 // biggest first
])
// Result: { _id: "books", total: 400 }, ...
```

```
// Top 3 customers by amount spent
```

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $unwind: "$items" },
  { $group: {
    _id: "$customer",
    spent: { $sum: { $multiply: ["$items.price", "$items.qty"] } },
    orders: { $addToSet: "$_id" }           // collect unique order ids
  } },
  { $addFields: { orderCount: { $size: "$orders" } } }, // how many orders
  { $project: { orders: 0 } },              // hide the id list
  { $sort: { spent: -1 } },
  { $limit: 3 }
])
```

Tip: put `$match` as early as possible so the pipeline can use an index and handles fewer documents.

What does \$facet do? Build a search result with data and total count in one query.

SENIOR

In simple words: `$facet` runs several small pipelines on the same input at once and returns all results together. It's handy for "one page of results plus the total count" or filter counts in a sidebar.

```
db.products.aggregate([
  { $match: { category: "phones" } },           // shared filter
  { $facet: {
    data: [ { $sort: { price: 1 } }, { $skip: 0 }, { $limit: 10 } ], // page 1
    total: [ { $count: "count" } ],              // how many in total
    brands: [ { $group: { _id: "$brand", n: { $sum: 1 } } } ]        // counts per brand
  } }
])
// Result: { data: [...10 phones], total: [{ count: 42 }], brands: [{ _id: "Acme", n: 12 },
... ] }
```

What kinds of indexes does MongoDB have? What is the ESR rule?

MID

In simple words: An index is a sorted lookup list that lets MongoDB find documents without scanning the whole collection.

- **Single field:** `{ email: 1 }`
- **Compound:** several fields, e.g. `{ status: 1, createdAt: -1 }`
- **Multikey:** made automatically when you index an array field; one entry per array item.
- **Text:** word search on string fields (Atlas Search is better for real search).
- **TTL:** deletes documents automatically after a time.
- **Unique:** no two documents may have the same value.
- **Partial:** only indexes documents that match a filter (smaller and faster).

ESR rule for compound index field order: **E**quality fields first, then **S**ort fields, then **R**ange fields.

```
// Query: status = "paid", sorted by date, amount > 100
db.orders.find({ status: "paid", amount: { $gt: 100 } }).sort({ createdAt: -1 })

// ESR: Equality (status), Sort (createdAt), Range (amount)
db.orders.createIndex({ status: 1, createdAt: -1, amount: 1 })

db.users.createIndex({ email: 1 }, { unique: true }) // unique
db.orders.createIndex({ customerId: 1 }, { partialFilterExpression: { status: "open" } }) //
partial
db.articles.createIndex({ title: "text", body: "text" }) // text
```

How do you check if a query uses an index? COLLSCAN vs IXSCAN.

MID

In simple words: Add `.explain("executionStats")` to a query. **COLLSCAN** means it read every document (slow). **IXSCAN** means it used an index (fast).

```
db.orders.find({ customerId: 42 }).explain("executionStats")

// Look at these parts of the output:
// winningPlan.stage          -> "COLLSCAN" (bad) or "IXSCAN" inside "FETCH" (good)
// executionStats.nReturned   -> how many documents came back, e.g. 5
// executionStats.totalDocsExamined -> how many documents were read, e.g. 1,000,000
// executionStats.executionTimeMillis -> how long it took

// Fix a COLLSCAN by adding an index on the filtered field
db.orders.createIndex({ customerId: 1 })
// Now totalDocsExamined should be close to nReturned (e.g. 5 and 5)
```

Goal: docs examined $\approx$ docs returned. Also watch for an in-memory `SORT` stage, which means the sort isn't covered by the index.

What is a TTL index? Use it for sessions and password-reset tokens.

MID

In simple words: A TTL (time to live) index tells MongoDB to delete documents automatically once a date field is older than a set time. Perfect for sessions, OTPs and reset tokens.

```
// Option 1: delete 30 minutes after "createdAt"
db.reset_tokens.createIndex({ createdAt: 1 }, { expireAfterSeconds: 1800 })

// Option 2: store the exact expiry time and delete at that moment
db.sessions.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 })
db.sessions.insertOne({
  userId: ObjectId("66f1c2a9e4b0a1b2c3d4e5f6"),
  expiresAt: new Date(Date.now() + 14 * 24 * 3600 * 1000) // 14 days from now
})
```

Important: the background cleanup runs about once a minute, so documents can live a little past expiry. Always also check the expiry in your code.

Store users and password-reset tokens in MongoDB securely.

MID

In simple words: Keep emails unique with a unique index, store only a hash of the password and of the reset token, and let a TTL index delete old tokens automatically.

```
// users: one account per email
db.users.createIndex({ email: 1 }, { unique: true })
db.users.insertOne({
  email: "ana@example.com",           // store lowercased
  passwordHash: "$argon2id$v=19$...", // Argon2 hash, never the real password
  emailVerified: false,
  createdAt: new Date()
})

// reset_tokens: short-lived and single-use
db.reset_tokens.createIndex({ tokenHash: 1 }, { unique: true }) // fast lookup by hash
db.reset_tokens.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 }) // auto-delete when expired
db.reset_tokens.insertOne({
  userId: ObjectId("66f1c2a9e4b0a1b2c3d4e5f6"),
  tokenHash: "sha256-of-the-random-token", // the email gets the real token, the DB only the hash
  expiresAt: new Date(Date.now() + 30 * 60 * 1000) // 30 minutes
})

// When the user clicks the link: find the hash, check expiry, then delete it (single use)
db.reset_tokens.findOneAndDelete({ tokenHash: "sha256-of-the-random-token", expiresAt: { $gt: new Date() } })
```

Does MongoDB support transactions? When do you need them?

SENIOR

In simple words: A single-document update is always atomic, even if it changes many fields and arrays. For changes across several documents or collections that must all succeed or all fail, MongoDB has multi-document transactions (needs a replica set or sharded cluster).

```
// Transfer money between two accounts (mongosh)
const session = db.getMongo().startSession()
const accounts = session.getDatabase("bank").accounts
session.startTransaction()
try {
  accounts.updateOne({ _id: "A" }, { $inc: { balance: -100 } }) // take from A
  accounts.updateOne({ _id: "B" }, { $inc: { balance: 100 } }) // give to B
  session.commitTransaction() // save both together
} catch (err) {
  session.abortTransaction() // undo both on any error
  throw err
} finally {
  session.endSession()
}
```

Tip: transactions cost performance. Good schema design (embedding related data in one document) removes the need for most of them.

Explain write concern, read concern and read preference.

SENIOR

In simple words: These settings trade speed for safety in a replica set.

- **Write concern** = how many servers must confirm a write before it counts. `w: 1` only the primary (fast); `w: "majority"` most members (safe, survives failover; the default in modern versions). Add `j: true` to wait for the on-disk journal.
- **Read concern** = how "confirmed" the data you read must be. `"local"` newest data on that server (may be rolled back); `"majority"` only data confirmed by most members.
- **Read preference** = which server to read from. `primary` (default, always fresh), `primaryPreferred`, `secondary`, `secondaryPreferred`, `nearest` (lowest latency). Secondaries can be slightly behind.

```
db.orders.insertOne({ total: 500 }, { writeConcern: { w: "majority", j: true } })
// Connection string example:
// mongodb+srv://.../shop?readPreference=secondaryPreferred&w=majority
```

What is a replica set, and how does failover work?

SENIOR

In simple words: A replica set is a group of MongoDB servers holding the same data: one **primary** takes writes, and **secondaries** copy its changes. If the primary goes down, the others vote and promote a secondary to be the new primary automatically.

- Usually 3 members (an odd number so a majority can vote).
- Secondaries copy changes from the primary's **oplog** (a log of every write).
- An election typically finishes within about 10–12 seconds; drivers retry writes automatically (`retryWrites=true`).
- Writes with `w: "majority"` are never lost in a failover.

What is sharding, and how do you choose a shard key?

SENIOR

In simple words: Sharding splits one big collection across several servers (shards) so each holds part of the data. The **shard key** is the field MongoDB uses to decide which shard a document goes to.

- **Good shard key:** many different values (high cardinality), spreads writes evenly, and appears in most queries so MongoDB can go to one shard.
- **Bad:** a steadily increasing value like a timestamp or default ObjectId alone, which sends all new writes to one "hot" shard.
- **Options:** a hashed key for even spread (`{ userId: "hashed" }`), or a compound key like `{ tenantId: 1, createdAt: 1 }` .
- Queries without the shard key hit every shard ("scatter-gather"), which is slower.

```
sh.shardCollection("shop.orders", { customerId: "hashed" }) // spread orders evenly by
customer
```


How do you enforce a schema in MongoDB?

MID

In simple words: MongoDB is flexible by default, but you can add a `$jsonSchema` validator so the database rejects documents with missing or wrong fields. In apps you usually also validate with Mongoose or Pydantic.

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email", "passwordHash"],           // these fields must exist
      properties: {
        email: { bsonType: "string", pattern: "^.+@.+$" }, // must look like an email
        age: { bsonType: "int", minimum: 13 },           // whole number, 13 or more
        role: { enum: ["user", "admin"] }               // only these values allowed
      }
    }
  },
  validationAction: "error" // reject bad documents ("warn" only logs them)
})
```

What are change streams and when would you use them?

SENIOR

In simple words: A change stream lets your app listen to inserts, updates and deletes on a collection in real time, without polling. It is built on the replica set oplog.

- Uses: live notifications, syncing data to a search index or cache, audit logs, triggering emails when an order is created.
- Save the **resume token** so you can continue from where you stopped after a restart.

```
// Node.js
const stream = db.collection("orders").watch([
  { $match: { operationType: "insert" } } // only new orders
])
stream.on("change", (change) => {
  console.log("New order:", change.fullDocument) // the inserted document
  saveResumeToken(change._id)                    // remember where we are
})
```

Use Mongoose in a Next.js / Node API: schema, model, validation, populate, lean.

MID

In simple words: Mongoose adds schemas, validation and helper methods on top of the MongoDB driver for Node.js. `populate` fills in referenced documents; `lean()` returns plain objects, which is faster for read-only work.

```
// models/Post.js
import mongoose from "mongoose";

const postSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true, maxlength: 200 }, // validation rules
  author: { type: mongoose.Schema.Types.ObjectId, ref: "User" }, // reference to users
  tags: [String],
}, { timestamps: true }); // adds createdAt and updatedAt automatically

postSchema.index({ author: 1, createdAt: -1 }); // index for "posts by author, newest first"

// Reuse the model if it already exists (Next.js hot reload creates modules again)
export default mongoose.models.Post || mongoose.model("Post", postSchema);

// app/api/posts/route.js
import Post from "@models/Post";
import { connectDB } from "@lib/db"; // caches one connection across requests

export async function GET() {
  await connectDB();
  const posts = await Post.find()
    .sort({ createdAt: -1 })
    .limit(20)
    .populate("author", "name") // replace author id with { _id, name }
    .lean(); // plain JS objects, faster
  return Response.json(posts);
}
```

Use MongoDB from Python / FastAPI with PyMongo's async client (or Motor).

MID

In simple words: PyMongo is the official Python driver. For async FastAPI apps use its async client (`AsyncMongoClient` , PyMongo 4.9+); older projects use Motor, which works the same way. Create one client at startup and reuse it.

```
from fastapi import FastAPI, HTTPException
from pymongo import AsyncMongoClient
from bson import ObjectId

app = FastAPI()
client = AsyncMongoClient("mongodb://localhost:27017")  # one client for the whole app
db = client["shop"]  # pick the database

@app.get("/products/{product_id}")
async def get_product(product_id: str):
    if not ObjectId.is_valid(product_id):  # reject bad ids early
        raise HTTPException(400, "Invalid id")
    product = await db.products.find_one({"_id": ObjectId(product_id)})
    if product is None:
        raise HTTPException(404, "Product not found")
    product["_id"] = str(product["_id"])  # ObjectId is not JSON, convert to
    string
    return product

@app.post("/products", status_code=201)
async def create_product(body: dict):
    result = await db.products.insert_one(body)  # insert the new product
    return {"id": str(result.inserted_id)}  # send back the new id
```

Tip: in real code, validate `body` with a Pydantic model instead of a plain dict.

What are common MongoDB performance problems?

SENIOR

In simple words: Most slowness comes from missing indexes or documents that grow too big.

- **Missing indexes:** COLLSCAN on large collections. Check with `explain()` and the Atlas Performance Advisor.
- **Unbounded arrays:** e.g. pushing every comment into the post document forever. Documents get huge and slow to update. Use a separate collection or the subset pattern.
- **16 MB document limit:** a single document can't grow past 16 MB. Use GridFS or blob storage for files.
- **Huge \$in lists** (thousands of values) and large `skip()` values.
- **Returning whole documents** when you need 2 fields: use a projection.
- **Too many indexes:** every index slows writes and uses memory. Keep the working set (hot data + indexes) in RAM.
- **Regex that doesn't start with ^** can't use an index well.

What is NoSQL injection and how do you prevent it?

MID

In simple words: If you put user input straight into a query, an attacker can send a query operator instead of a value. For example, a login body with `{"$ne": null}` as the password matches every user.

```
// Attacker sends this JSON body to /login:
// { "email": "admin@site.com", "password": { "$ne": null } }

// VULNERABLE: the object becomes a query operator
db.users.findOne({ email: req.body.email, password: req.body.password }) // matches admin!

// SAFE: make sure inputs are plain strings, then compare the hash in code
const email = String(req.body.email)
const password = String(req.body.password)
const user = await db.users.findOne({ email: email }) // look up by email only
const ok = user && await argon2.verify(user.passwordHash, password) // check password in code
```

- Validate request bodies with Zod / Joi / Pydantic (types, not just presence).
- Use Mongoose schemas with strict query filtering (`sanitizeFilter`), or strip keys starting with `$`.
- Never pass user input to `$where` or `$function`.

MongoDB Atlas vs Azure Cosmos DB for MongoDB (vCore): what's the difference?

MID

In simple words: Atlas is MongoDB's own managed cloud service and runs on Azure, AWS or GCP. Azure Cosmos DB for MongoDB is Microsoft's service that speaks the MongoDB protocol, so most MongoDB drivers and code work with it.

- **Atlas:** real MongoDB engine with all the newest features, Atlas Search and Vector Search, Performance Advisor, and can be deployed in Azure regions.
- **Cosmos DB for MongoDB (vCore):** billed per cluster like a normal database, fits well with Azure networking, Entra ID and billing, and supports vector search. It is compatible with most, but not every, MongoDB feature and version.
- **Cosmos DB for MongoDB (RU-based):** the older serverless-style option billed by request units, with more compatibility limits.

Interview answer: check feature compatibility (aggregation stages, transactions, change streams) for your app, then compare cost and how the team already works on Azure.

Redis

Data types, expiry and eviction, persistence, caching patterns, sessions, rate limiting, distributed locks, Pub/Sub and Streams, clustering, Azure Cache for Redis.

What is Redis and why is it so fast?

JUNIOR

In simple words: Redis is a key-value database that keeps all its data in memory (RAM), so reads and writes take well under a millisecond. It is mostly used as a cache, a session store, a rate limiter and a lightweight message broker.

- **In memory:** no disk seek on reads; disk is only used for backups (persistence).
- **Single-threaded command execution:** one main thread runs commands one after another, so there are no locks between commands and every command is atomic.
- **I/O threads (Redis 6+):** extra threads can read and write network data, while commands still run on the main thread.
- **Simple data structures** and an efficient event loop (epoll) handling thousands of connections.

Redis data types: String, Hash, List, Set, Sorted Set (with redis-cli examples).

JUNIOR

In simple words: Each key holds one type of value. Pick the type that matches how you read the data.

```
# String: text, numbers, JSON
SET user:1:name "Ana"           # save a value
GET user:1:name                 # "Ana"
INCR page:views                 # add 1 -> (integer) 1

# Hash: an object with fields (like a small dict)
HSET user:1 name "Ana" age 30   # set two fields
HGET user:1 name                # "Ana"
HGETALL user:1                  # 1) "name" 2) "Ana" 3) "age" 4) "30"

# List: ordered, allows duplicates (queue / recent items)
LPUSH recent:1 "post:9"        # add to the front
LRANGE recent:1 0 9             # first 10 items

# Set: unique items, no order
SADD tags:post:9 "python" "redis" # add two tags
SISMEMBER tags:post:9 "python"    # (integer) 1 = yes

# Sorted Set: unique items, each with a score, kept sorted by score
ZADD scores 150 "ana" 90 "bo"    # member with score
ZREVRANGE scores 0 2 WITHSCORES  # top 3, highest first
```

What are Streams, Bitmaps, HyperLogLog and Geo in Redis?

MID

In simple words: Special data types for specific jobs.

```
# Stream: an append-only log of events, readable by consumer groups
XADD orders * id 42 total 99.5   # add an event, Redis makes the ID
XGROUP CREATE orders workers $ MKSTREAM # create a consumer group
XREADGROUP GROUP workers w1 COUNT 10 STREAMS orders > # read new events
XACK orders workers 1700000000000-0 # mark an event as done

# Bitmap: one bit per user per day (very small)
SETBIT login:2026-09-29 1001 1   # user 1001 logged in today
BITCOUNT login:2026-09-29      # how many users logged in

# HyperLogLog: approximate count of unique items in ~12 KB
PFADD visitors:today "ip1" "ip2" "ip1" # add visitors
PFCOUNT visitors:today                 # (integer) 2

# Geo: store coordinates, search nearby
GEOADD shops 72.8777 19.0760 "mumbai-1" # longitude, latitude, name
GEOSEARCH shops FROMLONLAT 72.88 19.07 BYRADIUS 5 km ASC # shops within 5 km
```

How does key expiry work (EXPIRE, TTL, SET EX NX) and how are expired keys removed?

JUNIOR

In simple words: You can give any key a time to live. After it runs out, the key is gone.

```
SET otp:ana "482913" EX 300      # save for 300 seconds (5 minutes)
TTL otp:ana                      # seconds left, e.g. (integer) 287
EXPIRE session:abc 1800          # add a 30-minute expiry to an existing key
PERSIST session:abc              # remove the expiry
SET lock:job1 "worker-7" NX EX 30 # set ONLY if the key does not exist
TTL missing:key                  # (integer) -2 = key does not exist
TTL no:expiry:key                 # (integer) -1 = key exists, no expiry
```

Redis removes expired keys in two ways:

- **Lazily:** when you try to read a key, Redis checks if it expired and deletes it.
- **Actively:** a background job regularly samples keys that have an expiry and deletes the expired ones.

Follow-up: `SET` without `KEEPTTL` removes an existing expiry.

What happens when Redis runs out of memory? Explain eviction policies.

MID

In simple words: You set a memory limit with `maxmemory`. When Redis reaches it, the `maxmemory-policy` decides what to delete to make room, or whether to refuse writes.

- `noeviction` (default): refuse new writes with an error. Good for data you must not lose.
- `allkeys-lru`: delete the least recently used key. Best default for a pure cache.
- `allkeys-lfu`: delete the least frequently used key.
- `volatile-lru` / `volatile-lfu` / `volatile-ttl`: only delete keys that have an expiry.
- `allkeys-random` / `volatile-random`: delete a random key.

```
CONFIG SET maxmemory 2gb          # memory limit
CONFIG SET maxmemory-policy allkeys-lru # evict least recently used keys
```

Redis uses an approximate LRU: it samples a few keys and evicts the oldest of the sample.

Redis persistence: RDB vs AOF vs both?

MID

In simple words: Redis lives in RAM, so it saves to disk to survive a restart.

- **RDB (snapshots):** saves the whole dataset every so often. Small files, fast restart, but you can lose the writes since the last snapshot.
- **AOF (append-only file):** logs every write. With `appendfsync everysec` you lose at most about 1 second of data. Bigger files; Redis rewrites the file in the background to shrink it.
- **Both:** the usual choice when Redis holds data you care about. On restart Redis loads the AOF because it is more complete.
- **None:** fine for a pure cache that can be rebuilt from the database.

```
save 900 1           # RDB: snapshot if 1 change in 900 seconds
appendonly yes       # turn on AOF
appendfsync everysec # flush AOF to disk once a second
```


Explain cache-aside with a Python example. How do you invalidate the cache?

In simple words: Check Redis first. If the data is missing, load it from the database, save it in Redis with an expiry, and return it. When the data changes, delete the cache key so the next read loads fresh data.

```
import json
import redis

r = redis.Redis(host="localhost", port=6379, decode_responses=True)

def get_user(user_id):
    key = f"user:{user_id}"          # one key per user
    cached = r.get(key)              # 1. try the cache
    if cached is not None:
        return json.loads(cached)    # cache hit: return it
    user = db_get_user(user_id)      # 2. cache miss: read the database
    r.set(key, json.dumps(user), ex=600) # 3. save for 10 minutes
    return user

def update_user(user_id, data):
    db_update_user(user_id, data)    # 1. write to the database first
    r.delete(f"user:{user_id}")      # 2. then delete the cached copy
```

- **Write-through:** write to the cache and the database together on every update. Reads are always warm, writes are slower.
- **Write-behind (write-back):** write to the cache, and a background job saves to the database later. Fast, but you can lose data if Redis fails.

Why delete instead of update the cache? Two updates racing can leave an old value in the cache. Deleting is simpler, and the TTL is your safety net.

What is a cache stampede (thundering herd) and how do you prevent it?

SENIOR

In simple words: A popular key expires, and hundreds of requests all miss the cache at the same moment and hit the database together, which can take it down.

- **Lock so only one request rebuilds:** the first request takes a short lock with `SET lock:key 1 NX EX 10`; the others wait briefly and read the cache again.
- **Add jitter to TTLs:** `ex=600 + random.randint(0, 60)` so keys don't all expire together.
- **Stale-while-revalidate:** keep serving the old value while one background job refreshes it.
- **Pre-warm** very hot keys before they expire.

```
import random, time

def get_with_lock(key, load_fn):
    value = r.get(key)                # try the cache
    if value is not None:
        return value
    got_lock = r.set("lock:" + key, "1", nx=True, ex=10)  # only one winner
    if got_lock:
        value = load_fn()              # winner loads from the DB
        r.set(key, value, ex=600 + random.randint(0, 60)) # TTL with jitter
        r.delete("lock:" + key)        # release the lock
        return value
    time.sleep(0.05)                  # others wait a little
    return r.get(key) or load_fn()     # then read the fresh cache
```

What are cache penetration and cache avalanche?

MID

In simple words:

- **Cache penetration:** requests for data that doesn't exist (e.g. random IDs) always miss the cache and always hit the database. Fix: cache the "not found" result for a short time, validate IDs early, or use a Bloom filter.
- **Cache avalanche:** many keys expire at the same time, or Redis goes down, so all traffic hits the database. Fix: random jitter on TTLs, a replica / high-availability setup, and rate limiting or circuit breakers in front of the database.
- **Cache stampede** is the one-hot-key version of this (see the previous question).

```
user = db_get_user(user_id)
if user is None:
    r.set(f"user:{user_id}", "NULL", ex=60)  # remember "missing" for 1 minute
```

How do you store login sessions in Redis?

MID

In simple words: After login, create a random session ID, store `session:ID → user data` in Redis with an expiry, and put only the ID in an HttpOnly cookie. Every request looks the session up; logout deletes the key.

```
import json, secrets

SESSION_TTL = 60 * 60 * 24          # 1 day in seconds

def create_session(user_id):
    session_id = secrets.token_urlsafe(32)    # long random ID, impossible to guess
    data = json.dumps({"user_id": user_id})
    r.set("session:" + session_id, data, ex=SESSION_TTL)    # save with expiry
    r.sadd("user_sessions:" + str(user_id), session_id)    # track all sessions of this user
    return session_id                                     # goes into an HttpOnly cookie

def get_session(session_id):
    data = r.get("session:" + session_id)    # look it up on every request
    if data is None:
        return None                            # expired or logged out
    r.expire("session:" + session_id, SESSION_TTL)    # sliding expiry: extend on activity
    return json.loads(data)

def logout(session_id):
    r.delete("session:" + session_id)          # the session is gone instantly

def logout_everywhere(user_id):
    key = "user_sessions:" + str(user_id)    # e.g. after a password reset
    for session_id in r.smembers(key):
        r.delete("session:" + session_id)
    r.delete(key)
```

This is why server sessions can be revoked instantly, unlike plain JWTs.

How do you use Redis to revoke JWTs or keep a token denylist?

MID

In simple words: When a user logs out, save the token's unique ID (`jti`) in Redis until the token would expire anyway. On every request, check that the ID is not in the denylist.

```
import time

def revoke_token(payload):
    jti = payload["jti"] # unique ID inside the JWT
    seconds_left = payload["exp"] - int(time.time()) # time until it expires
    if seconds_left > 0:
        r.set("revoked:" + jti, "1", ex=seconds_left) # auto-deletes when useless

def is_revoked(payload):
    return r.exists("revoked:" + payload["jti"]) == 1 # 1 = revoked

# Refresh tokens: store them (hashed) with a TTL, and delete on use (rotation)
r.set("refresh:" + token_hash, user_id, ex=14 * 24 * 3600)
```

Because each entry expires with the token, the denylist never grows forever.

Build a rate limiter with Redis (fixed window and sliding window).

In simple words: Count requests per user or IP in Redis. If the count goes over the limit, return HTTP 429.

```
import time, uuid

# Fixed window: "max 5 login attempts per minute"
def allow_fixed(ip, limit=5, window=60):
    key = "rl:login:" + ip + ":" + str(int(time.time() // window)) # one key per minute
    count = r.incr(key) # add 1, returns the new count
    if count == 1:
        r.expire(key, window) # first hit: make the key expire
    return count <= limit # True = allowed

# Sliding window: counts requests in the LAST 60 seconds (smoother)
def allow_sliding(ip, limit=5, window=60):
    key = "rl:sliding:" + ip
    now = time.time()
    pipe = r.pipeline() # send all commands together
    pipe.zremrangebyscore(key, 0, now - window) # drop hits older than the window
    pipe.zadd(key, {str(uuid.uuid4()): now}) # record this hit (score = time)
    pipe.zcard(key) # count hits in the window
    pipe.expire(key, window) # clean up idle keys
    results = pipe.execute()
    count = results[2] # the ZCARD result
    return count <= limit
```

Trade-off: fixed window allows a burst of 2x the limit around the boundary (end of one minute + start of the next). Sliding window avoids that but uses more memory.

How do you build a distributed lock with Redis? What are the caveats?

SENIOR

In simple words: Use `SET key value NX PX` : it only succeeds if nobody holds the lock, and the expiry frees the lock if the holder crashes. Store a random token so you only ever delete **your own** lock.

```
import uuid

RELEASE_SCRIPT = """
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
else
    return 0
end
"""

def acquire(name, ttl_ms=10000):
    token = str(uuid.uuid4()) # my unique token
    ok = r.set("lock:" + name, token, nx=True, px=ttl_ms) # only if free
    return token if ok else None

def release(name, token):
    # check-and-delete in ONE atomic step, so we never delete someone else's lock
    return r.eval(RELEASE_SCRIPT, 1, "lock:" + name, token)
```

- **Why not just DEL?** If your work took longer than the TTL, the lock expired and another worker took it; a plain DEL would remove their lock.
- **Redlock** (locking across several independent Redis nodes) is debated: pauses (GC, network) can still let two clients think they hold the lock. For correctness-critical work, use fencing tokens or a database lock (e.g. Postgres `SELECT ... FOR UPDATE`).

Pub/Sub vs Streams vs a real message queue: which one?

MID

In simple words:

- **Pub/Sub:** fire-and-forget. Subscribers that are offline miss the message; nothing is stored. Good for live notifications, chat typing indicators, cache-invalidation broadcasts.
- **Streams:** messages are stored, consumer groups share work, and `XACK` confirms processing, so crashed workers can be retried. Good for lightweight job queues and event logs.
- **A real broker** (Azure Service Bus, RabbitMQ, Kafka): use it when you need dead-letter queues, guaranteed delivery at large scale, long retention, complex routing, or cross-team integration.

```
SUBSCRIBE notifications # client 1 listens
PUBLISH notifications "order 42 shipped" # client 2 sends; returns number of listeners
```

Build a game leaderboard with a sorted set.

JUNIOR

In simple words: A sorted set keeps members ordered by score automatically, so top-N and "what is my rank" are single fast commands.

```
ZADD leaderboard 1200 "ana"      # add player with score
ZADD leaderboard 950 "bo" 1500 "cy" # add two more
ZINCRBY leaderboard 100 "bo"      # bo scores 100 more -> 1050
ZREVRANGE leaderboard 0 2 WITHSCORES # top 3: cy 1500, ana 1200, bo 1050
ZREVRANK leaderboard "ana"        # (integer) 1 (0 = first place)
ZSCORE leaderboard "ana"          # "1200"
```

```
# Python
r.zadd("leaderboard", {"ana": 1200})      # add or update a score
r.zincrby("leaderboard", 100, "bo")       # add points
top3 = r.zrevrange("leaderboard", 0, 2, withscores=True) # [('cy', 1500.0), ...]
```

Adding and ranking are $O(\log n)$, even with millions of players.

Count unique visitors per day cheaply.

MID

In simple words: A Set gives the exact count but stores every visitor, which gets large. HyperLogLog gives an estimate (about 0.81% error) in at most ~12 KB, no matter how many visitors.

```
PFADD uv:2026-09-29 "user:1" "user:2" "user:1" # add visitors (duplicates ignored)
PFCOUNT uv:2026-09-29                          # (integer) 2
PFMERGE uv:week uv:2026-09-28 uv:2026-09-29    # combine days into a week
PFCOUNT uv:week                                 # unique visitors this week
```

Use a Set when you need the exact list of IDs, HyperLogLog when you only need the number.

Pipelining vs MULTI/EXEC transactions vs WATCH.

MID

In simple words:

- **Pipelining:** send many commands in one network round trip. Faster, but not atomic: other clients' commands can run in between.
- **MULTI/EXEC:** queue commands and run them all together, with nothing else running in between. There is **no rollback**: if one command fails at run time, the others still run.
- **WATCH:** optimistic locking. If a watched key changes before EXEC, the whole transaction is cancelled and you retry.

```
# Python: move 10 credits between users only if the balance didn't change meanwhile
with r.pipeline() as pipe:
    while True:
        try:
            pipe.watch("credits:ana")                # watch the key
            balance = int(pipe.get("credits:ana"))    # read current value
            if balance < 10:
                raise ValueError("not enough credits")
            pipe.multi()                             # start the transaction
            pipe.decrby("credits:ana", 10)
            pipe.incrby("credits:bo", 10)
            pipe.execute()                           # fails if credits:ana changed
            break                                    # success
        except redis.WatchError:
            continue                                # someone changed it: retry
```

What are Lua scripts in Redis and when would you use one?

SENIOR

In simple words: A Lua script runs on the Redis server as one atomic step: no other command runs in the middle. Use it for "read, decide, then write" logic that must not be interrupted, like the safe lock release or a rate limiter.

```
# Python: increment a counter but never above a maximum
SCRIPT = """
local current = tonumber(redis.call('GET', KEYS[1]) or '0')
if current < tonumber(ARGV[1]) then
    return redis.call('INCR', KEYS[1])
end
return -1
"""

limited_incr = r.register_script(SCRIPT)  # loaded once, called by its hash
print(limited_incr(keys=["seats:show42"], args=[100]))  # 1, 2, ... then -1 when full
```

Caveat: keep scripts short; a slow script blocks every other client. Redis 7 also adds **Functions**, stored scripts that survive restarts.

Replication, Sentinel and Redis Cluster: what's the difference?

SENIOR

In simple words:

- **Replication:** one primary takes writes and copies data to replicas (asynchronously). Replicas can serve reads and act as backups.
- **Sentinel:** watches the primary; if it dies, Sentinel promotes a replica to primary and tells clients the new address. High availability, but all data still fits on one node.
- **Redis Cluster:** splits data across several primaries (sharding). Keys are mapped to 16,384 **hash slots** (CRC16 of the key mod 16384), and each primary owns a range of slots, each with its own replicas.

```
# Multi-key commands in a cluster only work if all keys are in the same slot.  
# A "hash tag" in braces forces keys into the same slot:  
SET {user:1}:profile "..."  
SET {user:1}:cart "..."      # same slot as above, because only "user:1" is hashed
```

Note: replication is async, so a failover can lose the last few writes.

Common Redis mistakes in production?

MID

- **KEYS *** scans every key and blocks the server. Use **SCAN**, which walks keys in small batches.
- **Big keys:** a hash or list with millions of items is slow to read, delete or replicate. Split it up; delete big keys with **UNLINK** (non-blocking).
- **Hot keys:** one key hit by all traffic overloads one node. Add a small in-app cache or split the key.
- **No TTLs on cache keys:** memory fills up forever.
- **Using Redis as the only copy** of important data without persistence or replicas.
- **Opening a new connection per request** instead of reusing a connection pool.

```
SCAN 0 MATCH session:* COUNT 100    # returns a cursor + up to ~100 keys  
SCAN 17 MATCH session:* COUNT 100   # continue from the returned cursor until it is 0  
MEMORY USAGE user:1                  # bytes used by one key  
redis-cli --bigkeys                  # find the biggest keys
```

Use Redis from FastAPI with `redis.asyncio`.

MID

In simple words: Create one async client when the app starts, share it through a dependency, and close it on shutdown. Never create a client per request.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, Depends, HTTPException, Request
import redis.asyncio as redis

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = redis.from_url("redis://localhost:6379/0", decode_responses=True) # one
    shared pool
    yield
    await app.state.redis.aclose() # close when the app stops

app = FastAPI(lifespan=lifespan)

def get_redis(request: Request):
    return request.app.state.redis # hand the shared client to routes

@app.get("/products/{product_id}")
async def product(product_id: int, r = Depends(get_redis)):
    key = f"product:{product_id}"
    cached = await r.get(key) # async call, doesn't block the event loop
    if cached:
        return {"source": "cache", "data": cached}
    data = await load_product(product_id) # your DB call
    if data is None:
        raise HTTPException(404, "Product not found")
    await r.set(key, data, ex=300) # cache for 5 minutes
    return {"source": "db", "data": data}
```

Use Redis from Next.js / Node with ioredis.

MID

In simple words: Create one client and reuse it. In Next.js dev mode, files reload often, so keep the client on `globalThis` to avoid opening a new connection on every reload. Only use it in server code (Server Components, Route Handlers, Server Actions).

```
// lib/redis.ts
import Redis from 'ioredis';

const globalForRedis = globalThis;
export const redis = globalForRedis.redis || new Redis(process.env.REDIS_URL); // reuse if it exists
if (process.env.NODE_ENV !== 'production') globalForRedis.redis = redis;

// app/api/views/[slug]/route.ts
import { redis } from '@lib/redis';
export async function POST(req, { params }) {
  const { slug } = await params;
  const views = await redis.incr('views:' + slug); // count a page view
  return Response.json({ views });
}
```

On serverless/edge platforms, an HTTP-based Redis client is often used instead of long-lived TCP connections.

Azure Cache for Redis / Azure Managed Redis: what should you know?

MID

In simple words: Azure runs Redis for you: patching, replicas, failover and backups.

- **Azure Cache for Redis** tiers: Basic (single node, no SLA, dev only), Standard (primary + replica), Premium (clustering, persistence, VNet, geo-replication), Enterprise tiers (Redis Enterprise modules).
- **Azure Managed Redis** is the newer offering built on Redis Enterprise; Microsoft recommends it for new projects. Check the current docs for tier names and retirement dates.
- **TLS:** Azure Cache for Redis uses port **6380** for TLS; the non-TLS port 6379 is disabled by default. Azure Managed Redis uses port 10000.
- **Auth:** prefer **Microsoft Entra ID** with a managed identity over access keys; otherwise keep keys in Key Vault.
- Use **Private Endpoints** so Redis isn't reachable from the public internet.

```
# Python connection to Azure Cache for Redis with TLS
r = redis.Redis(host="myapp.redis.cache.windows.net", port=6380,
                ssl=True, password=access_key_from_key_vault,
                decode_responses=True)
```

What happened with Redis licensing, and what is Valkey?

SENIOR

In simple words: Redis was BSD-licensed open source. In 2024 (Redis 7.4) it moved to source-available licenses (RSALv2 / SSPLv1), which restrict offering Redis as a managed service. In response, the Linux Foundation started **Valkey**, a BSD-licensed fork of Redis 7.2 backed by several cloud providers. In 2025, **Redis 8** added **AGPLv3** as an open-source license option.

- For app developers using Redis as a cache or session store, day-to-day usage and client libraries are basically the same.
- Managed services (Azure, AWS, Google Cloud) handle licensing for you; some now offer Valkey.
- In an interview, just show you're aware of it and that the API is compatible for common use.

Databases & APIs

SQL, indexing, transactions, REST vs GraphQL, caching.

SQL vs NoSQL: when to use each?

JUNIOR

SQL (PostgreSQL, Azure SQL) for relational data, joins and ACID transactions (orders, users, payments). NoSQL (Cosmos DB, MongoDB, Redis) for flexible schemas, huge scale with simple access patterns, caching, or document data. Many systems use both.

What are REST principles and common HTTP status codes?

JUNIOR

Resources as nouns (`/users/42`), HTTP verbs for actions, stateless requests. Codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthenticated, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Validation error, 429 Too Many Requests, 500 Server error.

PUT vs PATCH? Which methods are idempotent?

JUNIOR

PUT replaces the whole resource; PATCH changes some fields. GET, PUT, DELETE are idempotent (repeating gives the same result); POST is not, so use idempotency keys for payments.

What is an index and when does it hurt?

MID

A B-tree (usually) that speeds lookups, sorts and joins on the indexed columns. It costs storage and slows writes. Composite indexes follow the leftmost-prefix rule. Check with `EXPLAIN ANALYZE`.

Explain ACID and transaction isolation levels.

MID

Atomicity, Consistency, Isolation, Durability. Isolation levels trade safety for concurrency: Read Uncommitted, Read Committed (Postgres default), Repeatable Read, Serializable. Know dirty reads, non-repeatable reads, phantom reads.

Types of JOIN?

MID

INNER (matches only), LEFT (all left + matches), RIGHT, FULL OUTER, CROSS (cartesian). Be ready to write a query like "users with no orders" using LEFT JOIN ... WHERE o.id IS NULL.

REST vs GraphQL vs tRPC?

MID

REST: simple, cacheable, standard. GraphQL: client picks fields, one endpoint, avoids over/under-fetching, harder caching and N+1 risk on the server. tRPC: end-to-end types in TypeScript monorepos only. With a Python backend, REST + OpenAPI-generated types is the usual pick.

How do you paginate an API?

MID

Offset (`?page=3`) is simple but slow on deep pages and shifts when rows are inserted.
Cursor/keyset (`WHERE id > :last_id ORDER BY id LIMIT 20`) is stable and fast for infinite scroll.

Normalization vs denormalization?

SENIOR

Normalize (1NF–3NF) to remove duplication and keep writes consistent. Denormalize selectively for read-heavy paths (counters, read models, materialized views) and accept the cost of keeping copies in sync.

How do you handle database migrations with zero downtime?

SENIOR

Expand and contract: add the new column (nullable), deploy code that writes both, backfill, switch reads, then remove the old column in a later release. Avoid long table locks; create indexes
`CONCURRENTLY` in Postgres.

Auth Architecture

Signup, login, email verification, password reset, sessions vs JWT, OAuth, MFA. Frequently asked as a system design round.

[See the signup, login and reset-password diagrams](#)

Authentication vs authorization?

JUNIOR

Authentication (AuthN) proves who you are (login). Authorization (AuthZ) decides what you may do (roles, permissions). A 401 means "not authenticated"; a 403 means "authenticated but not allowed".

How should passwords be stored?

JUNIOR

Never plaintext or plain SHA/MD5. Use a slow, salted, adaptive hash: **Argon2id** (preferred) or bcrypt. The salt is unique per user and stored with the hash. Compare in constant time.

```
from argon2 import PasswordHasher
ph = PasswordHasher()
hash = ph.hash(password)
ph.verify(hash, attempt) # raises on mismatch
```

Design the signup and email verification flow.

MID

1. Validate email + password strength (check against breached-password lists).
2. Create user with `email_verified=false`, hashed password.
3. Generate a random token (32 bytes, `secrets.token_urlsafe`), store **its hash** with expiry (e.g. 24h).
4. Queue email with link `/verify?token=...` (async via a job queue).
5. On click: hash the token, look it up, check expiry, mark verified, delete token.
6. Allow resend with rate limits.

Follow-up: if the email already exists, show the same message ("check your inbox") to prevent account enumeration.

Design a secure forgot / reset password flow.

MID

1. User enters email. Always respond "If an account exists, we sent a link" (no enumeration).
2. Generate a single-use, random, high-entropy token; store only its hash, expire in 15–60 minutes.
3. Email link over HTTPS. Rate-limit requests per email and IP.
4. On submit: verify token hash and expiry, set new hashed password, delete the token (and all other reset tokens).
5. **Revoke all existing sessions / refresh tokens** for that user.
6. Send a "your password was changed" notification email.

Common mistakes: predictable tokens, tokens that never expire, reusable tokens, leaking token in Referer headers (set `Referrer-Policy: no-referrer` on the reset page), using a host header to build the link.

Session cookies vs JWT: trade-offs?

MID

- **Server sessions:** random session id in an httpOnly cookie, data in Redis/DB. Instant revocation, simple; needs a shared store.
- **JWT:** signed token carrying claims; stateless verification, good across services. Hard to revoke before expiry, larger, easy to misuse.
- Common hybrid: short-lived JWT access token (5–15 min) + long-lived refresh token stored server-side and rotated.

What is inside a JWT and how is it verified?

MID

`header.payload.signature`, base64url-encoded. Header has `alg / kid`; payload has claims (`sub`, `exp`, `iat`, `iss`, `aud`, roles). The payload is **readable by anyone**; don't put secrets in it. Verify signature with the expected algorithm (reject `alg: none`), then check `exp`, `iss`, `aud`. HS256 uses a shared secret; RS256/ES256 use a private key to sign and a public key (JWKS) to verify.

Where should tokens be stored in the browser?

MID

Refresh token (or session id) in an `HttpOnly; Secure; SameSite=Lax/Strict` cookie so JS can't read it. Access token in memory. Avoid localStorage (XSS can read it). If using cookies, protect state-changing requests against CSRF (SameSite + CSRF token or checking `Origin`).

Explain refresh token rotation and reuse detection.

SENIOR

Each time a refresh token is used, issue a new one and invalidate the old one. All tokens from one login form a "family". If an old (already used) token shows up again, someone stole it: revoke the whole family and force re-login. Store refresh tokens hashed with device info and expiry.

Explain OAuth 2.0 Authorization Code flow with PKCE and OpenID Connect.

SENIOR

1. App creates `code_verifier` and its SHA-256 `code_challenge`.
2. Redirect user to the provider (Google, Entra ID) with `client_id`, `redirect_uri`, `scope`, `state`, `code_challenge`.
3. User logs in and consents; provider redirects back with a `code`.
4. App's backend exchanges code + `code_verifier` for tokens.
5. OIDC adds an **ID token** (who the user is); OAuth's access token is for calling APIs.

Follow-up: `state` prevents CSRF on the callback; PKCE stops intercepted codes from being used. Implicit flow is deprecated.

How do you protect login against brute force and credential stuffing?

SENIOR

- Rate limit per IP and per account (sliding window in Redis), progressive delays.
- Temporary lockout or CAPTCHA after repeated failures (careful: lockouts can be abused for DoS).
- Check passwords against breached lists (HaveIBeenPwned k-anonymity API).
- MFA, anomaly detection (new device/location email), generic error "Invalid email or password".

How would you add MFA and passkeys?

SENIOR

- **TOTP:** store an encrypted shared secret, show QR code, verify 6-digit codes with a small time window; give one-time recovery codes (stored hashed).
- **Passkeys / WebAuthn:** device creates a key pair; server stores the public key and verifies signed challenges. Phishing-resistant, no shared secret.
- SMS OTP is the weakest option (SIM swapping).

Design RBAC for a multi-tenant SaaS.

SENIOR

Tables: users, tenants, memberships (user, tenant, role), roles → permissions. Every query is scoped by `tenant_id` (or Postgres row-level security). Check permissions server-side in a single policy function, not in UI code. For complex rules consider ABAC or tools like OpenFGA/Cerbos.

Azure

App Service, Functions, Entra ID, Key Vault, storage, DevOps.

IaaS vs PaaS vs SaaS with Azure examples?

JUNIOR

IaaS: Azure Virtual Machines (you manage OS). PaaS: App Service, Azure SQL, Functions (Azure manages OS and runtime). SaaS: Microsoft 365. Full stack apps usually run on PaaS.

3. Issues short-lived access token + rotated refresh token (httpOnly cookie): records session

What is Azure App Service?

JUNIOR

Managed hosting for web apps and APIs (Python, Node, .NET, containers). Provides autoscaling, deployment slots, custom domains, TLS, built-in auth ("Easy Auth"), and CI/CD integration.

6. Audit log for logins, resets and MFA changes

What are resource groups, subscriptions and regions?

JUNIOR

A subscription is a billing and access boundary. Resource groups are logical containers for resources that share a lifecycle (delete the group, delete everything). Regions are the physical datacenter locations; pick close to users and pair for DR.

Below the resource group level, you can create resource groups and then create resource groups within

What are deployment slots and why use them?

MID

Separate live environments (e.g. `staging`) of the same App Service. Deploy to staging, warm it up, test, then **swap** with production for zero-downtime release and instant rollback by swapping back.

Azure Functions: triggers, bindings, and hosting plans?

MID

Serverless functions started by triggers (HTTP, Timer, Queue, Service Bus, Blob, Event Grid, Cosmos DB). Bindings connect inputs/outputs declaratively. Plans: Consumption / Flex Consumption (pay per execution, cold starts), Premium (pre-warmed, VNet), Dedicated (App Service plan). Durable Functions add stateful workflows.

What is Microsoft Entra ID (Azure AD) and how does it relate to app login?

MID

Azure's identity service: users, groups, app registrations, and OAuth 2.0 / OpenID Connect tokens. Use it for employee SSO and API protection. For customer-facing login (sign-up, social logins, password reset) use **Entra External ID** (successor to Azure AD B2C). In code, use MSAL (`msal-browser`, `msal` for Python) or Auth.js with the Entra provider.

What is a Managed Identity? System vs user-assigned?

MID

An identity in Entra ID that Azure manages for a resource, so code can access Key Vault, Storage or SQL **without any secret** (`DefaultAzureCredential`). System-assigned lives and dies with one resource; user-assigned is standalone and can be shared across resources.

How do you manage secrets on Azure?

MID

Store in Azure Key Vault; grant access with managed identity + RBAC; reference them in App Service settings as `@Microsoft.KeyVault(SecretUri=...)`. Never commit secrets; rotate them; enable soft-delete and purge protection.

Blob Storage tiers and SAS tokens?

MID

Tiers: Hot, Cool, Cold, Archive (cheaper storage, costlier access). A SAS (Shared Access Signature) grants time-limited, scoped access to a blob, e.g. let the browser upload a file directly without going through your API. Prefer user-delegation SAS (signed with Entra credentials).

Service Bus vs Storage Queues vs Event Grid vs Event Hubs?

MID

- **Storage Queue:** simple, cheap, large backlog.
- **Service Bus:** enterprise messaging: FIFO sessions, topics/subscriptions, dead-lettering, transactions.
- **Event Grid:** reactive event routing (blob created → function).
- **Event Hubs:** high-throughput streaming/telemetry.

How would you architect a Next.js + Python app on Azure?

SENIOR

- Front Door (CDN + WAF) → Next.js on App Service / Container Apps.
- FastAPI on Container Apps or App Service, autoscaled.
- Azure Database for PostgreSQL / Azure SQL; Azure Cache for Redis for sessions and rate limits.
- Blob Storage for uploads; Service Bus + Functions for emails and background jobs; Azure Communication Services or SendGrid for email.
- Entra External ID for customer auth; Key Vault + managed identity for secrets.
- Application Insights + Log Analytics for monitoring; private endpoints + VNet for DB.
- Bicep/Terraform for infra; GitHub Actions or Azure DevOps for CI/CD.

How do you set up CI/CD to Azure?

SENIOR

GitHub Actions or Azure Pipelines: lint + test → build container → push to Azure Container Registry → deploy to staging slot → smoke tests → swap to production. Authenticate with OIDC federated credentials (no stored secrets). Infra as code with Bicep or Terraform in the same pipeline.

How do you monitor and troubleshoot an Azure app?

SENIOR

Application Insights (requests, dependencies, exceptions, distributed tracing via OpenTelemetry), Log Analytics with KQL queries, alerts on failure rate and latency, availability tests, App Service diagnostics and log stream.

Azure RBAC vs Entra ID roles?

SENIOR

System Design & DevOps

Azure RBAC controls access to Azure resources (Contributor on a resource group) with scope inheritance: management group → subscription → resource group → resource. Entra ID roles

How would you scale a web application?

MID

Stateless app servers behind a load balancer (scale horizontally), CDN for static assets, caching (Redis) for hot reads, database read replicas and indexing, queues for slow work, and later sharding or splitting services where there is a real bottleneck.

Caching strategies and invalidation?

MID

Cache-aside (app reads cache, falls back to DB, then fills cache), write-through, write-behind. Invalidate by TTL, by explicit delete on write, or by versioned keys. Browser/CDN caching via Cache-Control, ETag. Watch for cache stampedes (use locks or stale-while-revalidate).

Monolith vs microservices?

MID

Start with a well-structured modular monolith: simpler deploys, transactions and debugging. Split out services when teams or scaling needs differ; microservices add network failures, distributed data and ops overhead.

How do you implement rate limiting?

MID

Token bucket or sliding window counters in Redis, keyed by user/IP/API key. Return 429 with Retry-After. Apply at the gateway (Front Door, API Management) and in the app for sensitive endpoints like login and password reset.

Design a URL shortener / notification system / file upload service.

SENIOR

Use a repeatable structure:

1. Clarify requirements and scale (users, QPS, read/write ratio).
2. API design and data model.
3. High-level diagram: client, CDN, LB, services, DB, cache, queue.
4. Deep dive into the hardest part (ID generation, fan-out, large uploads via presigned/SAS URLs).
5. Bottlenecks, failure modes, monitoring, trade-offs.

What is the CAP theorem?

SENIOR

During a network partition a distributed store must choose Consistency (reject or block) or Availability (serve possibly stale data). Without partitions you trade latency vs consistency (PACELC). Cosmos DB exposes five consistency levels for this reason.

How do you make a system observable?

SENIOR

Structured logs with a correlation/request id, metrics (RED: rate, errors, duration), distributed traces with OpenTelemetry, dashboards and SLO-based alerts. Log auth events without logging passwords or tokens.

What is idempotency and why does it matter for APIs and queues?

SENIOR

Repeating an operation has the same effect as doing it once. Queues deliver at-least-once and clients retry on timeouts, so handlers must dedupe (idempotency key stored with the result, unique constraints, upserts).

What happens when you type a URL and press Enter?

JUNIOR

DNS lookup → TCP + TLS handshake → HTTP request → server/CDN responds → browser parses HTML, builds DOM and CSSOM, runs JS, lays out, paints. Mention caching, HTTP/2 or HTTP/3, and where SSR fits.

Web Security

XSS, CSRF, CORS, SSI injection, headers. Docker, basics: image vs container, multi-stage builds?

MID

What is XSS and how do you prevent it?

JUNIOR

Cross-Site Scripting: attacker-controlled script runs in your page. React escapes text by default; danger comes from `dangerouslySetInnerHTML`, `href="javascript:..."`, and unsanitized HTML. Prevent with output encoding, DOMPurify for user HTML, a Content-Security-Policy, and httpOnly cookies.

What is CSRF and how do you prevent it?

MID

Cross-Site Request Forgery: another site makes the victim's browser send an authenticated request with their cookies. Prevent with `SameSite=Lax/Strict` cookies, CSRF tokens, checking `Origin`, and never changing state on GET. Next.js Server Actions check Origin automatically.

What is CORS?

MID

A browser rule that blocks JS from reading responses from another origin unless the server allows it with `Access-Control-Allow-Origin`. Non-simple requests send an OPTIONS preflight. CORS is not a security wall for your API (curl ignores it); it protects users' browsers. Never use `*` with credentials.

How do you prevent SQL injection?

MID

Parameterized queries or an ORM; never build SQL with string formatting. Validate input, least-privilege DB user.

```
cur.execute("SELECT * FROM users WHERE email = %s", (email,))
```

What are the OWASP Top 10?

MID

Broken access control, cryptographic failures, injection, insecure design, security misconfiguration, vulnerable components, identification and authentication failures, software and data integrity failures, logging and monitoring failures, SSRF. Broken access control is number one: always check ownership on the server (`WHERE id=:id AND user_id=:me`).

Which security headers do you set?

SENIOR

`Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `X-Frame-Options / frame-ancestors`, `Permissions-Policy`. Set in `next.config.js` headers or middleware.

What is SSRF?

SENIOR

Server-Side Request Forgery: the attacker makes your server fetch a URL they choose, e.g. an internal service or cloud metadata endpoint (`169.254.169.254`). Prevent by allow-listing domains, blocking private IP ranges after DNS resolution, and isolating fetchers.

DSA: Strings, Arrays, Lists

The data structures and algorithms problems asked most often in full stack rounds, with Python and JavaScript solutions and complexity.

Reverse a string and check if it is a palindrome (ignore case and non-letters).

JUNIOR

Idea: To reverse, build a new string by adding each character to the front. For the palindrome check, keep only letters and digits in lowercase, then compare the first character with the last, the second with the second-last, and so on.

Time: $O(n)$ because we look at each character once. **Space:** $O(n)$ because we build a cleaned copy of the string.

```
# Python
def reverse_string(text):
    reversed_text = ""                # start with an empty string
    for ch in text:                  # go through each character
        reversed_text = ch + reversed_text  # put it in front of what we have
    return reversed_text

def is_palindrome(text):
    cleaned = ""                     # will hold only letters and digits
    for ch in text:
        if ch.isalnum():             # skip spaces, commas, etc.
            cleaned = cleaned + ch.lower()  # store it in lowercase
    left = 0                          # pointer at the start
    right = len(cleaned) - 1          # pointer at the end
    while left < right:               # stop when the pointers meet
        if cleaned[left] != cleaned[right]: # a mismatch means not a palindrome
            return False
        left = left + 1              # move start pointer forward
        right = right - 1            # move end pointer backward
    return True                       # every pair matched

print(reverse_string("hello"))        # olleh
print(is_palindrome("A man, a plan, a canal: Panama")) # True

# Shorter way (good to mention in the interview):
# reversed_text = text[::-1]
# cleaned == cleaned[::-1]
```

```
// JavaScript
function reverseString(text) {
    return text.split('').reverse().join(''); // split, reverse, join back
}

function isPalindrome(text) {
    const cleaned = text.toLowerCase().replace(/[^\a-z0-9]/g, ''); // keep letters/digits
    let left = 0;
    let right = cleaned.length - 1;
    while (left < right) {
        if (cleaned[left] !== cleaned[right]) return false;
        left++;
        right--;
    }
}
```

```
    return true;
}
```

Valid anagram: are two strings anagrams?

JUNIOR

Idea: Two words are anagrams if they use exactly the same letters the same number of times. Count the letters of the first word, then subtract for each letter of the second word. If every count ends at zero, they match.

Time: $O(n)$ because each string is read once. **Space:** $O(1)$ for English letters (at most 26 keys).

```
# Python
def is_anagram(first, second):
    if len(first) != len(second):      # different lengths can never match
        return False
    count = {}                         # letter -> how many times it appears
    for ch in first:
        count[ch] = count.get(ch, 0) + 1  # add 1 for each letter in first
    for ch in second:
        if count.get(ch, 0) == 0:        # letter missing or used up
            return False
        count[ch] = count[ch] - 1        # use up one of that letter
    return True

print(is_anagram("listen", "silent"))    # True
print(is_anagram("rat", "car"))          # False

# Shorter way (good to mention in the interview):
# from collections import Counter
# return Counter(first) == Counter(second)
```

```
// JavaScript
function isAnagram(first, second) {
    if (first.length !== second.length) return false;
    const count = {};
    for (const ch of first) {
        count[ch] = (count[ch] || 0) + 1;  // count letters of first word
    }
    for (const ch of second) {
        if (!count[ch]) return false;      // missing or used up
        count[ch]--;
    }
    return true;
}
```


First non-repeating character in a string.

Idea: First count how many times each character appears. Then walk through the string again from the start and return the first character whose count is 1.

Time: $O(n)$ because we loop over the string twice. **Space:** $O(k)$ for the counts, where k is the number of different characters.

```
# Python
def first_unique(text):
    count = {}                                # character -> times seen
    for ch in text:
        count[ch] = count.get(ch, 0) + 1
    for ch in text:                            # second pass keeps the original order
        if count[ch] == 1:                    # appears only once
            return ch
    return None                                # every character repeats

print(first_unique("swiss"))    # w

# Shorter way (good to mention in the interview):
# from collections import Counter
# count = Counter(text)
```

```
// JavaScript
function firstUnique(text) {
    const count = new Map();
    for (const ch of text) {
        count.set(ch, (count.get(ch) || 0) + 1);
    }
    for (const ch of text) {
        if (count.get(ch) === 1) return ch;
    }
    return null;
}
```

Count character / word frequency; find the most frequent character.

JUNIOR

Idea: Use a dictionary as a tally sheet: each time you see a character, add 1 to its count. Keep track of the character with the highest count as you go.

Time: $O(n)$ because each character is visited once. **Space:** $O(k)$ for the dictionary of different characters.

```
# Python
def most_frequent(text):
    count = {}                    # character -> times seen
    best_char = ""               # the winner so far
    best_count = 0               # how many times the winner appeared
    for ch in text:
        count[ch] = count.get(ch, 0) + 1    # add one to this character
        if count[ch] > best_count:         # new leader?
            best_count = count[ch]
            best_char = ch
    return best_char

def word_frequency(sentence):
    count = {}
    for word in sentence.split():          # split on spaces into words
        count[word] = count.get(word, 0) + 1
    return count

print(most_frequent("banana"))            # a
print(word_frequency("to be or not to be")) # {'to': 2, 'be': 2, 'or': 1, 'not': 1}

# Shorter way (good to mention in the interview):
# from collections import Counter
# Counter("banana").most_common(1)      # [('a', 3)]
```

```
// JavaScript
function mostFrequent(text) {
    const count = {};
    let bestChar = '';
    let bestCount = 0;
    for (const ch of text) {
        count[ch] = (count[ch] || 0) + 1;
        if (count[ch] > bestCount) {
            bestCount = count[ch];
            bestChar = ch;
        }
    }
    return bestChar;
}
```

Two Sum: return indices of two numbers adding to a target.

JUNIOR

Idea: Walk through the list once. For each number, work out the partner it needs (target minus the number). If we've already seen that partner, we're done. Otherwise remember this number and its position in a dictionary.

Time: $O(n)$ because we pass through the list once and dictionary lookups are instant. **Space:** $O(n)$ for the dictionary.

```
# Python
def two_sum(nums, target):
    seen = {} # number -> its index
    for i in range(len(nums)):
        current = nums[i]
        needed = target - current # the partner we are looking for
        if needed in seen: # did we see the partner earlier?
            return [seen[needed], i] # return both positions
        seen[current] = i # remember this number for later
    return [] # no pair found

print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
```

```
// JavaScript
function twoSum(nums, target) {
    const seen = new Map(); // number -> index
    for (let i = 0; i < nums.length; i++) {
        const needed = target - nums[i];
        if (seen.has(needed)) {
            return [seen.get(needed), i];
        }
        seen.set(nums[i], i);
    }
    return [];
}
```

Remove duplicates from an array (keep order); remove duplicates in place from a sorted array.

Idea: Keep a set of numbers already added. Only add a number to the result if it's not in the set yet. For a sorted list, duplicates sit next to each other, so we copy a number forward only when it differs from the last kept one.

Time: $O(n)$ for both. **Space:** $O(n)$ for the first (the set), $O(1)$ for the in-place version.

```
# Python
def remove_duplicates(nums):
    seen = set()                # numbers we already kept
    result = []                 # the answer, in original order
    for num in nums:
        if num not in seen:     # first time we see it
            result.append(num)
            seen.add(num)
    return result

def dedupe_sorted(nums):
    if len(nums) == 0:
        return 0
    write = 1                   # next position to write a new unique number
    for read in range(1, len(nums)): # look at every number after the first
        if nums[read] != nums[write - 1]: # different from the last kept number
            nums[write] = nums[read]      # keep it
            write = write + 1
    return write                 # count of unique numbers at the front

print(remove_duplicates([3, 1, 3, 2, 1])) # [3, 1, 2]
data = [1, 1, 2, 3, 3]
print(dedupe_sorted(data), data[:3])     # 3 [1, 2, 3]

# Shorter way (good to mention in the interview):
# list(dict.fromkeys(nums))
```

```
// JavaScript
function removeDuplicates(nums) {
    return [...new Set(nums)]; // a Set keeps only unique values, in order
}
```

Find the largest and second largest numbers in one pass.

JUNIOR

Idea: Keep two variables: the largest and the second largest so far. When a new number beats the largest, the old largest drops to second place. If it only beats second place (and isn't equal to the largest), it becomes the new second.

Time: $O(n)$, one pass. **Space:** $O(1)$, just two variables.

```
# Python
def top_two(nums):
    largest = float("-inf")          # smaller than any real number
    second = float("-inf")
    for num in nums:
        if num > largest:             # new champion
            second = largest          # old champion moves to second place
            largest = num
        elif num > second and num != largest:  # only beats second place
            second = num
    return largest, second

print(top_two([10, 20, 4, 45, 99, 99]))  # (99, 45)
```

```
// JavaScript
function topTwo(nums) {
    let largest = -Infinity;
    let second = -Infinity;
    for (const num of nums) {
        if (num > largest) {
            second = largest;
            largest = num;
        } else if (num > second && num !== largest) {
            second = num;
        }
    }
    return [largest, second];
}
```

Move all zeroes to the end, keeping order of others.

Idea: Keep a pointer called `write` for where the next non-zero number should go. Walk through the list; every time you find a non-zero, put it at `write` and move `write` forward. At the end, fill the rest with zeroes.

Time: $O(n)$. **Space:** $O(1)$ because we change the list in place.

```
# Python
def move_zeroes(nums):
    write = 0                                # where the next non-zero goes
    for num in nums:
        if num != 0:
            nums[write] = num                # place non-zero at the front part
            write = write + 1
    while write < len(nums):                  # everything after that becomes 0
        nums[write] = 0
        write = write + 1
    return nums

print(move_zeroes([0, 1, 0, 3, 12]))        # [1, 3, 12, 0, 0]
```

```
// JavaScript
function moveZeroes(nums) {
    let write = 0;
    for (const num of nums) {
        if (num !== 0) {
            nums[write] = num;
            write++;
        }
    }
    while (write < nums.length) {
        nums[write] = 0;
        write++;
    }
    return nums;
}
```

Longest substring without repeating characters.

Idea: Use a "window" with a left and right edge. Move the right edge one character at a time. If the new character is already inside the window, move the left edge past its previous position. Track the biggest window size you ever see.

Time: $O(n)$ because each character is visited once. **Space:** $O(k)$ for the dictionary of last positions.

```
# Python
def longest_unique(text):
    last_seen = {}                # character -> last index where we saw it
    left = 0                      # start of the current window
    best = 0                      # longest window found so far
    for right in range(len(text)): # right is the end of the window
        ch = text[right]
        if ch in last_seen and last_seen[ch] >= left: # repeat inside window
            left = last_seen[ch] + 1 # shrink window to just after the repeat
        last_seen[ch] = right       # update where we last saw this char
        window_size = right - left + 1
        if window_size > best:
            best = window_size
    return best

print(longest_unique("abcabcbb")) # 3 ("abc")
```

```
// JavaScript
function longestUnique(text) {
    const lastSeen = new Map();
    let left = 0;
    let best = 0;
    for (let right = 0; right < text.length; right++) {
        const ch = text[right];
        if (lastSeen.has(ch) && lastSeen.get(ch) >= left) {
            left = lastSeen.get(ch) + 1;
        }
        lastSeen.set(ch, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Valid parentheses: `()[]{}` balanced?

MID

Idea: Use a stack (a list where you add and remove from the end). Push every opening bracket. When you see a closing bracket, the top of the stack must be its matching opener. At the end the stack must be empty.

Time: $O(n)$. **Space:** $O(n)$ for the stack in the worst case.

```
# Python
def is_valid(text):
    matching = {")": "(", "]": "[", "}": "{"} # closer -> its opener
    stack = []
    for ch in text:
        if ch in "([{":
            # an opening bracket
            stack.append(ch)
        else:
            # a closing bracket
            if len(stack) == 0:
                # nothing to close
                return False
            top = stack.pop()
            if top != matching[ch]:
                # wrong type of bracket
                return False
    return len(stack) == 0 # leftover openers mean unbalanced

print(is_valid("([]{})")) # True
print(is_valid("()"))     # False
```

```
// JavaScript
function isValid(text) {
    const matching = { ')': '(', ']' : '[', '}' : '{' };
    const stack = [];
    for (const ch of text) {
        if ('([{'.includes(ch)) {
            stack.push(ch);
        } else {
            const top = stack.pop();
            if (top !== matching[ch]) return false;
        }
    }
    return stack.length === 0;
}
```


Group anagrams together.

MID

Idea: Anagrams become the same word when their letters are sorted ("eat", "tea" → "aet"). Use that sorted word as a dictionary key and put every word into the list for its key.

Time: $O(n \cdot k \log k)$ where k is word length (sorting each word). **Space:** $O(n \cdot k)$ to store the groups.

```
# Python
def group_anagrams(words):
    groups = {}                                # sorted letters -> list of words
    for word in words:
        key = "".join(sorted(word))           # "tea" -> "aet"
        if key not in groups:
            groups[key] = []                   # start a new group
        groups[key].append(word)
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

# Shorter way (good to mention in the interview):
# from collections import defaultdict
# groups = defaultdict(list)  # no need to check "if key not in groups"
```

```
// JavaScript
function groupAnagrams(words) {
    const groups = new Map();
    for (const word of words) {
        const key = word.split('').sort().join('');
        if (!groups.has(key)) groups.set(key, []);
        groups.get(key).push(word);
    }
    return [...groups.values()];
}
```

Longest common prefix of an array of strings.

MID

Idea: Take the first word as the starting guess. Check every other word; while the word doesn't start with the guess, chop the last letter off the guess.

Time: $O(\text{total characters})$ in the worst case. **Space:** $O(1)$ extra.

```
# Python
def longest_common_prefix(words):
    if len(words) == 0:
        return ""
    prefix = words[0]                # start with the whole first word
    for word in words[1:]:           # compare with every other word
        while not word.startswith(prefix): # prefix doesn't fit this word
            prefix = prefix[:-1]        # remove the last letter
        if prefix == "":
            return ""                  # nothing in common
    return prefix

print(longest_common_prefix(["flower", "flow", "flight"])) # fl
```

```
// JavaScript
function longestCommonPrefix(words) {
    if (words.length === 0) return '';
    let prefix = words[0];
    for (const word of words) {
        while (!word.startsWith(prefix)) {
            prefix = prefix.slice(0, -1); // drop the last letter
        }
    }
    return prefix;
}
```

String compression: `aaabbc` → `a3b2c1` .

MID

Idea: Walk through the string and count how many times the same character repeats in a row. When the character changes, write the character and its count. If the result isn't shorter, return the original.

Time: $O(n)$. **Space:** $O(n)$ for the output.

```
# Python
def compress(text):
    if text == "":
        return ""
    result = ""
    current = text[0]                # the character we are counting
    count = 1                        # how many in a row so far
    for ch in text[1:]:
        if ch == current:            # same character continues
            count = count + 1
        else:                        # character changed
            result = result + current + str(count)
            current = ch              # start counting the new one
            count = 1
    result = result + current + str(count) # add the last group
    if len(result) < len(text):
        return result
    return text                      # compression didn't help

print(compress("aaabbc")) # a3b2c1 is 6 long, not shorter, so prints aaabbc
print(compress("aaaaabb")) # a5b2
```

```
// JavaScript
function compress(text) {
    let result = '';
    let i = 0;
    while (i < text.length) {
        let j = i;
        while (j < text.length && text[j] === text[i]) j++; // find end of run
        result += text[i] + (j - i);
        i = j;
    }
    return result.length < text.length ? result : text;
}
```

Maximum subarray sum (Kadane's algorithm).

MID

Idea: Walk through the list keeping a running sum. At each number ask: "is it better to keep adding to my running sum, or start fresh from this number?" Remember the best sum ever seen.

Time: $O(n)$, one pass. **Space:** $O(1)$.

```
# Python
def max_subarray(nums):
    current_sum = nums[0]          # best sum ending at this position
    best = nums[0]                 # best sum anywhere so far
    for num in nums[1:]:
        if current_sum + num > num: # adding helps: keep extending
            current_sum = current_sum + num
        else:                       # old sum is dragging us down: start over
            current_sum = num
        if current_sum > best:
            best = current_sum
    return best

print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])) # 6 (4 + -1 + 2 + 1)
```

```
// JavaScript
function maxSubarray(nums) {
    let currentSum = nums[0];
    let best = nums[0];
    for (let i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]); // extend or restart
        best = Math.max(best, currentSum);
    }
    return best;
}
```

Best time to buy and sell stock (one transaction).

MID

Idea: Go day by day. Remember the cheapest price seen so far (the best day to have bought). On each day, check how much profit you'd make selling today, and keep the biggest profit.

Time: $O(n)$. **Space:** $O(1)$.

```
# Python
def max_profit(prices):
    cheapest = float("inf")          # lowest price seen so far
    best_profit = 0
    for price in prices:
        if price < cheapest:         # found a cheaper day to buy
            cheapest = price
        profit = price - cheapest    # profit if we sell today
        if profit > best_profit:
            best_profit = profit
    return best_profit

print(max_profit([7, 1, 5, 3, 6, 4])) # 5 (buy at 1, sell at 6)
```

```
// JavaScript
function maxProfit(prices) {
    let cheapest = Infinity;
    let bestProfit = 0;
    for (const price of prices) {
        cheapest = Math.min(cheapest, price);
        bestProfit = Math.max(bestProfit, price - cheapest);
    }
    return bestProfit;
}
```

Product of array except self (no division).

MID

Idea: For each position, the answer is (product of everything to its left) $\times$ (product of everything to its right). First pass left-to-right fills in the left products. Second pass right-to-left multiplies in the right products.

Time: $O(n)$, two passes. **Space:** $O(1)$ extra besides the answer list.

```
# Python
def product_except_self(nums):
    n = len(nums)
    answer = [1] * n           # start every slot at 1

    left_product = 1           # product of numbers to the left
    for i in range(n):
        answer[i] = left_product # store left product for position i
        left_product = left_product * nums[i]

    right_product = 1          # product of numbers to the right
    for i in range(n - 1, -1, -1): # walk backwards
        answer[i] = answer[i] * right_product
        right_product = right_product * nums[i]

    return answer

print(product_except_self([1, 2, 3, 4])) # [24, 12, 8, 6]
```

```
// JavaScript
function productExceptSelf(nums) {
    const answer = new Array(nums.length).fill(1);
    let leftProduct = 1;
    for (let i = 0; i < nums.length; i++) {
        answer[i] = leftProduct;
        leftProduct *= nums[i];
    }
    let rightProduct = 1;
    for (let i = nums.length - 1; i >= 0; i--) {
        answer[i] *= rightProduct;
        rightProduct *= nums[i];
    }
    return answer;
}
```

Rotate an array right by k steps in place.

MID

Idea: A neat trick: reverse the whole list, then reverse the first k items, then reverse the rest.

Example with k=2: [1,2,3,4,5] → [5,4,3,2,1] → [4,5,3,2,1] → [4,5,1,2,3].

Time: O(n). **Space:** O(1) because we only swap inside the list.

```
# Python
def reverse_part(nums, start, end):
    while start < end:
        # swap from both ends toward the middle
        temp = nums[start]
        nums[start] = nums[end]
        nums[end] = temp
        start = start + 1
        end = end - 1

def rotate(nums, k):
    n = len(nums)
    k = k % n
    # rotating by n does nothing
    reverse_part(nums, 0, n - 1) # 1. reverse everything
    reverse_part(nums, 0, k - 1) # 2. reverse the first k
    reverse_part(nums, k, n - 1) # 3. reverse the rest
    return nums

print(rotate([1, 2, 3, 4, 5], 2)) # [4, 5, 1, 2, 3]

# Shorter way (good to mention in the interview, uses extra space):
# nums[:] = nums[-k:] + nums[:-k]
```

```
// JavaScript
function reversePart(nums, start, end) {
    while (start < end) {
        const temp = nums[start];
        nums[start] = nums[end];
        nums[end] = temp;
        start++;
        end--;
    }
}

function rotate(nums, k) {
    k = k % nums.length;
    reversePart(nums, 0, nums.length - 1);
    reversePart(nums, 0, k - 1);
    reversePart(nums, k, nums.length - 1);
    return nums;
}
```

Merge overlapping intervals.

MID

Idea: Sort the intervals by their start. Then go through them: if the current interval starts before the last merged one ends, they overlap, so stretch the last one's end. Otherwise add it as a new interval.

Time: $O(n \log n)$ because of sorting. **Space:** $O(n)$ for the result.

```
# Python
def merge_intervals(intervals):
    intervals.sort()          # sort by start time
    merged = []
    for interval in intervals:
        start = interval[0]
        end = interval[1]
        if len(merged) > 0 and start <= merged[-1][1]: # overlaps the last one
            if end > merged[-1][1]:
                merged[-1][1] = end          # stretch the last interval
        else:
            merged.append([start, end]) # no overlap: new interval
    return merged

print(merge_intervals([[1, 3], [8, 10], [2, 6], [15, 18]]))
# [[1, 6], [8, 10], [15, 18]]
```

```
// JavaScript
function mergeIntervals(intervals) {
    intervals.sort((a, b) => a[0] - b[0]); // sort by start
    const merged = [];
    for (const [start, end] of intervals) {
        const last = merged[merged.length - 1];
        if (last && start <= last[1]) {
            last[1] = Math.max(last[1], end); // overlap: extend
        } else {
            merged.push([start, end]);
        }
    }
    return merged;
}
```


Merge two sorted arrays into one sorted array.

MID

Idea: Put one finger at the start of each list. Compare the two numbers under your fingers, take the smaller one, and move that finger forward. When one list runs out, add whatever is left of the other.

Time: $O(n + m)$. **Space:** $O(n + m)$ for the result.

```
# Python
def merge_sorted(first, second):
    i = 0                                # finger in the first list
    j = 0                                # finger in the second list
    result = []
    while i < len(first) and j < len(second):
        if first[i] <= second[j]:        # take the smaller number
            result.append(first[i])
            i = i + 1
        else:
            result.append(second[j])
            j = j + 1
    while i < len(first):                 # leftovers from the first list
        result.append(first[i])
        i = i + 1
    while j < len(second):               # leftovers from the second list
        result.append(second[j])
        j = j + 1
    return result

print(merge_sorted([1, 3, 5], [2, 4, 6, 8])) # [1, 2, 3, 4, 5, 6, 8]
```

```
// JavaScript
function mergeSorted(first, second) {
    const result = [];
    let i = 0;
    let j = 0;
    while (i < first.length && j < second.length) {
        if (first[i] <= second[j]) {
            result.push(first[i]);
            i++;
        } else {
            result.push(second[j]);
            j++;
        }
    }
    return result.concat(first.slice(i), second.slice(j)); // add leftovers
}
```

Top K frequent elements.

MID

Idea: Count how often each number appears. Then sort the numbers by their count, biggest first, and take the first k.

Time: $O(n \log n)$ because of sorting. **Space:** $O(n)$ for the counts.

```
# Python
def top_k_frequent(nums, k):
    count = {}                                # number -> how many times
    for num in nums:
        count[num] = count.get(num, 0) + 1

    def get_count(num):                       # helper: tells sort what to compare
        return count[num]

    by_frequency = sorted(count, key=get_count, reverse=True)  # most common first
    return by_frequency[:k]                  # take the first k

print(top_k_frequent([1, 1, 1, 2, 2, 3], 2))  # [1, 2]

# Shorter way (good to mention in the interview):
# from collections import Counter
# [num for num, _ in Counter(nums).most_common(k)]
```

```
// JavaScript
function topKFrequent(nums, k) {
    const count = new Map();
    for (const num of nums) {
        count.set(num, (count.get(num) || 0) + 1);
    }
    const byFrequency = [...count.keys()].sort((a, b) => count.get(b) - count.get(a));
    return byFrequency.slice(0, k);
}
```

3Sum: all unique triplets that sum to zero.

MID

Idea: Sort the list. Fix one number, then use two pointers (one just after it, one at the end) to find pairs that cancel it out. If the sum is too small, move the left pointer right; too big, move the right pointer left. Skip repeated numbers to avoid duplicate triplets.

Time: $O(n^2)$. **Space:** $O(1)$ extra besides the result.

```
# Python
def three_sum(nums):
    nums.sort()
    result = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:    # same fixed number as before: skip
            continue
        left = i + 1
        right = len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left = left + 1                # need a bigger sum
            elif total > 0:
                right = right - 1              # need a smaller sum
            else:
                result.append([nums[i], nums[left], nums[right]])
                left = left + 1
                right = right - 1
                while left < right and nums[left] == nums[left - 1]:
                    left = left + 1            # skip duplicate values
    return result

print(three_sum([-1, 0, 1, 2, -1, -4]))    # [[-1, -1, 2], [-1, 0, 1]]
```

Binary search, and find first/last position of a target.

MID

Idea: In a sorted list, look at the middle item. If it's the target, done. If the target is bigger, throw away the left half; if smaller, throw away the right half. Repeat. To find the first position, don't stop on a match: remember it and keep searching the left half.

Time: $O(\log n)$ because the search space halves each step. **Space:** $O(1)$.

```
# Python
def binary_search(nums, target):
    low = 0
    high = len(nums) - 1
    while low <= high:
        mid = (low + high) // 2          # middle index
        if nums[mid] == target:
            return mid                  # found it
        elif nums[mid] < target:
            low = mid + 1               # target is in the right half
        else:
            high = mid - 1              # target is in the left half
    return -1                          # not in the list

def first_position(nums, target):
    low = 0
    high = len(nums) - 1
    answer = -1
    while low <= high:
        mid = (low + high) // 2
        if nums[mid] == target:
            answer = mid                # remember it...
            high = mid - 1              # ...but keep looking further left
        elif nums[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return answer

print(binary_search([1, 3, 5, 7, 9], 7))    # 3
print(first_position([1, 2, 2, 2, 3], 2))   # 1
# For the last position: on a match, set low = mid + 1 instead.
```

```
// JavaScript
function binarySearch(nums, target) {
    let low = 0;
    let high = nums.length - 1;
    while (low <= high) {
        const mid = Math.floor((low + high) / 2);
        if (nums[mid] === target) return mid;
        if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
}
```

```
return -1;  
}
```

Reverse a singly linked list (iterative and recursive).

MID

Idea: Walk through the list and flip each arrow so it points backwards. You need three names: `previous` (already reversed part), `current` (the node you're on) and `next_node` (saved so you don't lose the rest of the list).

Time: $O(n)$. **Space:** $O(1)$ for the loop version, $O(n)$ for recursion (call stack).

```
# Python
class Node:
    def __init__(self, value, next=None):
        self.value = value          # the data
        self.next = next           # arrow to the next node

def reverse_list(head):
    previous = None                # nothing is reversed yet
    current = head                 # start at the first node
    while current is not None:
        next_node = current.next   # 1. save the rest of the list
        current.next = previous    # 2. flip the arrow backwards
        previous = current         # 3. move previous one step forward
        current = next_node        # 4. move current one step forward
    return previous               # previous is the new head

def reverse_recursive(head):
    if head is None or head.next is None: # empty or one node: already reversed
        return head
    new_head = reverse_recursive(head.next) # reverse everything after head
    head.next.next = head                 # make the next node point back to head
    head.next = None                     # head is now the last node
    return new_head

def to_list(head):                    # helper to print a linked list
    values = []
    while head is not None:
        values.append(head.value)
        head = head.next
    return values

head = Node(1, Node(2, Node(3)))
print(to_list(reverse_list(head)))    # [3, 2, 1]
```

```
// JavaScript
function reverseList(head) {
    let previous = null;
    let current = head;
    while (current) {
        const nextNode = current.next; // save the rest
        current.next = previous;        // flip the arrow
        previous = current;
        current = nextNode;
    }
}
```

```
    return previous;
}
```

Detect a cycle in a linked list; find the middle node.

MID

Idea: Use two runners: `slow` moves one step, `fast` moves two steps. If there's a loop, fast eventually laps slow and they meet. If fast reaches the end, there's no loop. The same trick finds the middle: when fast reaches the end, slow is halfway.

Time: $O(n)$. **Space:** $O(1)$.

```
# Python
def has_cycle(head):
    slow = head
    fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next           # one step
        fast = fast.next.next      # two steps
        if slow is fast:           # they met: there is a loop
            return True
    return False                  # fast hit the end: no loop

def middle_node(head):
    slow = head
    fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
    return slow                  # slow is in the middle

head = Node(1, Node(2, Node(3, Node(4, Node(5))))) # Node class from the reverse question
print(has_cycle(head))           # False
print(middle_node(head).value)   # 3
```

Merge two sorted linked lists.

MID

Idea: Same as merging two sorted arrays, but with nodes. Start with a dummy node so you always have something to attach to. Repeatedly attach the smaller front node, then attach whatever is left at the end.

Time: $O(n + m)$. **Space:** $O(1)$ because we reuse the existing nodes.

```
# Python
def merge_lists(first, second):
    dummy = Node(0)                # fake start node, makes the code simpler
    tail = dummy                   # last node of the merged list
    while first is not None and second is not None:
        if first.value <= second.value:
            tail.next = first       # attach the smaller node
            first = first.next
        else:
            tail.next = second
            second = second.next
        tail = tail.next           # move tail to the node we just attached
    if first is not None:          # attach leftovers
        tail.next = first
    else:
        tail.next = second
    return dummy.next              # skip the fake start node

a = Node(1, Node(3, Node(5)))
b = Node(2, Node(4))
print(to_list(merge_lists(a, b))) # [1, 2, 3, 4, 5]
```


Design an LRU cache with O(1) get and put.

SENIOR

Idea: LRU means "Least Recently Used": when the cache is full, throw out the item that hasn't been used for the longest time. Python's `OrderedDict` remembers order, so we move an item to the end whenever it's used, and remove from the front when we run out of space.

Time: O(1) for get and put. **Space:** O(capacity).

```
# Python
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity          # max number of items
        self.items = OrderedDict()       # oldest at the front, newest at the end

    def get(self, key):
        if key not in self.items:
            return None                    # not in cache
        self.items.move_to_end(key)       # mark as most recently used
        return self.items[key]
```

Behavioral & Scenario

Project stories and debugging scenarios. Answer with Situation, Task, Action, Result.

Walk me through a project you built end to end.

JUNIOR

Structure: the problem and users → your stack and why (e.g. Next.js + FastAPI + PostgreSQL on Azure) → one hard technical decision → what you'd do differently → measurable result. Keep it to 2–3 minutes.

Tell me about a production bug you debugged.

MID

Use STAR. Show method: reproduce, read logs/traces (App Insights), form a hypothesis, isolate, fix, add a test and monitoring so it can't recur. Mention communication with the team during the incident.

An API endpoint became slow. How do you investigate?

MID

1. Confirm with metrics: which endpoint, since when, p95/p99.
2. Check recent deploys and traffic changes.
3. Trace the request: DB time vs external calls vs app CPU.
4. Run `EXPLAIN ANALYZE` on slow queries; look for missing indexes or N+1.
5. Fix, add caching if appropriate, and set an alert.

How do you handle disagreement on a technical decision?

MID

Understand their reasons, compare options against agreed criteria (cost, risk, time), prototype or measure when possible, decide and document (ADR), then commit fully even if your option lost.

```
out(key, value) {
```

How do you use AI coding tools and make sure the code is correct?

SENIOR

Use them for scaffolding, tests and exploring APIs; review every line, run tests and type checks, check security-sensitive code (auth, SQL, secrets) by hand, and never paste secrets or customer data into prompts. Interviewers in 2026 often ask this directly.

How do you estimate and break down a feature?

SENIOR

Split into thin vertical slices (UI + API + DB) that each ship value, identify unknowns and spike them first, estimate in ranges, and flag risks early. Example: "password reset" → token table + endpoint, email job, reset page, session revocation, rate limiting, tests.

Why should we hire you for a 4-year full stack role? What is your ownership story?

SENIOR

Show scope growth: a feature you owned from design to production, a decision you made (and its trade-off), how you mentored someone or improved a team process (code review, CI, testing), and a metric that moved (latency, conversion, incidents).

Implement a stack and a queue; a queue using two stacks.

MID

Idea: A **stack** is last-in, first-out (like a pile of plates): use a list with `append` and `pop`. A **queue** is first-in, first-out (like a line at a shop): use `deque`. To build a queue from two stacks, push into an "in" stack; when you need to remove, pour everything into an "out" stack, which flips the order.

Time: $O(1)$ per operation (the two-stack queue is $O(1)$ on average). **Space:** $O(n)$.

```
# Python
from collections import deque

stack = []
stack.append(1)          # push
stack.append(2)
print(stack.pop())       # 2 (last in, first out)

queue = deque()
queue.append(1)          # join the back of the line
queue.append(2)
print(queue.popleft())   # 1 (first in, first out)

class QueueWithTwoStacks:
    def __init__(self):
        self.in_stack = []    # new items go here
        self.out_stack = []   # items leave from here

    def push(self, item):
        self.in_stack.append(item)

    def pop(self):
        if len(self.out_stack) == 0:    # out stack empty?
            while len(self.in_stack) > 0:    # pour everything across
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack.pop()

q = QueueWithTwoStacks()
q.push("a")
q.push("b")
print(q.pop())           # a
```

Follow-up: `list.pop(0)` and JS `array.shift()` are slow ($O(n)$); use `deque` in Python.

Binary tree: max depth, level order (BFS), and inorder traversal.

MID

Idea: Depth: a tree's depth is 1 + the deeper of its two sides. **Level order:** use a queue; take out a node, record it, add its children to the back. **Inorder:** visit left side, then the node, then right side.

Time: $O(n)$ for each, every node is visited once. **Space:** $O(n)$ in the worst case.

```
# Python
from collections import deque

class TreeNode:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right

def max_depth(node):
    if node is None:                # empty tree has depth 0
        return 0
    left_depth = max_depth(node.left)
    right_depth = max_depth(node.right)
    return 1 + max(left_depth, right_depth)

def level_order(root):
    if root is None:
        return []
    result = []
    queue = deque([root])
    while len(queue) > 0:
        level = []                  # values on this level
        level_size = len(queue)     # how many nodes are on this level
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.value)
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
        result.append(level)
    return result

def inorder(node, result):
    if node is None:
        return result
    inorder(node.left, result)      # 1. left side
    result.append(node.value)        # 2. the node itself
    inorder(node.right, result)     # 3. right side
    return result

#      1
#     / \
#    2   3
root = TreeNode(1, TreeNode(2), TreeNode(3))
```

```
print(max_depth(root))      # 2
print(level_order(root))    # [[1], [2, 3]]
print(inorder(root, []))    # [2, 1, 3]
```

Validate a binary search tree.

MID

Idea: In a BST, everything on the left is smaller and everything on the right is bigger, not just the direct children. So pass down an allowed range (low, high). Going left lowers the high limit; going right raises the low limit.

Time: $O(n)$. **Space:** $O(\text{height of tree})$ for recursion.

```
# Python
def is_valid_bst(node, low=float("-inf"), high=float("inf")):
    if node is None:                # empty tree is valid
        return True
    if node.value <= low or node.value >= high:    # outside allowed range
        return False
    left_ok = is_valid_bst(node.left, low, node.value)    # left must be smaller
    right_ok = is_valid_bst(node.right, node.value, high) # right must be bigger
    return left_ok and right_ok

good = TreeNode(5, TreeNode(3), TreeNode(8))
bad = TreeNode(5, TreeNode(3, None, TreeNode(6)), TreeNode(8))    # 6 is on the left of 5
print(is_valid_bst(good))    # True
print(is_valid_bst(bad))    # False
```

Common mistake: only comparing a node with its direct children.

Number of islands (grid DFS/BFS).

MID

Idea: Scan every cell. When you find land ("1"), you've found a new island: count it, then "sink" it by turning it and all connected land into water ("0") so you don't count it again.

Time: $O(\text{rows} \times \text{cols})$. **Space:** $O(\text{rows} \times \text{cols})$ for recursion in the worst case.

```
# Python
def num_islands(grid):
    rows = len(grid)
    cols = len(grid[0])

    def sink(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols:    # outside the grid
            return
        if grid[r][c] != "1":                          # water or already sunk
            return
        grid[r][c] = "0"                                # sink this piece of land
        sink(r + 1, c)                                  # down
        sink(r - 1, c)                                  # up
        sink(r, c + 1)                                  # right
        sink(r, c - 1)                                  # left

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":                      # found new land
                count = count + 1
                sink(r, c)                              # remove the whole island
    return count

grid = [["1", "1", "0"],
        ["0", "1", "0"],
        ["0", "0", "1"]]
print(num_islands(grid))    # 2
```

Climbing stairs / Fibonacci with memoization and bottom-up DP.

MID

Idea: To reach step n you came from step $n-1$ or $n-2$, so $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$, just like Fibonacci. Plain recursion repeats the same work many times; either remember answers (memoization) or build up from the bottom with two variables.

Time: $O(n)$. **Space:** $O(n)$ with memo, $O(1)$ bottom-up.

```
# Python
memo = {}                                # n -> answer we already worked out
def fib(n):
    if n < 2:
        return n
    if n in memo:                          # already calculated? reuse it
        return memo[n]
    memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]

def climb_stairs(n):
    two_back = 1                          # ways to reach step 0
    one_back = 1                          # ways to reach step 1
    for step in range(2, n + 1):
        current = one_back + two_back    # from one step below or two below
        two_back = one_back
        one_back = current
    return one_back

print(fib(10))                            # 55
print(climb_stairs(5))                    # 8

# Shorter way (good to mention in the interview):
# from functools import cache -> put @cache above def fib(n)
```

Coin change: minimum coins to make an amount.

SENIOR

Idea: Build a table where `best[a]` is the fewest coins to make amount `a`. Start with `best[0] = 0`. For every amount, try each coin: if using that coin plus the best way to make the rest is fewer coins, keep it.

Time: $O(\text{amount} \times \text{number of coins})$. **Space:** $O(\text{amount})$.

```
# Python
def coin_change(coins, amount):
    impossible = amount + 1          # more coins than could ever be needed
    best = [impossible] * (amount + 1) # best[a] = fewest coins for amount a
    best[0] = 0                      # zero coins make amount 0
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:             # coin fits in this amount
                with_this_coin = best[a - coin] + 1
                if with_this_coin < best[a]:
                    best[a] = with_this_coin
    if best[amount] == impossible:
        return -1                    # cannot make this amount
    return best[amount]

print(coin_change([1, 3, 4], 6))    # 2 (3 + 3)
print(coin_change([2], 3))          # -1
```

A greedy approach (always take the biggest coin) fails here: $4 + 1 + 1 = 3$ coins.

Kth largest element in an array.

SENIOR

Idea: Simple way: sort from biggest to smallest and pick position $k-1$. Faster way: keep a small "min-heap" of the k biggest numbers seen so far; its smallest item is the answer.

Time: $O(n \log n)$ with sorting, $O(n \log k)$ with a heap. **Space:** $O(n)$ or $O(k)$.

```
# Python
def kth_largest_simple(nums, k):
    sorted_desc = sorted(nums, reverse=True)  # biggest first
    return sorted_desc[k - 1]

import heapq
def kth_largest(nums, k):
    heap = []                                # holds the k biggest numbers so far
    for num in nums:
        heapq.heappush(heap, num)            # add the number
        if len(heap) > k:
            heapq.heappop(heap)              # drop the smallest, keep only k
    return heap[0]                           # smallest of the k biggest

print(kth_largest_simple([3, 2, 1, 5, 6, 4], 2))  # 5
print(kth_largest([3, 2, 1, 5, 6, 4], 2))        # 5

# Shorter way (good to mention in the interview):
# heapq.nlargest(k, nums)[-1]
```

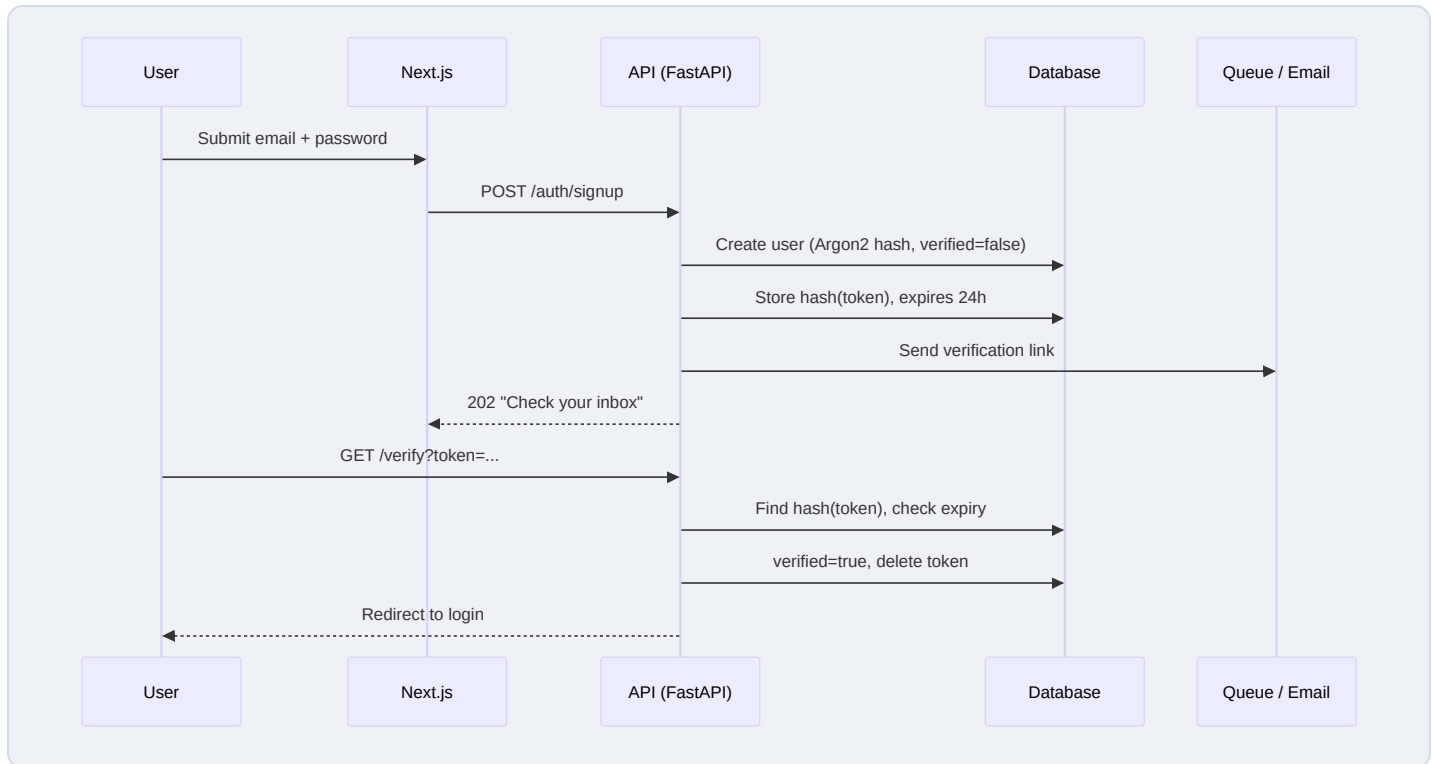
Subarray sum equals K (count).

SENIOR

Architecture diagrams

Idea: Keep a running total. If at some point the running total is S, and earlier it was S - k, then the numbers in between add up to k. So store how many times each running total has appeared in a

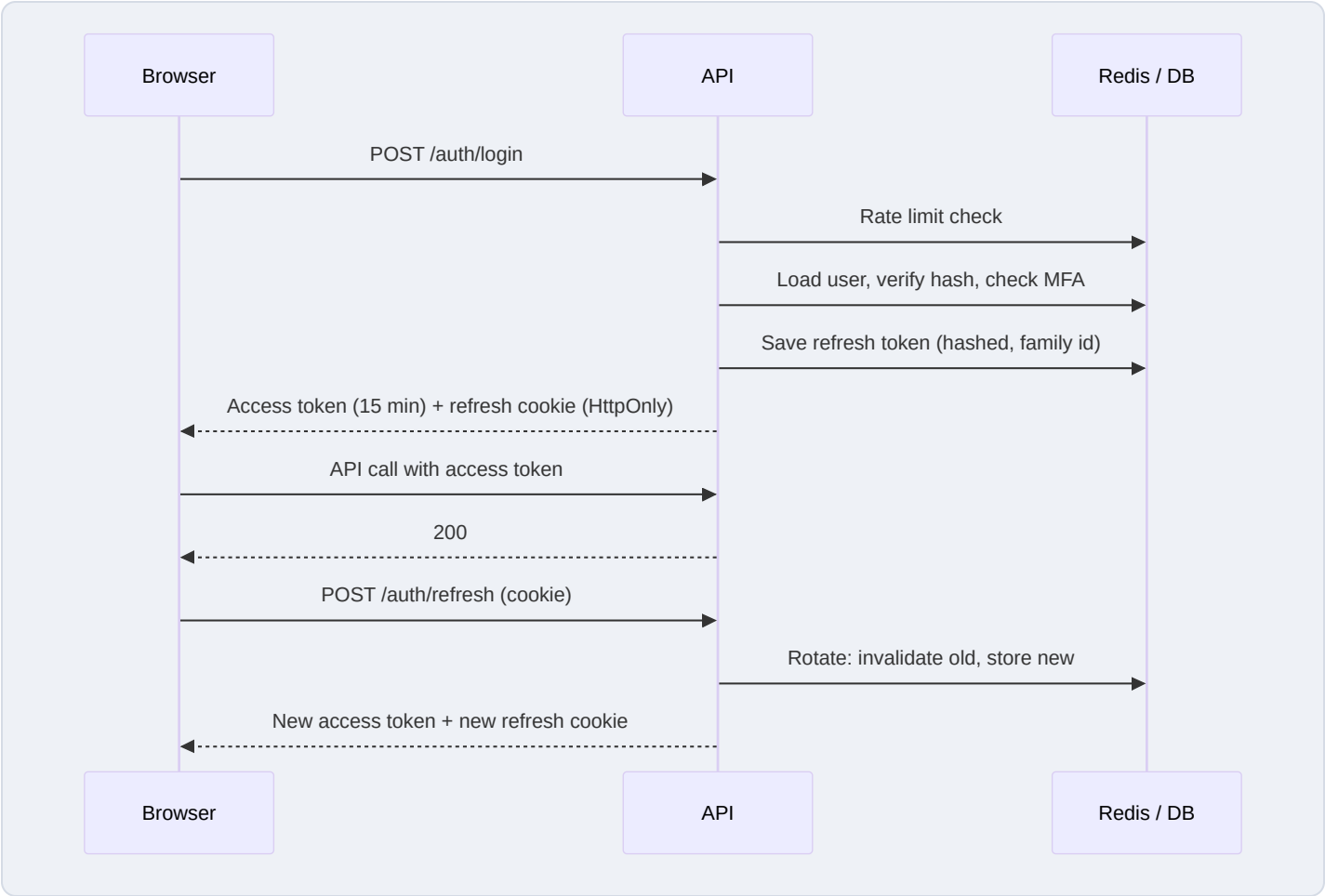
Signup and email verification



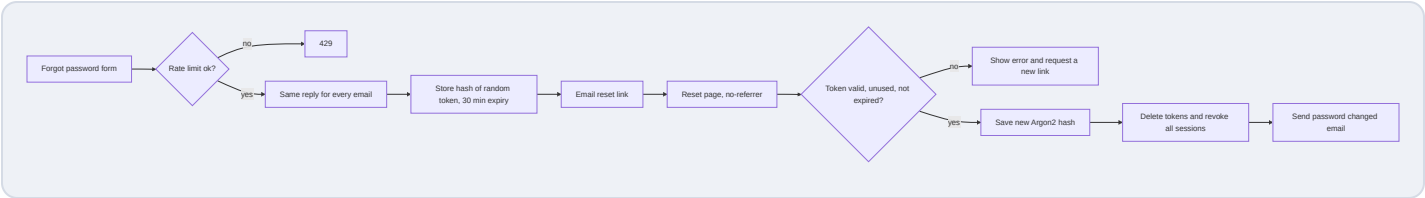
- $O(1)$: hash lookup, array index, stack push/pop.
- $O(\log n)$: binary search, heap push/pop, balanced BST.
- $O(n)$: single loop, two pointers, sliding window.
- $O(n \log n)$: good sorts (Python's Timsort, JS's sort).
- $O(n^2)$: nested loops; say how you'd improve it.

Interview habit: state brute force first, then optimise, then give time and space complexity and test edge cases (empty, one element, duplicates, negatives).

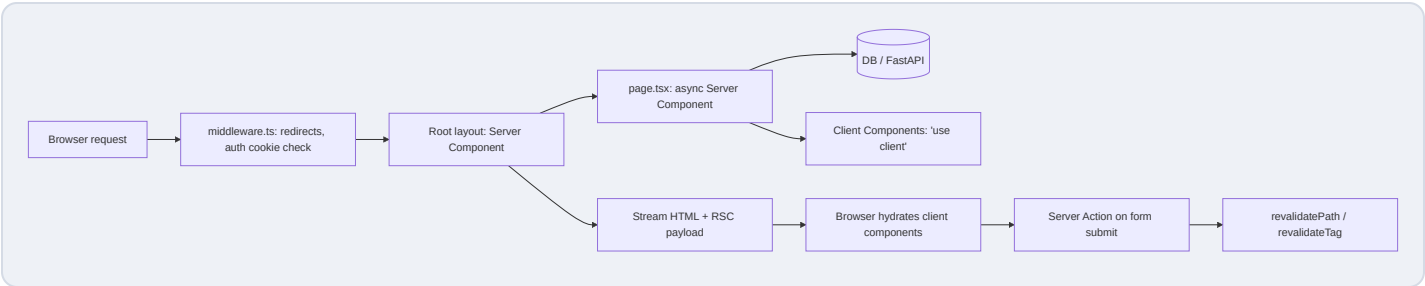
Login with access and refresh tokens



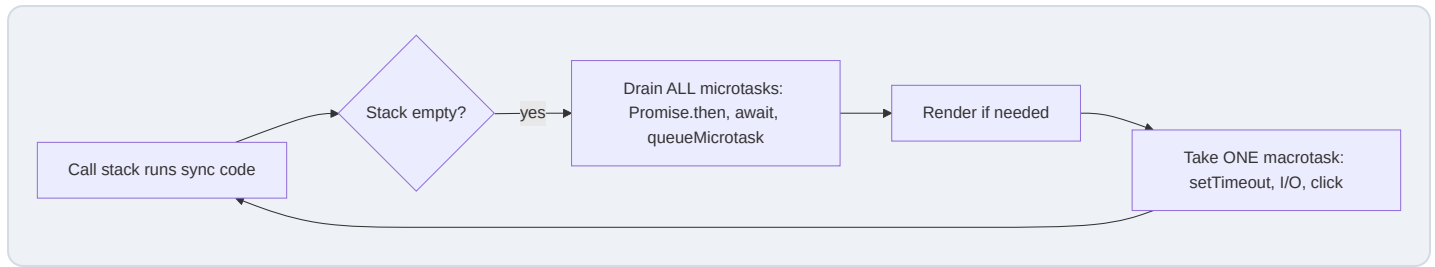
Forgot and reset password



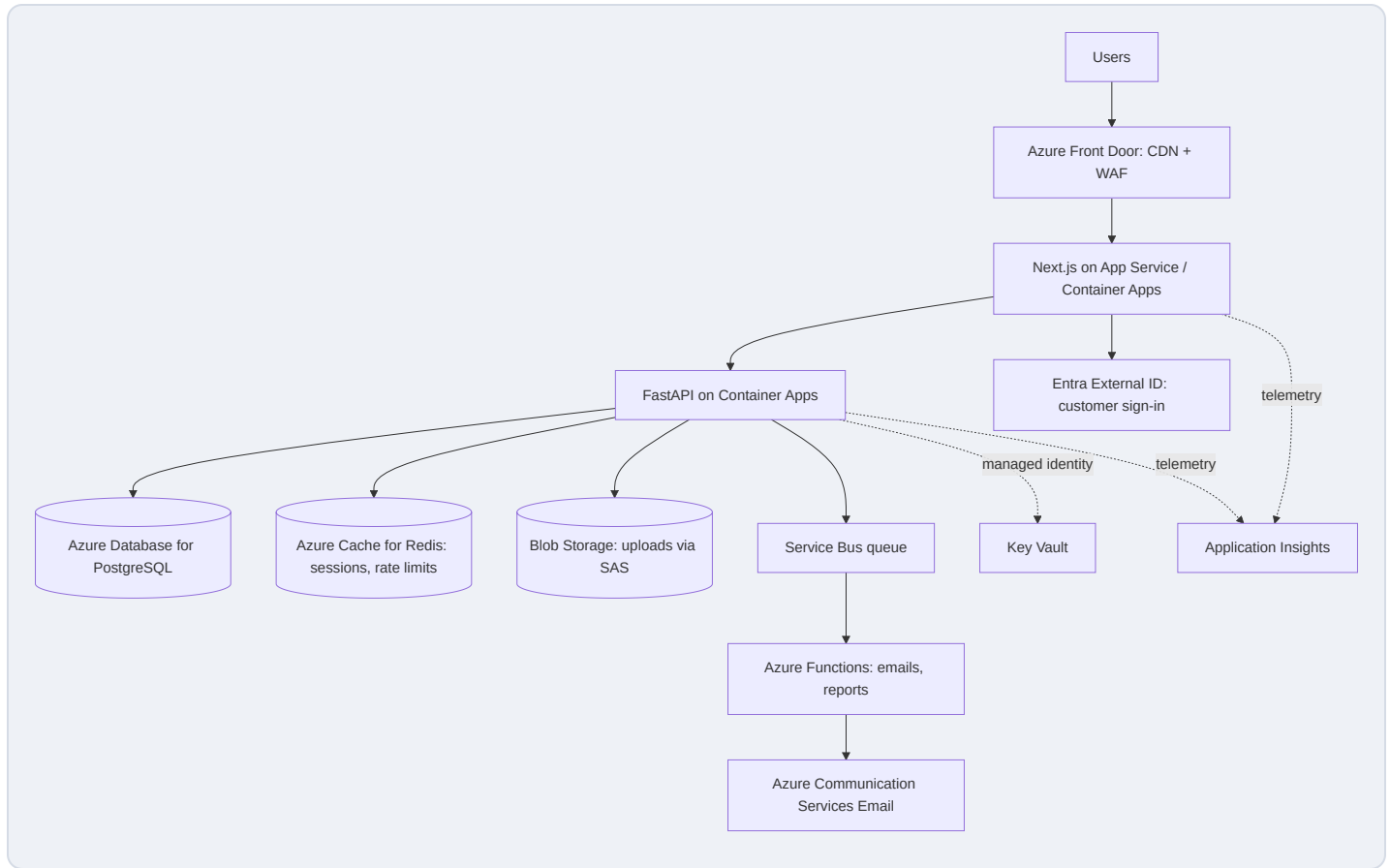
Next.js App Router request flow



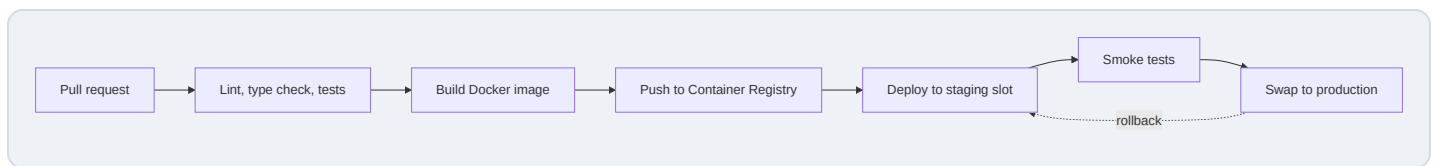
JavaScript event loop



Next.js + Python on Azure



CI/CD to Azure



Sources

Questions were cross-checked against these published guides (September 2026). Answers are written fresh and kept short; go deeper on anything you get asked twice.

- [Interview Kickstart: Full Stack Q&A 2026](#)
- [KORE1: Full Stack Developer Interview Questions 2026](#)
- [Nucamp: Top 30 Full Stack Questions 2026](#)
- [Edureka: 40+ Full Stack Questions](#)
- [TestMu: 100+ Full Stack Questions](#)
- [GreatFrontEnd: Next.js Questions \(Mid–Senior\)](#)
- [GreatFrontEnd: Next.js for Freshers](#)
- [InterviewBit: Next.js Questions](#)
- [DataCamp: 33 Azure Questions](#)
- [KodeKloud: Azure Questions 2026](#)
- [FullStack.Cafe: Azure Developer Questions](#)
- [Tenant365Admin: Microsoft Entra Questions](#)
- [GeeksforGeeks: Designing an Authentication System](#)
- [Chirag Hasija: Design a Login System](#)
- [freeCodeCamp: JWT and Refresh Tokens](#)
- [Interview Coder: 35 JWT Questions](#)
- [DEV: Sessions vs JWT vs Passkeys](#)